

# ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ

ΑΝΑΠΤΥΞΗ HYBRID CLOUD ΑΡΧΙΤΕΚΤΟΝΙΚΗΣ: L2 VPN BRIDGING,  
STORAGE TIERING ΚΑΙ API AUTOMATION

ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

Μάθημα: Υποδομές Cloud  
ΠΜΣ «Προηγμένες Τεχνολογίες Πληροφορικής και Εφαρμογές»

Ακαδημαϊκά μέλη μαθήματος:  
Καθηγητής κ. Θωμάς Καμαλάκης  
Καθηγητής κ. Αλέξανδρος Δημόπουλος



**ΧΑΡΟΚΟΠΕΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ**  
**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΜΑΤΙΚΗΣ**

Τμήμα Πληροφορικής και Τηλεματικής  
Χαροκόπειο Πανεπιστήμιο Αθηνών  
Φεβρουάριος, 2026



## ΔΡΟΣΟΣ ΚΑΤΣΙΜΠΡΑΣ

### Περίληψη

Η παρούσα εργασία εστιάζει στον σχεδιασμό και την υλοποίηση μιας πρότυπης αρχιτεκτονικής Υβριδικού Νέφους (Hybrid Cloud), με στόχο την ασφαλή και διαφανή ενοποίηση ετερογενών υπολογιστικών περιβαλλόντων. Η υποδομή αναπτύσσεται σε τρεις διακριτούς κόμβους: έναν δημόσιο πάροχο (Azure), ένα ιδιωτικό νέφος (Proxmox Lab) και έναν τοπικό κόμβο διαχείρισης.

Κεντρικός άξονας της υλοποίησης αποτελεί η εγκαθίδρυση ενός Εικονικού Ιδιωτικού Δικτύου (VPN) βασισμένου στο πρωτόκολλο OpenVPN σε διάταξη γέφυρας (TAP mode - Layer 2). Μέσω αυτής της τεχνικής, επιτυγχάνεται η δημιουργία ενός ενιαίου Broadcast Domain που επιτρέπει την απρόσκοπτη επικοινωνία μεταξύ των απομακρυσμένων κόμβων, με τον τοπικό κόμβο να αναλαμβάνει ρόλο κεντρικού διαχειριστή δικτύου και απόδοσης διευθύνσεων (DHCP Server).

Παράλληλα, μελετάται η βελτιστοποίηση της αποθήκευσης δεδομένων μέσω τεχνικών Software-Defined Storage. Συγκεκριμένα, υλοποιείται μια σύνθετη διάταξη εικονικών δίσκων που συνδυάζει επίπεδα RAID 5 και RAID 1 υπό τη διαχείριση του Logical Volume Manager (LVM), εξασφαλίζοντας ανοχή σε σφάλματα (fault tolerance) και επεκτασιμότητα. Τέλος, η εργασία ενσωματώνει πρακτικές αυτοματισμού (Infrastructure Automation), αξιοποιώντας τη γλώσσα Python και το API του Proxmox για τη συλλογή και καταγραφή μετρικών λειτουργίας σε πραγματικό χρόνο.

Η προσέγγιση αυτή προσομοιώνει ρεαλιστικά σενάρια παραγωγής, αναδεικνύοντας τις τεχνικές προκλήσεις και τις βέλτιστες πρακτικές στη σύγχρονη μηχανική δικτύων και υποδομών.

**Λέξεις Κλειδιά:** Hybrid Cloud Architecture, OpenVPN Layer 2 Bridging, Software-Defined Storage (RAID/LVM), Network Virtualization, Infrastructure Automation, Proxmox API, Cross-Provider Connectivity.



# Περιεχόμενα

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| Περίληψη                                                          | ii        |
| Κατάλογος Σχημάτων                                                | viii      |
| Κατάλογος Πινάκων                                                 | xi        |
| Λίστα Συντομογραφιών                                              | xiii      |
| <b>1 Κεφάλαιο 1: Εισαγωγή και Σκοπιμότητα</b>                     | <b>1</b>  |
| 1.1 Αντικείμενο της Εργασίας:                                     | 1         |
| 1.2 Σκοπιμότητα και Σενάριο Χρήσης (Business Scenario)            | 1         |
| 1.3 Στόχοι του Project                                            | 2         |
| 1.4 Μεθοδολογία υλοποίησης                                        | 3         |
| <b>2 Κεφάλαιο 2: Αρχιτεκτονικός σχεδιασμός συστήματος</b>         | <b>4</b>  |
| 2.1 Τοπολογία Δικτύου (Network Topology)                          | 4         |
| 2.2 Στρατηγική Διασύνδεσης (Connectivity Strategy)                | 5         |
| 2.3 Σχέδιο Διευθυνσιοδότησης (IP Addressing Plan)                 | 6         |
| 2.4 Αρχιτεκτονική Ασφάλειας (Security Architecture)               | 7         |
| <b>3 Κεφάλαιο 3: Υλοποίηση Υποδομής και Storage</b>               | <b>8</b>  |
| 3.1 Προετοιμασία Κόμβων (Node Provisioning)                       | 8         |
| 3.2 Node 0 (Cloud Environment)                                    | 8         |
| 3.2.1 Δημιουργία VM στο Azure και Ρύθμιση Firewall (NSG)          | 8         |
| 3.2.2 Απομακρυσμένη Διαχείριση (ssh)                              | 11        |
| 3.2.3 Προετοιμασία Ubuntu server και εγκατάσταση OpenVPN          | 12        |
| 3.3 Node 1 (Lab Private Cloud)                                    | 14        |
| 3.3.1 Εγκατάσταση και Παραμετροποίηση του Hypervisor (Node 1)     | 15        |
| 3.3.2 Προετοιμασία Ubuntu server                                  | 16        |
| 3.3.3 Απομακρυσμένη διαχείριση (ssh) και ολοκλήρωση προετοιμασίας | 17        |
| 3.4 Node 2 (Local Storage - VirtualBox)                           | 18        |
| 3.4.1 Προδιαγραφές και Ενσωμάτωση Τοπικού Κόμβου                  | 18        |
| 3.4.2 Υλοποίηση αρχιτεκτονικής αποθήκευσης (RAID και LVM)         | 18        |
| 3.4.3 Δημιουργία Συστοιχιών RAID (md0 και md1)                    | 20        |
| 3.4.4 Υλοποίηση Logical Volume Management (LVM)                   | 21        |
| <b>4 Κεφάλαιο 4: Ενοποίηση Δικτύου και VPN</b>                    | <b>24</b> |
| 4.1 Αρχιτεκτονική PKI και ο ρόλος της CA                          | 24        |

|          |                                                                                  |           |
|----------|----------------------------------------------------------------------------------|-----------|
| 4.2      | Node 0: Υλοποίηση VPN Server και διαμόρφωση virtual bridge . . . . .             | 25        |
| 4.2.1    | Ρύθμιση της υποδομής PKI και δημιουργία της CA . . . . .                         | 26        |
| 4.2.2    | Έκδοση πιστοποιητικών server . . . . .                                           | 27        |
| 4.2.3    | Παραμετροποίηση και ρύθμιση της υπηρεσίας OpenVPN . . . . .                      | 29        |
| 4.2.4    | Έκδοση πιστοποιητικών clients και μεταφορά αρχείων (Node 1 και Node 2) . . . . . | 31        |
| 4.3      | Node 1: Παραμετροποίηση VPN Client (Proxmox) . . . . .                           | 33        |
| 4.4      | Node 2: Παραμετροποίηση VPN Client και DHCP Server . . . . .                     | 34        |
| 4.4.1    | Επαλήθευση και συμπληρωματική εγκατάσταση υπηρεσιών . . . . .                    | 35        |
| 4.4.2    | Παραμετροποίηση VPN Client (Node 2) . . . . .                                    | 35        |
| 4.4.3    | Παραμετροποίηση DHCP server . . . . .                                            | 36        |
| 4.4.4    | Καθορισμός Listening Interface . . . . .                                         | 36        |
| 4.5      | Αυτοματοποίηση και Ενεργοποίηση Υποδομής (Scripts) . . . . .                     | 37        |
| 4.5.1    | Node 0: Server-Side Bridge αυτοματισμός . . . . .                                | 38        |
| 4.5.2    | Node 1: Proxmox Bridge &VPN Connectivity . . . . .                               | 38        |
| 4.5.3    | Node 2: Gateway &DHCP Deployment . . . . .                                       | 40        |
| 4.6      | Επαλήθευση Εγκαθίδρυσης Tunnel (Logs) . . . . .                                  | 41        |
| 4.7      | Συνολική Αρχιτεκτονική Διασύνδεσης . . . . .                                     | 41        |
| 4.7.1    | Επαλήθευση Λειτουργίας (Verification) . . . . .                                  | 42        |
| <b>5</b> | <b>Κεφάλαιο 5: Αυτοματισμός και Monitoring</b>                                   | <b>45</b> |
| 5.1      | Ενεργοποίηση Guest-Agent στον Node 1 . . . . .                                   | 45        |
| 5.2      | Ανάλυση του Proxmox API και Endpoints . . . . .                                  | 46        |
| 5.3      | Ανάπτυξη του Python Automation Script . . . . .                                  | 46        |
| 5.3.1    | Αυθεντικοποίηση και Ασφάλεια API . . . . .                                       | 47        |
| 5.3.2    | Αρχιτεκτονική και Λογική Κώδικα . . . . .                                        | 47        |
| 5.4      | Προγραμματισμός Εργασιών (Cron Job) . . . . .                                    | 48        |
| 5.5      | Επαλήθευση και Εξαγωγή Δεδομένων . . . . .                                       | 50        |
| 5.5.1    | Δομή και Περιεχόμενο Αρχείου CSV . . . . .                                       | 50        |
| 5.5.2    | Επιβεβαίωση Περιοδικότητας . . . . .                                             | 50        |
| 5.6      | Οπτικοποίηση Δεδομένων Τηλεμετρίας . . . . .                                     | 51        |
| <b>6</b> | <b>Κεφάλαιο 6: Προβλήματα &amp;Αντιμετώπιση</b>                                  | <b>52</b> |
| 6.1      | Ρυθμίσεις Ασφαλείας και Firewalls (Azure &Local) . . . . .                       | 52        |
| 6.2      | Πολυπλοκότητα Διαχείρισης Πιστοποιητικών (PKI) . . . . .                         | 52        |
| 6.3      | Σειρά Εκκίνησης και Συγχρονισμός Interfaces . . . . .                            | 52        |
| 6.4      | Διαχείριση του Layer 2 Bridging . . . . .                                        | 52        |
|          | <b>Συμπεράσματα</b>                                                              | <b>52</b> |

|                                       |    |
|---------------------------------------|----|
| Παράρτημα: Αποθετήριο Κώδικα (GitHub) | 54 |
| Βιβλιογραφικές Αναφορές               | 56 |





## Κατάλογος Σχημάτων

|      |                                                                                                                              |    |
|------|------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Τοπολογία Δικτύου και Διασύνδεση Κόμβων . . . . .                                                                            | 4  |
| 2.2  | Διάγραμμα Ενθυλάκωσης Πακέτων (Packet Encapsulation) . . . . .                                                               | 5  |
| 2.3  | Αρχιτεκτονική OpenVPN (Onion Model) . . . . .                                                                                | 7  |
| 3.1  | Επιτυχής δημιουργία του virtual machine στο Azure . . . . .                                                                  | 8  |
| 3.2  | Βασικές ρυθμίσεις στο virtual machine (Azure) . . . . .                                                                      | 9  |
| 3.3  | Βασικές ρυθμίσεις του δικτύου στο virtual machine (Azure) . . . . .                                                          | 10 |
| 3.4  | Δημιουργία inbound security rule για την θύρα 1194 . . . . .                                                                 | 11 |
| 3.5  | Είσοδος στο Azure VM με χρήση του ssh πρωτοκόλλου . . . . .                                                                  | 11 |
| 3.6  | Εκτέλεση εντολής για το upgrade του συστήματος (Azure) . . . . .                                                             | 12 |
| 3.7  | Εγκατάσταση του OpenVPN και δικτυακών εργαλείων (Azure) . . . . .                                                            | 13 |
| 3.8  | Επιτυχής έλεγχος του vm στο Azure για OpenVPN και δικτύου . . . . .                                                          | 13 |
| 3.9  | Nested Virtualization αρχιτεκτονική Node 1 - Proxmox . . . . .                                                               | 14 |
| 3.10 | Proxmox - VM 904 στο περιβάλλον του εργαστηρίου (Layer 1) . . . . .                                                          | 15 |
| 3.11 | Ο Nested Hypervisor (Layer 2) σε πλήρη λειτουργία μετά την εγκατάσταση. . . . .                                              | 16 |
| 3.12 | Ρύθμιση υλικού (1 vCPU / 1.5GB RAM) για τον Node 1 . . . . .                                                                 | 16 |
| 3.13 | Ολοκλήρωση της εγκατάστασης του Ubuntu Server και login στο Node 1 . . . . .                                                 | 17 |
| 3.14 | Επιτυχής απομακρυσμένη σύνδεση στο vm του Node 1 μέσω ssh . . . . .                                                          | 17 |
| 3.15 | Ρύθμιση των 5 εικονικών δίσκων στον ελεγκτή SATA του Node 2 . . . . .                                                        | 19 |
| 3.16 | Λίστα αναγνωρισμένων block devices στον Node 2 . . . . .                                                                     | 19 |
| 3.17 | Λογικό διάγραμμα ιεραρχίας RAID και LVM στον Node 2 . . . . .                                                                | 20 |
| 3.18 | Εκτέλεση εντολών δημιουργίας του RAID-5 (md0) . . . . .                                                                      | 20 |
| 3.19 | Εκτέλεση εντολών δημιουργίας του RAID-1 (md1) . . . . .                                                                      | 21 |
| 3.20 | Επιβεβαίωση της κατάστασης 'active' για τις συστοιχίες RAID-5 και RAID-1 . . . . .                                           | 21 |
| 3.21 | Υλοποίηση LVM, διαμόρφωση και προσάρτηση . . . . .                                                                           | 22 |
| 3.22 | Εντολή εγγραφής στο /etc/fstab για την αυτόματη προσάρτηση του lv_storage . . . . .                                          | 22 |
| 4.1  | Ιεραρχία εμπιστοσύνης και χάρτης διανομής κρυπτογραφικών αρχείων. . . . .                                                    | 24 |
| 4.2  | Φάσεις αμφίδρομης ταυτοποίησης και εγκαθίδρυσης ασφαλούς συνόδου. . . . .                                                    | 25 |
| 4.3  | Δημιουργία του dir openvpn-ca και προετοιμασία του περιβάλλοντος Easy-RSA στον Node 0 . . . . .                              | 26 |
| 4.4  | Δημιουργία της δομής του PKI . . . . .                                                                                       | 26 |
| 4.5  | Επιτυχής δημιουργία του πιστοποιητικού της Αρχής Πιστοποιητικών (CA) . . . . .                                               | 27 |
| 4.6  | Επιτυχής δημιουργία των αρχείων Private-Key και Public-Certificate-Request για τον Node 0 . . . . .                          | 28 |
| 4.7  | Επιτυχής ψηφιακή υπογραφή και έκδοση του πιστοποιητικού εξυπηρετητή. . . . .                                                 | 28 |
| 4.8  | Παραγωγή παραμέτρων Diffie-Hellman και κλειδιού TLS-Auth στον Node 0. . . . .                                                | 29 |
| 4.9  | Παραγωγή του κλειδιού ta.key για την ενίσχυση της ακεραιότητας και τη δημιουργία HMAC firewall στον OpenVPN server . . . . . | 29 |

|      |                                                                                                                                                                    |    |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.10 | Μεταφορά των κρυπτογραφικών κλειδιών στον κατάλογο λειτουργίας <code>/etc/openvpn/</code> και επαλήθευση της ακεραιότητας του καταλόγου. . . . .                   | 30 |
| 4.11 | Το αρχείο παραμέτρων <code>server.conf</code> . . . . .                                                                                                            | 30 |
| 4.12 | Παραγωγή και υπογραφή του κρυπτογραφικού πιστοποιητικού για τον Node 2 από την κεντρική CA του Node 0. . . . .                                                     | 31 |
| 4.13 | Node 1/Node2: Επαλήθευση αποθήκευσης των παραχθέντων αρχείων στον κατάλληλο κατάλογο. . . . .                                                                      | 32 |
| 4.14 | Διάγραμμα της δομής PKI και της ιεραρχίας των κρυπτογραφικών αρχείων στον Node 0. . . . .                                                                          | 33 |
| 4.15 | Το αρχείο <code>proxmox_vpn.conf</code> . . . . .                                                                                                                  | 34 |
| 4.16 | Επιτυχής εγκατάσταση και επαλήθευση των απαραίτητων πακέτων στον Node 2. . . . .                                                                                   | 35 |
| 4.17 | Παραμετροποίηση του αρχείου <code>client.conf</code> . . . . .                                                                                                     | 35 |
| 4.18 | Παραμετροποίηση του αρχείου <code>dhcpd.conf</code> για το δίκτυο 10.8.0.0/24. . . .                                                                               | 36 |
| 4.19 | Ορισμός της γέφυρας <code>br0</code> ως <code>INTERFACESv4</code> . . . . .                                                                                        | 37 |
| 4.20 | Επιτυχής εκτέλεση του script αυτοματισμού στον Node 0. . . . .                                                                                                     | 38 |
| 4.21 | Εκτέλεση του script <code>reset_proxmox.sh</code> και επιτυχής λήψη IP μέσω του tunnel. . . . .                                                                    | 39 |
| 4.22 | Επιτυχής εκτέλεση του script <code>reset_laptop.sh</code> στον Node 2. . . . .                                                                                     | 40 |
| 4.23 | Logs συστήματος στον Node 2: Επιτυχής ολοκλήρωση σύνδεσης με το μήνυμα "Initialization Sequence Completed". . . . .                                                | 41 |
| 4.24 | Ολοκληρωμένο διάγραμμα διασύνδεσης . . . . .                                                                                                                       | 42 |
| 4.25 | Logs του DHCP Server στον Node 2: Απόδοση IP 10.8.0.16 στον Node 1 και 10.8.0.10 στον Node 0. . . . .                                                              | 42 |
| 4.26 | Επαλήθευση συνδεσιμότητας μέσω Ping από τον Node 1 προς τον Node 2. . . .                                                                                          | 43 |
| 4.27 | Έλεγχος διαδρομής (Traceroute) προς τους απομακρυσμένους κόμβους. . . .                                                                                            | 43 |
| 4.28 | Πίνακας ARP του Node 2: Οι εγγραφές για τους Node 0 (10.8.0.10) και Node 1 (10.8.0.16) εμφανίζονται με τις MAC addresses τους στη διεπαφή <code>br0</code> . . . . | 44 |
| 5.1  | Εγκατάσταση και ενεργοποίηση του <code>qemu-guest-agent</code> στον Node 1 . . . . .                                                                               | 45 |
| 5.2  | Ενεργοποίηση του <code>qemu-guest-agent</code> στον Node 1 . . . . .                                                                                               | 46 |
| 5.3  | Δημιουργία του token . . . . .                                                                                                                                     | 47 |
| 5.4  | Αρχιτεκτονική Python script . . . . .                                                                                                                              | 47 |
| 5.5  | Παραμετροποίηση του Cron για την ωριαία συλλογή τηλεμετρίας . . . . .                                                                                              | 49 |
| 5.6  | Επιβεβαίωση εκτέλεσης της διεργασίας από το αρχείο καταγραφής συστήματος. . .                                                                                      | 49 |
| 5.7  | Αρχείο <code>vm_metrics</code> . . . . .                                                                                                                           | 50 |
| 5.8  | Επαλήθευση εγγραφών στο <code>vm_metrics.csv</code> με χρήση της εντολής <code>tail</code> . . . .                                                                 | 50 |
| 5.9  | Εικόνα 5.9: Γράφημα διακύμανσης CPU και RAM Utilization του VM 100. . . .                                                                                          | 51 |



## Κατάλογος Πινάκων

|   |                                                                     |    |
|---|---------------------------------------------------------------------|----|
| 1 | Εργαλεία και Τεχνολογίες Υλοποίησης Υβριδικής Υποδομής . . . . .    | 3  |
| 2 | Σχέδιο Διευθυνσιοδότησης και Ρόλοι VPN Δικτύου . . . . .            | 6  |
| 3 | Κατανομή κόμβων υποδομής και τεχνικές προδιαγραφές . . . . .        | 8  |
| 4 | Βασικές Εντολές Προετοιμασίας και Εγκατάστασης Λογισμικού . . . . . | 12 |
| 5 | Σύγκριση και Ρόλος Τεχνολογιών Storage στον Node 2 . . . . .        | 23 |
| 6 | Ανάλυση και Δομή του Αυτοματοποιημένου Cron Job . . . . .           | 48 |



## Λίστα Συντομογραφιών

|                |                                                 |
|----------------|-------------------------------------------------|
| <b>ACL</b>     | Access Control List                             |
| <b>API</b>     | Application Programming Interface               |
| <b>CA</b>      | Certification Authority                         |
| <b>CIDR</b>    | Classless Inter-Domain Routing                  |
| <b>CPU</b>     | Central Processing Unit                         |
| <b>CSP</b>     | Cloud Service Provider                          |
| <b>CSV</b>     | Comma-Separated Values                          |
| <b>DHCP</b>    | Dynamic Host Configuration Protocol             |
| <b>HTTP/S</b>  | HyperText Transfer Protocol (Secure)            |
| <b>JSON</b>    | JavaScript Object Notation                      |
| <b>L2</b>      | Layer 2 (Data Link Layer)                       |
| <b>LAN</b>     | Local Area Network                              |
| <b>LV</b>      | Logical Volume                                  |
| <b>LVM</b>     | Logical Volume Manager                          |
| <b>MAC</b>     | Media Access Control                            |
| <b>NSG</b>     | Network Security Group                          |
| <b>PVE</b>     | Proxmox Virtual Environment                     |
| <b>QEMU</b>    | Quick Emulator                                  |
| <b>RAID</b>    | Redundant Array of Independent Disks            |
| <b>RAM</b>     | Random Access Memory                            |
| <b>REST</b>    | Representational State Transfer                 |
| <b>SSH</b>     | Secure Shell                                    |
| <b>TAP</b>     | Network TAP (Virtual Ethernet)                  |
| <b>TLS/SSL</b> | Transport Layer Security / Secure Sockets Layer |
| <b>UDP</b>     | User Datagram Protocol                          |
| <b>VG</b>      | Volume Group                                    |
| <b>VM</b>      | Virtual Machine                                 |
| <b>VPN</b>     | Virtual Private Network                         |



# 1 Κεφάλαιο 1: Εισαγωγή και Σκοπιμότητα

Το παρόν κεφάλαιο οριοθετεί το θεωρητικό και επιχειρησιακό πλαίσιο της εργασίας, εστιάζοντας στην ανάγκη σύγκλισης ετερογενών υπολογιστικών περιβαλλόντων. Αρχικά, προσδιορίζεται το τεχνικό αντικείμενο της μελέτης, το οποίο αφορά τη δημιουργία μιας ενιαίας, ασφαλούς υποδομής Υβριδικού Νέφους (Hybrid Cloud). Στη συνέχεια, αναλύεται το σενάριο χρήσης που υπαγορεύει τις αρχιτεκτονικές επιλογές, αναδεικνύοντας την πρακτική αξία της λύσης σε πραγματικές συνθήκες, ενώ τέλος, καταγράφονται οι στόχοι και η μεθοδολογία που ακολουθήθηκε για την επίτευξη της δικτυακής ενοποίησης και της διαχείρισης δεδομένων.

## 1.1 Αντικείμενο της Εργασίας:

Η παρούσα εργασία πραγματεύεται τον σχεδιασμό, την υλοποίηση και την αυτοματοποιημένη διαχείριση μιας υβριδικής υποδομής νέφους (**Hybrid Cloud Infrastructure**). Στο σύγχρονο επιχειρηματικό περιβάλλον, οι οργανισμοί καλούνται συχνά να γεφυρώσουν το χάσμα μεταξύ των παλαιότερων τοπικών συστημάτων (Legacy On-Premise) και των σύγχρονων υπηρεσιών δημόσιου νέφους (Public Cloud).

Ειδικότερα, το αντικείμενο της μελέτης εστιάζει στη δημιουργία ενός ενιαίου, ασφαλούς δικτύου επιπέδου ζεύξης (Layer 2 Bridging) που ενοποιεί τρεις ετερογενείς κόμβους:

- Έναν δημόσιο διακομιστή (Public Cloud - Azure).
- Έναν τοπικό εξυπηρετητή εργαστηρίου (Private Cloud - Proxmox).
- Έναν κεντρικό κόμβο διαχείρισης (Edge/Management Node).

Βασικός στόχος είναι η επίτευξη αδιάλειπτης επικοινωνίας μεταξύ των κόμβων και η κεντρική εποπτεία των πόρων (Resource Monitoring) σε πραγματικό χρόνο.

## 1.2 Σκοπιμότητα και Σενάριο Χρήσης (Business Scenario)

Σε αυτή την ενότητα, κρίνεται σκόπιμο να αναλυθεί γιατί η προτεινόμενη αρχιτεκτονική αποτελεί ένα χαρακτηριστικό πρότυπο Υβριδικής Ενοποίησης Συστημάτων (Hybrid System Integration). Στο σύγχρονο επιχειρησιακό τοπίο, υφίσταται έντονη η ανάγκη σύζευξης ετερογενών περιβαλλόντων: αφενός παλαιότερων εφαρμογών (Legacy Applications) που διατηρούνται σε ιδιωτικά κέντρα δεδομένων λόγω τεχνικών περιορισμών ή κανονιστικών πλαισίων, και αφετέρου σύγχρονων, κλιμακωσίμων υπηρεσιών που εκτελούνται στο δημόσιο νέφος.

Υπό αυτό το πρίσμα, το σενάριο χρήσης της εργασίας δεν αφορά απλώς τη διασύνδεση δύο σημείων, αλλά την ασφαλή και διαφανή επέκταση του τοπικού δικτύου (LAN Extension) προς το Cloud. Μέσω αυτής της προσέγγισης, δημιουργείται ένα ενιαίο λογικό δίκτυο που υπερβαίνει τους φυσικούς γεωγραφικούς διαχωρισμούς, εξυπηρετώντας τρεις κρίσιμες λειτουργικές απαιτήσεις:



- **Ενοποίηση Επιπέδου Ζεύξης (Layer 2 Connectivity):** Πολλές εφαρμογές παλαιού τύπου (Legacy ERP/Databases) απαιτούν την ύπαρξη των κόμβων στο ίδιο υποδίκτυο (subnet) για να λειτουργήσουν, βασιζόμενες σε πρωτοκόλλα Broadcast/-Multicast που δεν υποστηρίζονται από τις τυπικές συνδέσεις Layer 3 VPN. Η λύση μας γεφυρώνει αυτό το χάσμα, επιτρέποντας στον Cloud κόμβο (Node 0) και τον Lab κόμβο (Node 1) να επικοινωνούν σαν να συνδέονται στο ίδιο φυσικό switch.
- **Κυριαρχία Δεδομένων:** Παρόλο που η επεξεργασία μπορεί να γίνεται στο Cloud, η πολιτική ασφαλείας συχνά επιβάλλει ότι τα ευαίσθητα δεδομένα καταγραφής (Logs και Audit Trails) πρέπει να αποθηκεύονται αποκλειστικά σε ελεγχόμενη τοπική υποδομή. Ο κόμβος διαχείρισης (Node 2) αναλαμβάνει αυτόν τον ρόλο, συγκεντρώνοντας τα logs σε κρυπτογραφημένους και πλεονασματικούς δίσκους (RAID).
- **Κεντρική διαχείριση πόρων:** Η ανάγκη για ενιαία εποπτεία (monitoring) ετερογενών συστημάτων επιβάλλει τη χρήση ενός κεντρικού κόμβου (Node 2) ο οποίος, μέσω αυτοματισμών, ελέγχει την υγεία των απομακρυσμένων μηχανημάτων, ανεξάρτητα από το που αυτά φιλοξενούνται φυσικά.

### 1.3 Στόχοι του Project

Σε αυτή την υποενότητα, είναι αναγκαίο να αναφερθεί ο κύριος στόχος της εργασίας, ο οποίος είναι η πρακτική εφαρμογή αρχιτεκτονικών νέφους (Cloud Architectures) για την δημιουργία μιας ενιαίας, διαχειρίσιμης και ανθεκτικής υποδομής. Οι επιμέρους τεχνικοί στόχοι μπορούν να αναλυθούν ως εξής:

- **Υλοποίηση Ενιαίου Δικτύου Layer 2 (Network Unification):** Η δημιουργία ενός ασφαλούς **VPN tunnel** σε λειτουργία TAP, η οποία θα επιτρέπει τη γεφύρωση (Bridging) ετερογενών δικτύων. Το ζητούμενο είναι οι κόμβοι σε Public Cloud (Azure) και Private Cloud (Proxmox) να ενσωματωθούν στο ίδιο Broadcast domain με τον τοπικό κόμβο διαχείρισης, λαμβάνοντας ip address από τον κεντρικό DHCP server.
- **Σχεδιασμός Ανθεκτικού Συστήματος Αποθήκευσης (Resilient Storage):** Η εφαρμογή τεχνικών **Software-Defined Storage (SDS)** στον κεντρικό κόμβο (Node 2), με σκοπό την ακεραιότητα των δεδομένων. Συγκεκριμένα, στοχεύουμε στην συνδιαστική χρήση συστοιχιών RAID (για ανοχή σε σφάλματα υλικού) και διαχείρισης λογικών τομών (LVM) για την ευέλικτη κατανομή του αποθηκευτικού χώρου.
- **Ανάπτυξη Μηχανισμού Τηλεμετρίας Αυτοματισμού (Telemetry & Automation):** Η δημιουργία **scripts** σε γλώσσα Python για την περιοδική και αυτόματη συλλογή μετρήσεων απόδοσης (CPU, Connectivity status) από όλους τους κόμβους. Τα δεδομένα αυτά πρέπει να συγκεντρώνονται, να μορφοποιούνται και να αποθηκεύονται ασφαλώς στο κεντρικό σύστημα storage για μελλοντική ανάλυση.

## 1.4 Μεθοδολογία υλοποίησης

Σε αυτή την υποενότητα, σημαντικό είναι να αναφερθούμε στην μεθοδολογία που θα χρησιμοποιηθεί για την υλοποίηση της. Θα ακολουθήσουμε μια σταδιακή προσέγγιση (**Phased Implementation**), ξεκινώντας από το χαμηλότερο επίπεδο της υποδομής και προχωρώντας προς τις εφαρμογές δικτύωσης και αυτοματισμού. Για την επίτευξη των στόχων, θα αξιοποιηθεί ένα σύνολο εργαλείων ανοιχτού κώδικα (open source), διασφαλίζοντας τη φορητότητα και την επεκτασιμότητα της λύσης. Πιο συγκεκριμένα, θα χρησιμοποιηθούν τα επιμέρους τεχνολογικά εργαλεία ανά κατηγορία:

Πίνακας 1: Εργαλεία και Τεχνολογίες Υλοποίησης Υβριδικής Υποδομής

| Κατηγορία             | Τεχνολογία        | Περιγραφή & Ρόλος στην Υποδομή                      |
|-----------------------|-------------------|-----------------------------------------------------|
| <b>Εικονικοποίηση</b> | Microsoft Azure   | Φιλοξενία του δημόσιου κόμβου (Public Cloud Node).  |
|                       | Proxmox VE        | Προσομοίωση του ιδιωτικού κέντρου δεδομένων.        |
|                       | Oracle VirtualBox | Τοπικός κόμβος διαχείρισης και περιβάλλον δοκιμών.  |
|                       | Ubuntu Server     | Το βασικό λειτουργικό σύστημα όλων των κόμβων.      |
| <b>Δικτύωση</b>       | OpenVPN           | Κρυπτογραφημένο tunnel Layer 2 (TAP Mode).          |
|                       | Bridge-utils      | Γεφύρωση (Bridging) φυσικών και εικονικών διεπαφών. |
|                       | WireGuard         | Ασφαλής απομακρυσμένη πρόσβαση στην υποδομή.        |
|                       | SSH / RSA         | Ασφαλής διαχείριση και μεταφορά αρχείων.            |
| <b>Αποθήκευση</b>     | mdadm             | Υλοποίηση Software RAID 1 & 5 για πλεονασμό.        |
|                       | LVM2              | Δυναμική διαχείριση και κατανομή χώρου.             |
| <b>Αυτοματισμός</b>   | Python 3          | Scripts για monitoring και επεξεργασία δεδομένων.   |
|                       | Bash / Crontab    | Αυτοματοποίηση εργασιών και διαχείριση συστήματος.  |

Η επιλογή της συγκεκριμένης τεχνολογικής στοίβας (Technology Stack) δεν έγινε μόνο με κριτήριο τη διαθεσιμότητα, αλλά με γνώμονα τη δημιουργία μιας αρθρωτής και επεκτάσιμης υποδομής. Έχοντας καθορίσει τα δομικά εργαλεία της υλοποίησης, το επόμενο βήμα αφορά τη σύνθεσή τους σε ένα λειτουργικό σύνολο.

Στο κεφάλαιο που ακολουθεί (Κεφάλαιο 2: Αρχιτεκτονικός Σχεδιασμός Συστήματος), θα αναλυθεί ο τρόπος με τον οποίο οι παραπάνω τεχνολογίες διασυνδέονται. Θα παρουσιαστεί η επιλεγμένη τοπολογία δικτύου, η στρατηγική διασύνδεσης (Connectivity Strategy) που εξασφαλίζει τη διαφάνεια στην επικοινωνία, καθώς και το σχέδιο διευθυνσιοδότησης (IP Addressing Plan) που αποτελεί τη βάση για την ορθή δρομολόγηση των δεδομένων στο υβριδικό περιβάλλον.

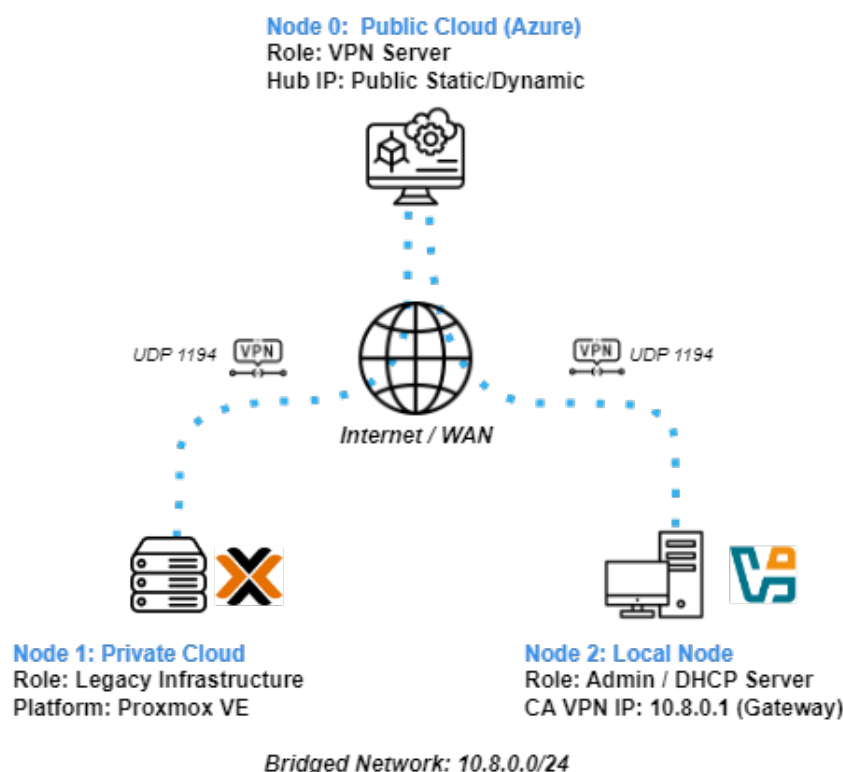
## 2 Κεφάλαιο 2: Αρχιτεκτονικός σχεδιασμός συστήματος

Σε αυτό το κεφάλαιο, είναι σημαντικό να αναλύσουμε τις θεμελιώδεις αρχιτεκτονικές επιλογές που διέπουν τη σχεδίαση της υβριδικής υποδομής. Θα παρουσιάσουμε τη συνολική τοπολογία του δικτύου, τη στρατηγική διασύνδεση των ετερογενών συστημάτων και το σχέδιο διευθυνσιοδότησης (IP Addressing Plan). Ο βασικός μας άξονας του σχεδιασμού αποτελεί η υιοθέτηση του μοντέλου Hub-and-Spoke, με τον κεντρικό κόμβο διαχείρισης (Node 2) να αναλαμβάνει τον ρόλο του ρυθμιστή του network traffic και του εγγυητή της ασφάλειας των δεδομένων. Συνεπώς η αρχιτεκτονική αυτή, σχεδιάστηκε με γνώμονα την επεκτασιμότητα, την απομόνωση των κρίσιμων πόρων και την εξασφάλιση της ακεραιότητας της επικοινωνίας σε επίπεδο ζεύξης (Data Link Layer).

### 2.1 Τοπολογία Δικτύου (Network Topology)

Αρχικά, θα μιλήσουμε για την διασύνδεση των τριών κόμβων όπου επιλέχθηκε η star topology. Στο κέντρο αυτής της λογικής τοπολογίας τοποθετείται ο OpenVPN Server (Node 0), ο οποίος λειτουργεί ως κεντρικός κόμβος (hub) που υποδέχεται τις κρυπτογραφημένες συνδέσεις από τους περιφερειακούς κόμβους (spokes/clients).

Ωστόσο, σε επίπεδο ροής δεδομένων και ελέγχου, η αρχιτεκτονική σχεδιάστηκε ώστε να αναδεικνύει τον Node 2 ως το κέντρο διαχείρισης. Μέσω της τεχνικής της γεφύρωσης (**bridging**) δημιουργείται ένα ενιαίο λογικό δίκτυο όπου όλοι οι κόμβοι επικοινωνούν ισότιμα, προσομοιώνοντας μια τοπολογία αρτηρίας (bus topology) πάνω από το εικονικό κανάλι.



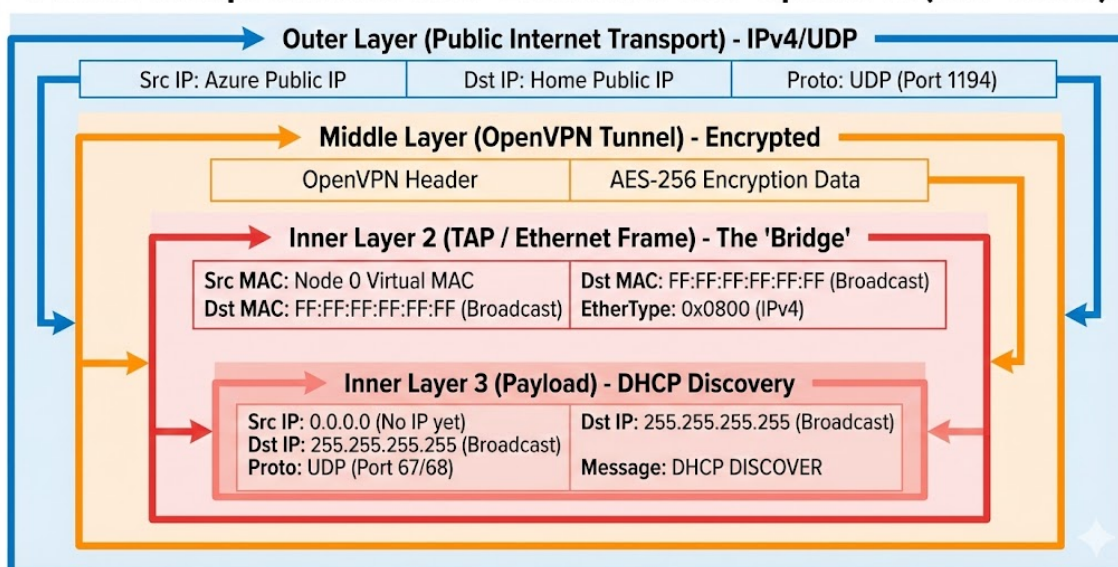
Εικόνα 2.1: Τοπολογία Δικτύου και Διασύνδεση Κόμβων

## 2.2 Στρατηγική Διασύνδεσης (Connectivity Strategy)

Συνεχίζοντας σε αυτή την υποενότητα, σημαντικό είναι να αναφέρουμε την επιλογή του πρωτοκόλλου διασύνδεσης που χρησιμοποιήθηκε για την κρίσιμη σχεδιαστική απόφαση. Μεταξύ των μοντέλων Routing (Layer 3 - TUN) και Bridging (Layer 2 - TAP), επιλέχθηκε το δεύτερο για τους εξής τεχνικούς λόγους:

- **Υποστήριξη Broadcasts:** Σε εταιρικά περιβάλλοντα, κρίσιμες εφαρμογές παλαιότερης τεχνολογίας (legacy apps) που φιλοξενούνται σε υποδομές όπως ο Node 1, συχνά βασίζονται σε μηχανισμούς Layer 2 Broadcast για την ανακάλυψη υπηρεσιών (service discovery). Η επιλογή της λειτουργίας TAP καθίσταται αναγκαίος σε αυτές τις περιπτώσεις, καθώς επιτρέπει την ενθυλάκωση και διέλευση **Ethernet frames** μέσα από το tunnel, λειτουργία που δεν υποστηρίζεται στο επίπεδο Layer 3 (Routing).

### Packet Encapsulation: DHCP Broadcast over OpenVPN (TAP Mode)



Εικόνα 2.2: Διάγραμμα Ενθυλάκωσης Πακέτων (Packet Encapsulation)

Όπως φαίνεται στην Εικόνα 2.2, η τεχνική του Bridging (TAP) μας επιτρέπει να μεταφέρουμε ολόκληρο το πλαίσιο δεδομένων του επιπέδου ζεύξης (Data Link Layer). Ωστόσο, αυτή η αρχιτεκτονική επιλογή συνοδεύεται από συγκεκριμένους σχεδιαστικούς περιορισμούς (design trade-offs).

Ενώ το Bridging είναι ιδανικό για την παρούσα τοπολογία μικρής κλίμακας (3 κόμβοι) και για την ανάγκη εξυπηρέτησης Broadcasts, δεν συνίσταται για δίκτυα μεγάλης κλίμακας (π.χ. >1000 κόμβοι). Σε ένα τέτοιο σενάριο, η ανεξέλεγκτη διάδοση των πακέτων Broadcast (Broadcast Storms) θα προκαλούσε εκθετική αύξηση της κίνησης (overhead), οδηγώντας σε κορεσμό του εύρους ζώνης (bandwidth saturation). Σε περιπτώσεις μαζικής κλίμακας, η βέλτιστη πρακτική επιτάσσει τη χρήση του μοντέλου Routing (TUN), το οποίο απομονώνει τα Broadcast domains και προσφέρει καλύτερη αποδοτικότητα, θυσιάζοντας όμως τη συμβατότητα με Layer 2 πρωτόκολλα.

- **Ενιαίο Δίκτυο (Unified Network Topology):** Η τεχνική του Bridging ενοποιεί τους γεωγραφικά απομακρυσμένους κόμβους (Node 0, Node 1, Node 2) σε ένα ενιαίο IP subnet (10.8.0.0/24). Αυτή η αρχιτεκτονική απλοποιεί δραστικά τη διαχείριση του δικτύου, καθώς εξαλείφει την ανάγκη για πολύπλοκους πίνακες δρομολόγησης (static routing tables) και ρυθμίσεις NAT μεταξύ των κόμβων, προσομοιώνοντας ένα τοπικό δίκτυο (LAN) πάνω από το Internet.
- **Κεντρικός έλεγχος DHCP (Centralized DHCP Control):** Μέσω της tunnel Layer 2, ο DHCP server που φιλοξενείται στον κόμβο διαχείρισης (Node 2) καθίσταται ικανός να αποδίδει δυναμικά διευθύνσεις IP σε όλους τους clients, συμπεριλαμβανομένου του απομακρυσμένου εξυπηρετητή στο Azure (Node 0). Αυτό προσφέρει στον διαχειριστή τον απόλυτο και κεντρικό έλεγχο της απόδοσης πόρων και της εποπτείας του ιδιωτικού δικτύου, ανεξάρτητα από την τοποθεσία των τερματικών.

## 2.3 Σχέδιο Διευθυνσιοδότησης (IP Addressing Plan)

Σημαντικό λοιπόν είναι πως για την αποφυγή των ip conflicts με τα υφιστάμενα δίκτυα του Πανεπιστημίου (10.100.x.x) και του οικιακού δικτύου (192.168.1.x), να επιλεχθεί για το VPN το private network range **10.8.0.0/24**.

Ο σχεδιασμός προβλέπει την ακόλουθη κατανομή:

Πίνακας 2: Σχέδιο Διευθυνσιοδότησης και Ρόλοι VPN Δικτύου

| Κόμβος        | Ρόλος                       | Τύπος IP (VPN Interface) | Περιγραφή                                                       |
|---------------|-----------------------------|--------------------------|-----------------------------------------------------------------|
| <b>Node 0</b> | VPN Server                  | Dynamic (DHCP Client)    | Λαμβάνει IP από τον Node 2 μέσω του vpn tunnel (π.χ. 10.8.0.2). |
| <b>Node 1</b> | VPN Client                  | Dynamic (DHCP Client)    | Λαμβάνει IP από τον Node 2 μέσω του vpn tunnel (π.χ. 10.8.0.3). |
| <b>Node 2</b> | Gateway / DHCP / VPN client | Static (10.8.0.1)        | Λειτουργεί ως η πύλη και ο DHCP Server του VPN δικτύου.         |

Η τοποθέτηση του DHCP Server στον κόμβο client (Node 2) αντί για τον εξυπηρετητή (Node 0) αποτελεί μια επιλογή "Αντεστραμμένης Διαχείρισης", εξασφαλίζοντας ότι η λογική του δικτύου παραμένει υπό τον έλεγχο του διαχειριστή, ανεξάρτητα από τον πάροχο Cloud που φιλοξενεί τον Node 0.

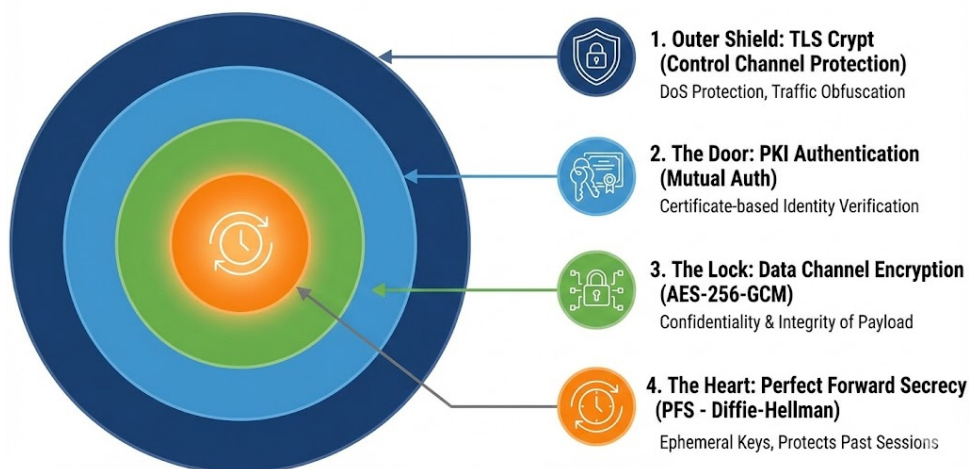


## 2.4 Αρχιτεκτονική Ασφάλειας (Security Architecture)

Κλείνοντας αυτό το κεφάλαιο, είναι επιτακτική ανάγκη να αναφερθούμε στην ασφάλεια της hybrid υποδομής. Δεδομένου ότι η κίνηση διέρχεται μέσω δημόσιων δικτύων, εφαρμόζονται πολλαπλά επίπεδα προστασίας για την διασφάλιση της εμπιστευτικότητας και της ακεραιότητας των δεδομένων:

- **Κρυπτογράφηση των δεδομένων:** Όλη η κίνηση εντός του VPN tunnel κρυπτογραφείται με χρήση του αλγορίθμου AES-256-GCM, ο οποίος αποτελεί βιομηχανικό πρότυπο για την προστασία ευαίσθητων δεδομένων.
- **Αυστηρή Πιστοποίηση (PKI):** Η πρόσβαση στο δίκτυο δεν επιτρέπεται με απλούς κωδικούς, αλλά απαιτείται αμοιβαία πιστοποίηση (Mutual Authentication) μέσω ψηφιακών υπογραφών. Κάθε κόμβος (server - clients) διαθέτει μοναδικό πιστοποιητικό, υπογεγραμμένο από την ιδιωτική αρχή πιστοποίησης (CA) της υποδομής.
- **Προστασία Καναλιού Ελέγχου (TLS Crypt):** Για την απόκρυψη της φύσης της κίνησης και την προστασία από επιθέσεις Dos (Denial of Service) κατά την φάση της σύνδεσης, προβλέπεται η χρήση επιπρόσθετου στατικού κλειδιού (Shared Secret) για την κρυπτογράφηση του Control Channel πριν την εγκαθύδριση της συνεδρίας TLS.
- **Προστασία Ιστορικότητας Δεδομένων (Perfect Forward Secrecy - PFS):** Η αρχιτεκτονική αξιοποιεί τον μηχανισμό ανταλλαγής κλειδιών Diffie-Hellman για την παραγωγή εφήμερων κλειδιών συνεδρίας (ephemeral session keys). Αυτή η πρακτική διασφαλίζει ότι, ακόμα και σε περίπτωση μελλοντικής διαρροής του ιδιωτικού κλειδιού (Private Key) του εξυπηρετητή, η ιστορικότητα της κίνησης παραμένει ασφαλής και μη αποκρυπτογραφήσιμη, καθώς κάθε σύνοδος (session) προστατεύεται από μοναδικό κλειδί που καταστρέφεται μετά τη χρήση του.

### OpenVPN Security Architecture: Defense in Depth (Onion Model)



Εικόνα 2.3: Αρχιτεκτονική OpenVPN (Onion Model)

### 3 Κεφάλαιο 3: Υλοποίηση Υποδομής και Storage

Στο παρόν κεφάλαιο περιγράφεται η διαδικασία υλοποίησης και παραμετροποίησης των τριών βασικών κόμβων (Nodes) που συνθέτουν την ετερογενή υβριδική υποδομή μας. Ο στόχος είναι η δημιουργία ενός περιβάλλοντος που συνδυάζει δημόσιες υπηρεσίες νέφους (Public Cloud), τοπικούς Hypervisors (Private Cloud) και εικονικοποίηση σε επίπεδο χρήστη (Local Node).

#### 3.1 Προετοιμασία Κόμβων (Node Provisioning)

Σε αυτή την υποενότητα θα αναφερθούμε στην υποδομή, όπου αποτελείται από τρία διακριτά υπολογιστικά συστήματα, καθένα με συγκεκριμένο ρόλο. Ως κοινό λειτουργικό σύστημα επιλέχθηκε το Ubuntu Linux 22.04 LTS, εξασφαλίζοντας ομοιογένεια και συμβατότητα με τα εργαλεία διαχείρισης. Ακολουθεί ο πίνακας με τα τεχνικά χαρακτηριστικά που ορίστηκαν για κάθε κόμβο:

Πίνακας 3: Κατανομή κόμβων υποδομής και τεχνικές προδιαγραφές

| Node ID | Role          | Infrastructure | OS / Stack <sup>1</sup> | Resources      | Networking   |
|---------|---------------|----------------|-------------------------|----------------|--------------|
| Node 0  | Public Cloud  | Azure          | Ubuntu Server           | 2 vCPU / 1GB   | Public IP    |
| Node 1  | Private Cloud | Proxmox VE     | Ubuntu Server           | 1 vCPU / 1.5GB | Internal LAN |
| Node 2  | Lab Edge      | VirtualBox     | Ubuntu Desktop          | 2 vCPU / 8GB   | Bridge       |

<sup>1</sup> Τα Node 0 και το vm μέσα στο Node1, χρησιμοποιούν την έκδοση Ubuntu Server 24.04 LTS.

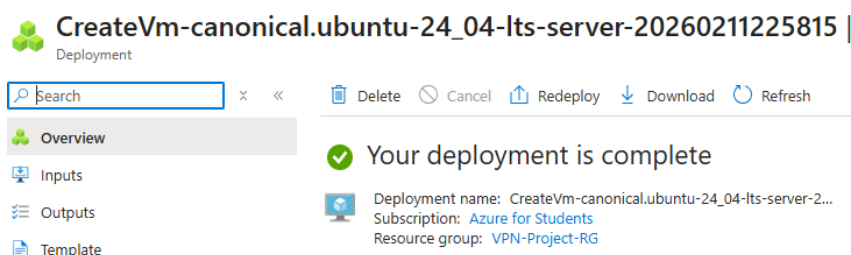
<sup>2</sup> Το Node 2 χρησιμοποιεί την έκδοση Ubuntu Desktop 24.04 LTS.

#### 3.2 Node 0 (Cloud Environment)

Αρχικά, ο συγκεκριμένος κόμβος λειτουργεί ως το κεντρικό σημείο παρουσίας στο Cloud, αναλαμβάνοντας τον ρόλο του VPN Gateway για τη διασύνδεση και την ασφαλή επικοινωνία με τους υπόλοιπους απομακρυσμένους κόμβους.

##### 3.2.1 Δημιουργία VM στο Azure και Πύθμιση Firewall (NSG)

Δημιουργήσαμε ένα εικονικό μηχάνημα τύπου Standard\_B2ts\_v2 στην περιοχή France Central.



Εικόνα 3.1: Επιτυχής δημιουργία του virtual machine στο Azure

Πριν όμως την οριστική έναρξη της ανάπτυξης (deployment), πραγματοποιήσαμε τον τεχνικό έλεγχο των παραμέτρων μέσω της διαδικασίας Validation. Όπως φαίνεται στην Εικόνα 3.2, το Azure επιβεβαίωσε την εγκυρότητα των ρυθμίσεων (**Validation passed**), παρέχοντας μια αναλυτική σύνοψη των χαρακτηριστικών του Node 0.



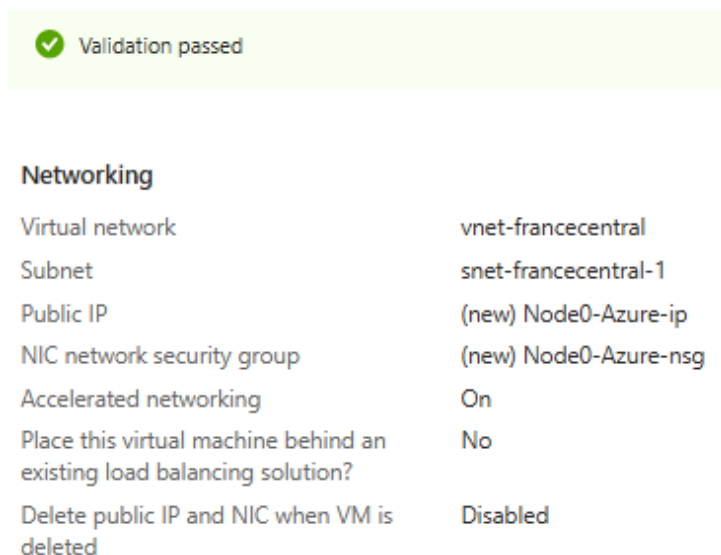
| Basics                 |                                          |
|------------------------|------------------------------------------|
| Subscription           | Azure for Students                       |
| Resource group         | (new) VPN-Project-RG                     |
| Virtual machine name   | Node0-Azure                              |
| Region                 | France Central                           |
| Availability options   | No infrastructure redundancy required    |
| Zone options           | Self-selected zone                       |
| Security type          | Trusted launch virtual machines          |
| Enable secure boot     | Yes                                      |
| Enable vTPM            | Yes                                      |
| Integrity monitoring   | No                                       |
| Image                  | Ubuntu Server 24.04 LTS - Gen2           |
| VM architecture        | x64                                      |
| Size                   | Standard B2ts v2 (2 vcpus, 1 GiB memory) |
| Enable Hibernation     | No                                       |
| Authentication type    | SSH public key                           |
| Username               | azureuser                                |
| SSH Key format         | RSA                                      |
| Key pair name          | node0-vpn-key                            |
| Azure Spot             | No                                       |
| Disks                  |                                          |
| OS disk size           | Image default                            |
| OS disk type           | Premium SSD LRS                          |
| Use managed disks      | Yes                                      |
| Delete OS disk with VM | Enabled                                  |
| Ephemeral OS disk      | No                                       |

Εικόνα 3.2: Βασικές ρυθμίσεις στο virtual machine (Azure)

- **Στοιχεία Ταυτότητας:** Το εικονικό μηχάνημα με όνομα Node0-Azure όπου εγκαταστάθηκε στην περιοχή **France Central**.
- **Τεχνικές Προδιαγραφές:** Επιλέχθηκε η έκδοση **Ubuntu Server 24.04 LTS** και το μέγεθος **Standard B2ts v2 (2 vCPUs)**, το οποίο καλύπτει πλήρως τις υπολογιστικές ανάγκες της υπηρεσίας VPN.
- **Ασφάλεια και Πρόσβαση:** Για την ενίσχυση της ασφάλειας, η αυθεντικοποίηση ορίστηκε αποκλειστικά μέσω **SSH public key** με τον χρήστη **azureuser** και το κλειδί **node0-vpn-key**, καθώς και η ενσωμάτωση ενός password για τον **azureuser**.
- **Υποδομή Storage:** Ο δίσκος λειτουργικού συστήματος ορίστηκε ως **Premium SSD LRS** για την εξασφάλιση υψηλών ταχυτήτων ανάγνωσης/γγραφής.



Ιδιαίτερη έμφαση δόθηκε στον σχεδιασμό της δικτυακής υποδομής του Node 0, καθώς αποτελεί τον κόμβο με τη μοναδική άμεσα προσβάσιμη δημόσια διεύθυνση. Όπως αναλύεται στην παρακάτω εικόνα, οι ρυθμίσεις δικτύου ορίστηκαν ως εξής:

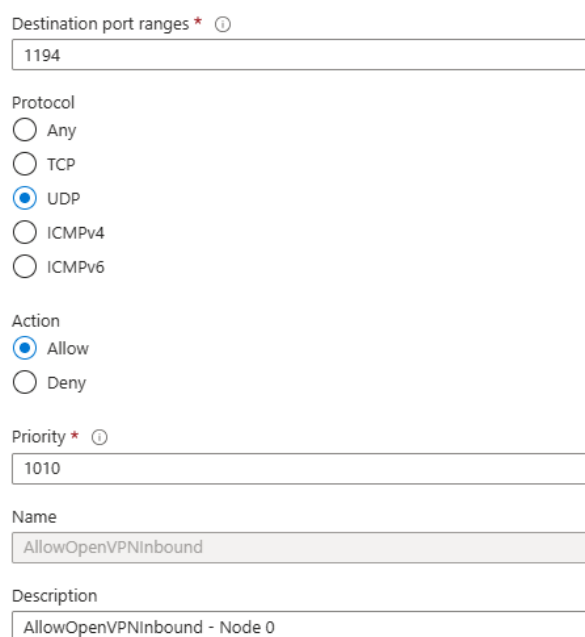


The image shows a screenshot of the 'Networking' section in the Azure portal for a virtual machine. At the top, a green banner indicates 'Validation passed'. Below this, the 'Networking' section is expanded, showing a list of configuration items and their values.

| Networking                                                             |                       |
|------------------------------------------------------------------------|-----------------------|
| Virtual network                                                        | vnet-francecentral    |
| Subnet                                                                 | snet-francecentral-1  |
| Public IP                                                              | (new) Node0-Azure-ip  |
| NIC network security group                                             | (new) Node0-Azure-nsg |
| Accelerated networking                                                 | On                    |
| Place this virtual machine behind an existing load balancing solution? | No                    |
| Delete public IP and NIC when VM is deleted                            | Disabled              |

Εικόνα 3.3: Βασικές ρυθμίσεις του δικτύου στο virtual machine (Azure)

- **Virtual Network & Subnet:** Το VM εντάχθηκε κανονικά στο εικονικό δίκτυο `vnet-francecentral` και στο subnet `snet-francecentral-1`, εξασφαλίζοντας ένα απομονωμένο περιβάλλον εργασίας εντός του Azure.
- **Public IP:** Δημιουργήθηκε μια νέα δημόσια διεύθυνση IP (`Node0-Azure-ip`), η οποία επιτρέπει την πρόσβαση μέσω **SSH** και τη λειτουργία του **OpenVPN tunnel**.
- **Accelerated Networking:** Η επιλογή ορίστηκε σε **On**, προκειμένου να μειωθεί το latency και να βελτιστοποιηθεί η ταχύτητα διαμεταγωγής (throughput) των δεδομένων του VPN.
- **Χειροκίνητη Παραμετροποίηση Ασφαλείας (NSG):** Πέρα από την εγκατάσταση του λογισμικού, η ορθή λειτουργία του Node 0 βασίζεται στην απρόσκοπτη επικοινωνία του με τον έξω κόσμο. Για τον σκοπό αυτό, πραγματοποιήσαμε μια κρίσιμη ρύθμιση στο επίπεδο της υποδομής του Azure:
  - **Άνοιγμα port 1194 (UDP):** Προσθέσαμε έναν νέο κανόνα εισερχόμενης κίνησης (Inbound Security Rule) στο Network Security Group (NSG) για τον Node 0. Επιλέξαμε το πρωτόκολλο UDP έναντι του TCP, καθώς προσφέρει σημαντικά χαμηλότερο overhead, κάτι που είναι ζωτικής σημασίας για τις επιδόσεις ενός κρυπτογραφημένου VPN tunnel.



Destination port ranges \* ⓘ  
1194

Protocol  
☐ Any  
☐ TCP  
☒ UDP  
☐ ICMPv4  
☐ ICMPv6

Action  
☒ Allow  
☐ Deny

Priority \* ⓘ  
1010

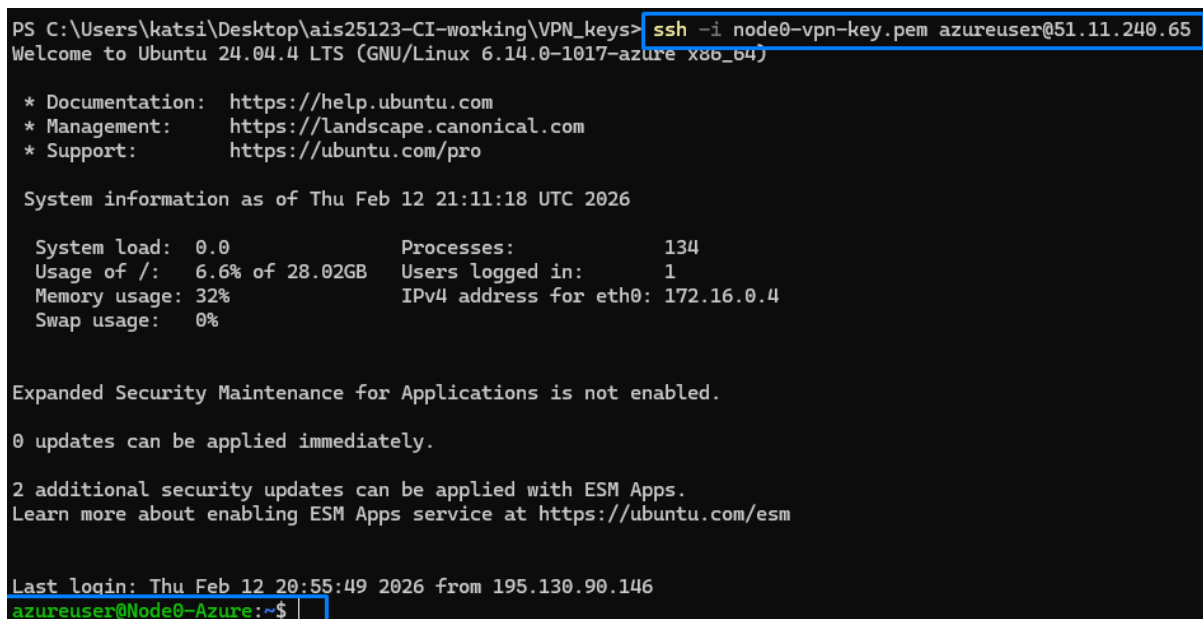
Name  
AllowOpenVPNInbound

Description  
AllowOpenVPNInbound - Node 0

Εικόνα 3.4: Δημιουργία inbound security rule για την θύρα 1194

### 3.2.2 Απομακρυσμένη Διαχείριση (ssh)

Αφού ολοκληρώσαμε την παραμετροποίηση της υποδομής στο Azure, προχωρήσαμε στην προετοιμασία του λειτουργικού συστήματος. Συνδεθήκαμε απομακρυσμένα στον Node 0 μέσω του πρωτοκόλλου SSH, χρησιμοποιώντας την δημόσια IP που αποδόθηκε από την πλατφόρμα.



```
PS C:\Users\katsi\Desktop\ais25123-CI-working\VPN_keys> ssh -i node0-vpn-key.pem azureuser@51.11.240.65
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.14.0-1017-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Feb 12 21:11:18 UTC 2026

System load:  0.0          Processes:            134
Usage of /:   6.6% of 28.02GB Users logged in:        1
Memory usage: 32%          IPv4 address for eth0: 172.16.0.4
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Thu Feb 12 20:55:49 2026 from 195.130.90.146
azureuser@Node0-Azure:~$
```

Εικόνα 3.5: Είσοδος στο Azure VM με χρήση του ssh πρωτοκόλλου

### 3.2.3 Προετοιμασία Ubuntu server και εγκατάσταση OpenVPN

Αφού εξασφαλίσουμε την απομακρυσμένη πρόσβαση μέσω SSH, προχωρήσαμε στην προετοιμασία του λειτουργικού συστήματος Ubuntu Server 24.04 LTS. Για να καταστήσουμε τον κόμβο έτοιμο να φιλοξενήσει τις υπηρεσίες VPN, εκτελέσαμε μια σειρά βασικών εντολών διαχείρισης, οι οποίες συνοψίζονται στον παρακάτω πίνακα:

Πίνακας 4: Βασικές Εντολές Προετοιμασίας και Εγκατάστασης Λογισμικού

| Εντολή                                                                           | Σκοπός και Αποτέλεσμα                                                                                                                          |
|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>sudo apt upgrade -y</code>                                                 | Επικαιροποιήσαμε το αποθετήριο πακέτων και αναβαθμίσαμε το σύστημα στην τελευταία σταθερή έκδοση.                                              |
| <code>sudo apt install openvpn<br/>easy-rsa net-tools<br/>bridge-utils -y</code> | Εγκαταστήσαμε το OpenVPN μαζί με το Easy-RSA, καθώς και τα απαραίτητα εργαλεία δικτύωσης (ifconfig, brctl) για την παραμετροποίηση του tunnel. |
| <code>ifconfig</code>                                                            | Επαληθεύσαμε τις δικτυακές διεπαφές (interfaces) και την εσωτερική IP του κόμβου για την ορθή δρομολόγηση των πακέτων.                         |

Μετά την επιτυχή εκτέλεση των εντολών, προχωρήσαμε στην οπτική επαλήθευση της κατάστασης του Node 0 μέσω του τερματικού:

```
azureuser@Node0-Azure:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  base-files gir1.2-glib-2.0 inetutils-telnet
```

Εικόνα 3.6: Εκτέλεση εντολής για το upgrade του συστήματος (Azure)

Στο παραπάνω screenshot βλέπουμε πως η επιλογή μας να ξεκινήσουμε με την πλήρη αναβάθμιση του συστήματος (upgrade) δεν ήταν τυπική, αλλά στρατηγική. Παρόλο που η πλατφόρμα του Azure παρέχει έτοιμα είδωλα (images) του Ubuntu 24.04, αυτά συχνά υστερούν σε κρίσιμες ενημερώσεις ασφαλείας που έχουν μεσολαβήσει από τη δημιουργία του ειδώλου μέχρι τη στιγμή του deployment.

Συνεπώς, έχουμε την επιτυχή ολοκλήρωση της αναβάθμισης των πακέτων, όπου το σύστημα αναγνώρισε την ταυτότητά μας (azureuser@Node0-Azure) και ενημέρωσε τις βιβλιοθήκες χωρίς σφάλματα και έτσι μας επέτρεψε να δημιουργήσουμε μια σταθερή και ασφαλή βάση ("baseline"), πάνω στην οποία βασίσαμε την υπόλοιπη παραμετροποίηση της υποδομής μας.

Στην παρακάτω εικόνα, επιβεβαιώνουμε την εγκατάσταση του OpenVPN και των εργαλείων net-tools.

```
azureuser@Node0-Azure:~$ sudo apt install openvpn easy-rsa net-tools bridge-utils -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libccid libeac3 libpcsc-lite1 libpkcs11-helper1t64 opensc opensc-pkcs11 pcscl
Suggested packages:
  ifupdown pcmciautils openvpn-dco-dkms openvpn-systemd-resolved
The following NEW packages will be installed:
  bridge-utils easy-rsa libccid libeac3 libpcsc-lite1 libpkcs11-helper1t64 net-tools op
0 upgraded, 11 newly installed, 0 to remove and 0 not upgraded.
```

Εικόνα 3.7: Εγκατάσταση του OpenVPN και δικτυακών εργαλείων (Azure)

Στην συνέχεια με τις εντολές **openvpn --version** και **ifconfig** επαληθεύσαμε ότι έχει γίνει ορθά η εγκατάσταση του εργαλείου και ότι η εσωτερική διεύθυνση IP που αναγνωρίζει το λειτουργικό σύστημα (π.χ. 172.16.0.4) ταυτίζεται απόλυτα με την IP που αποδόθηκε αυτόματα από την πλατφόρμα Azure. Η διασταύρωση αυτή είναι απαραίτητη για να διασφαλίσουμε ότι ο κόμβος έχει ενταχθεί ορθά στο εικονικό δίκτυο (vnet-francecentral) και είναι έτοιμος για τη δρομολόγηση των πακέτων του VPN.

```
azureuser@Node0-Azure:~$ openvpn --version
OpenVPN 2.6.14 x86_64-pc-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
library versions: OpenSSL 3.0.13 30 Jan 2024, LZO 2.10
DCO version: N/A
Originally developed by James Yonan
Copyright (C) 2002-2024 OpenVPN Inc <sales@openvpn.net>
Compile time defines: enable_async_push=no enable_comp_stub=no enable_crypto_ofb_cfb=yes enable_dco=yes en
endency_tracking=no enable_dlopen=unknown enable_dlopen_self=unknown enable_dlopen_self_static=unknown ena
s enable_iproute2=no enable_libtool_lock=yes enable_lz4=yes enable_lzo=yes enable_maintainer_mode=no enabl
nable_pam_dlopen=no enable_pedantic=no enable_pkcs11=yes enable_plugin_auth_pam=yes enable_plugin_down_roo
es enable_selinux=no enable_shared=yes enable_shared_with_static_runtimes=no enable_silent_rules=no enable
enable_strict_options=no enable_systemd=yes enable_unit_tests=no enable_werror=no enable_win32_dll=yes en
ername=yes with_aix_soname=aix with_crypto_library=openssl with_gnu_ld=yes with_mem_check=no with_openssl
azureuser@Node0-Azure:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.16.0.4 netmask 255.255.255.0 broadcast 172.16.0.255
    inet6 fe80::7ead:8dff:fe84:ad6 prefixlen 64 scopeid 0x20<link>
    ether 7c:ed:8d:84:0a:d6 txqueuelen 1000 (Ethernet)
    RX packets 11626 bytes 65858454 (65.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11145 bytes 1826405 (1.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Εικόνα 3.8: Επιτυχής έλεγχος του vm στο Azure για OpenVPN και δικτύου

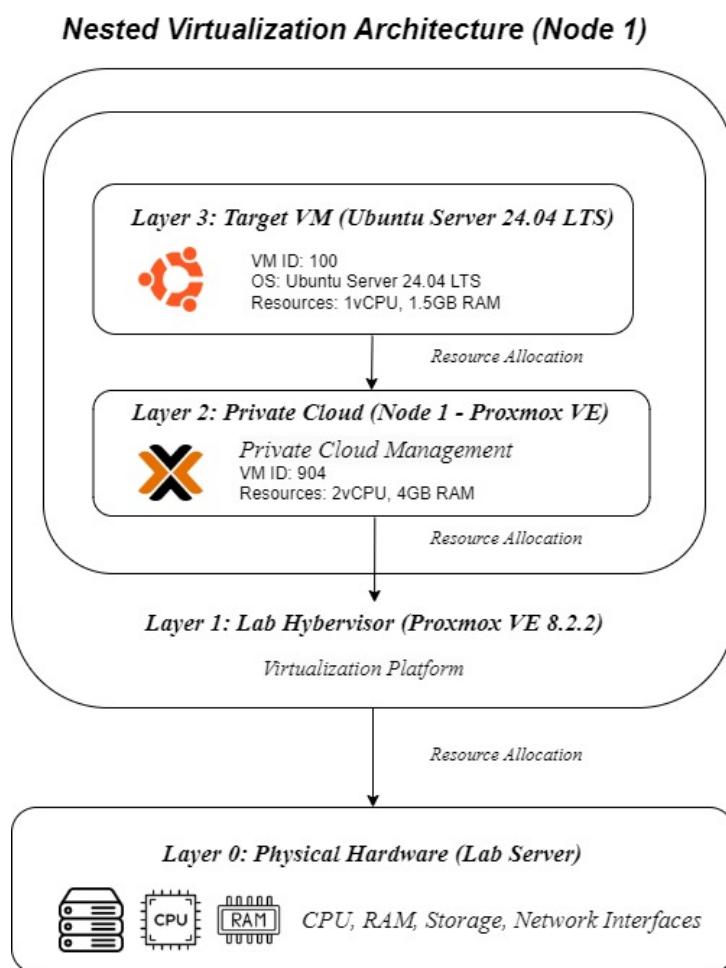
Με την ολοκλήρωση των παραπάνω βημάτων, ο Node 0 είναι πλέον πλήρως θωρακισμένος, επικαιροποιημένος και έτοιμος να λειτουργήσει ως το κεντρικό σημείο παρουσίας μας στο δημόσιο νέφος.

Κλείνοντας, σημαντικό να αναφερθεί είναι ότι με την επιτυχή λειτουργία για τον Node 0 στο δημόσιο νέφος, προχωρήσαμε στην υλοποίηση του Node 1 στο εσωτερικό δίκτυο (Private Cloud) του εργαστηρίου. Η συγκεκριμένη υλοποίηση παρουσιάζει ιδιαίτερο τεχνικό ενδιαφέρον, καθώς βασίζεται στην αρχιτεκτονική **Nested Virtualization**.

### 3.3 Node 1 (Lab Private Cloud)

Για την υλοποίηση του Node 1 (Private Cloud), εφαρμόσαμε την αρχιτεκτονική Nested Virtualization, εγκαθιστώντας έναν αυτόνομο Hypervisor εντός της εργαστηριακής υποδομής. Η διαστρωμάτωση τεσσάρων επιπέδων, όπως απεικονίζεται στο παρακάτω διάγραμμα, εξασφαλίζει πλήρη απομόνωση και απόλυτο έλεγχο των πόρων.

- **Layer 0 (Physical Hardware):** Ο φυσικός εξυπηρετητής του εργαστηρίου που παρέχει την απαραίτητη υπολογιστική ισχύ για όλη την υποδομή.
- **Layer 1 (Lab Hypervisor):** Η κεντρική πλατφόρμα εικονικοποίησης της σχολής, η οποία φιλοξενεί το ενδιαμέσο επίπεδο διαχείρισης.
- **Layer 2 (Node 1: Nested Hypervisor - Proxmox VE):** Σε αυτό το επίπεδο λοιπόν εγκαταστήσαμε το Proxmox VE 9.1.1, δημιουργώντας το δικό μας αυτόνομο, ιδιωτικό υπολογιστικό νέφος (Private Cloud).
- **Layer 3 (Nested VM - Ubuntu Server):** Εντός του δικού μας Proxmox, σχεδιάσαμε και εκκινήσαμε την εικονική μηχανή με **ID 100**. Στον κόμβο αυτόν εγκαταστάθηκε το λειτουργικό σύστημα Ubuntu Server 24.04 LTS.



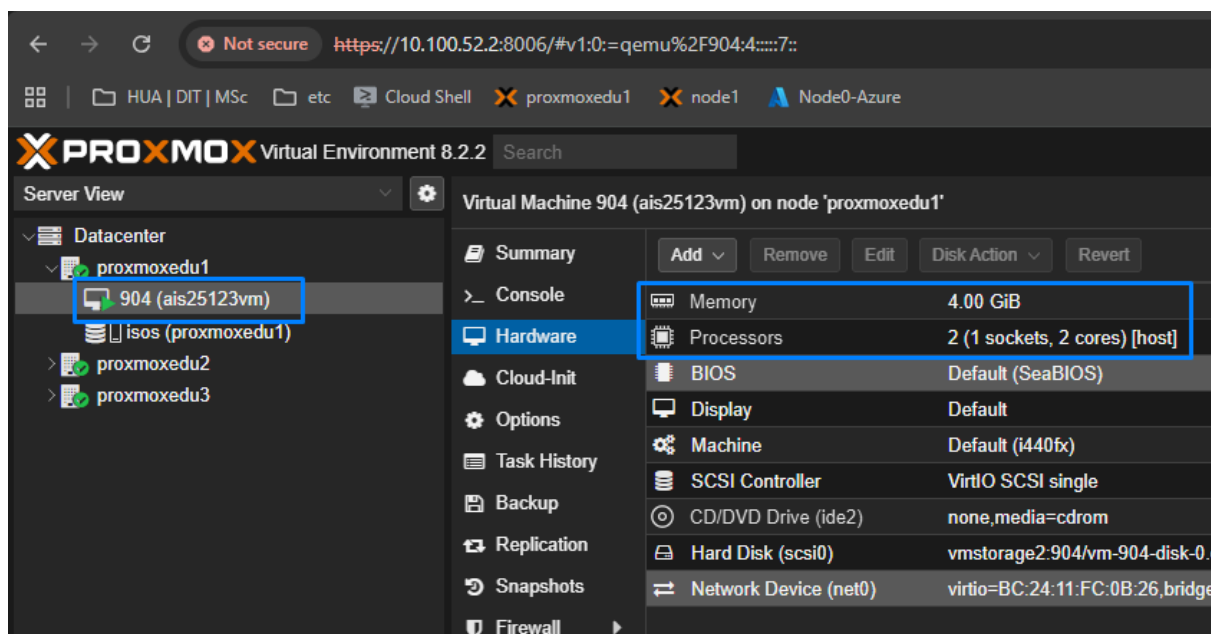
Εικόνα 3.9: Nested Virtualization αρχιτεκτονική Node 1 - Proxmox

Σημαντικό να αναφερθεί είναι το γεγονός ότι περιορίσαμε τους πόρους του vm στο node Node 1 (Layer 3) σε 1 vCPU και 1.5GB RAM, ώστε να αφήσουμε επαρκή διαθέσιμη μνήμη (2.5GB) και επεξεργαστική ισχύ για τη λειτουργία του ίδιου του Proxmox Hypervisor (Layer 2). Αποδώσαμε τους πόρους που προβλέπονταν στον αρχικό σχεδιασμό, δίνοντας έμφαση στην επεξεργαστική ισχύ για τη διαχείριση του κρυπτογραφημένου VPN tunnel.

### 3.3.1 Εγκατάσταση και Παραμετροποίηση του Hypervisor (Node 1)

Σε αυτό το στάδιο, μετατρέψαμε την εικονική μηχανή που μας παραχωρήθηκε από το εργαστήριο σε έναν αυτόνομο Hypervisor, δημιουργώντας το Layer 2 της υποδομής μας. Η επιλογή αυτή μας επέτρεψε να έχουμε τον πλήρη έλεγχο του Private Cloud περιβάλλοντος, ανεξάρτητα από τις ρυθμίσεις του κεντρικού Hypervisor της σχολής.

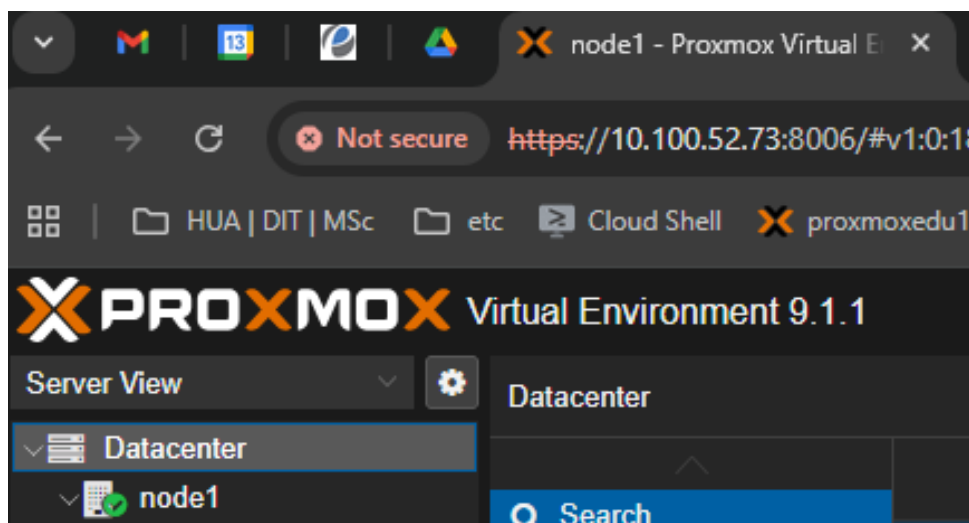
- **Πρόσβαση στην Υποδομή (Layer 1):** Αρχικά, χρησιμοποιώντας το wireguard ώστε να συνδεθούμε με το vpn της σχολής, έπειτα συνδεθήκαμε μέσω του Web Interface στην IP που μας δόθηκε από το εργαστήριο. Εκεί, εντοπίσαμε την εικονική μηχανή (VM) που είχε προ-δεσμευτεί για εμάς στον κεντρικό Hypervisor της σχολής (Proxmox VE 8.2.2).



Εικόνα 3.10: Proxmox - VM 904 στο περιβάλλον του εργαστηρίου (Layer 1)

- **Deployment του Proxmox VE 9.1.1:** Έγινε (mount) του αρχείου ISO του Proxmox και εκκίνηση της εγκατάστασης εντός του VM. Το δίκτυο έλαβε ip address και έτσι μας επέτρεψε να αποκτήσουμε πρόσβαση στο δικό μας GUI, ανεξάρτητα από το κεντρικό περιβάλλον της σχολής.



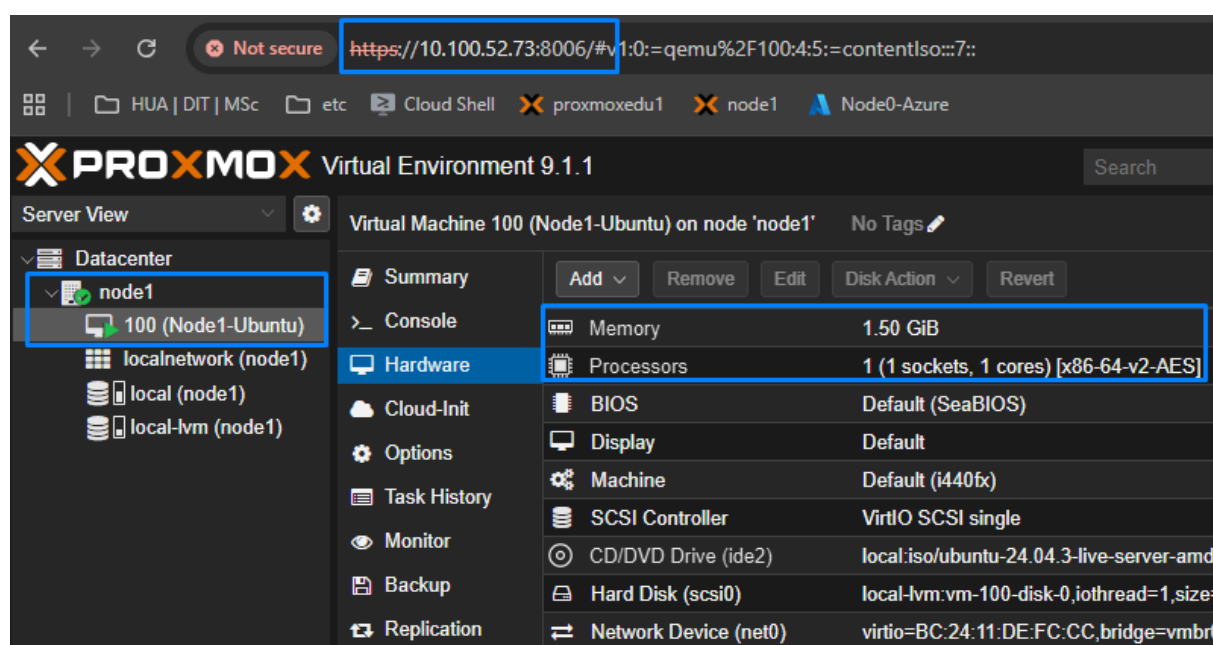


Εικόνα 3.11: Ο Nested Hypervisor (Layer 2) σε πλήρη λειτουργία μετά την εγκατάσταση.

### 3.3.2 Προετοιμασία Ubuntu server

Με τον Hypervisor (Layer 2) σε πλήρη λειτουργία, προχωρήσαμε στο επόμενο επίπεδο της αρχιτεκτονικής μας: τη δημιουργία της εικονικής μηχανής που θα φιλοξενήσει ο Node 1.

- **Καθορισμός Πόρων και VM ID:** Δημιουργήσαμε μια νέα εικονική μηχανή με ID 100. Για τον Node 1, αποδώσαμε συνειδητά 1 vCPU και 1.5GB RAM. Η επιλογή αυτή έγινε ώστε να διασφαλιστεί ότι ο Layer 2 Hypervisor θα διατηρήσει επαρκή μνήμη (2.5GB) για τις δικές του διεργασίες συστήματος, αποφεύγοντας την αστάθεια λόγω υπερκατανάλωσης πόρων (resource contention).



Εικόνα 3.12: Ρύθμιση υλικού (1 vCPU / 1.5GB RAM) για τον Node 1

- **Εγκατάσταση Λειτουργικού Συστήματος:** Ανεβάσαμε το ISO image του Ubuntu Server 24.04 LTS στον τοπικό αποθηκευτικό χώρο του Proxmox και εκκινήσαμε τη VM. Ακολούθησαμε τα βήματα του installer, παραμετροποιώντας το δίκτυο και τον χρήστη συστήματος. Η διαδικασία ολοκληρώθηκε επιτυχώς, όπως φαίνεται και στο παρακάτω screenshot, μπόρέσαμε να κάνουμε login κανονικά.

```
Ubuntu 24.04.4 LTS node1 tty1
node1 login: student
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-100-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Thu Feb 12 10:02:19 PM UTC 2026

System load:  0.23           Processes:            96
Usage of /:   33.3% of 13.67GB Users logged in:      0
Memory usage: 12%           IPv4 address for ens18: 10.100.52.234
Swap usage:   0%
```

Εικόνα 3.13: Ολοκλήρωση της εγκατάστασης του Ubuntu Server και login στο Node 1

### 3.3.3 Απομακρυσμένη διαχείριση (ssh) και ολοκλήρωση προετοιμασίας

Μετά την επιτυχή εκκίνηση του vm στον Node 1, εγκαταλείψαμε την κονσόλα του Hypervisor και μεταβήκαμε σε περιβάλλον απομακρυσμένης διαχείρισης μέσω SSH από το Windows Terminal.

```
azureuser@Node0-Azure: ~ X student@node1: ~ + v
PS C:\Users\katsi> ssh student@10.100.52.234
student@10.100.52.234's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-100-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/pro

System information as of Thu Feb 12 11:34:32 PM UTC 2026

System load:  0.0           Processes:            102
Usage of /:   33.3% of 13.67GB Users logged in:      1
Memory usage: 12%           IPv4 address for ens18: 10.100.52.234
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Thu Feb 12 23:26:24 2026 from 195.130.90.146
student@node1:~$
```

Εικόνα 3.14: Επιτυχής απομακρυσμένη σύνδεση στο vm του Node 1 μέσω ssh



Για την ολοκλήρωση του κόμβου, ακολουθήσαμε τη μεθοδολογία του **Πίνακα 4** και πέρα από τα updates που πραγματοποιήσαμε, κάναμε install και το OpenVPN ώστε και ο κόμβος αυτός να είναι έτοιμος.

Κλείνοντας αυτή την υποενότητα, σημαντικό να αναφερθεί είναι πως με την επιτυχή επαλήθευση της έκδοσης του OpenVPN και την καταγραφή της public IP (10.100.52.73), ο Node 1 είναι πλέον πλήρως παραμετροποιημένος και έτοιμος για τη φάση της διασύνδεσης. Η ολοκλήρωση αυτού του σταδίου διασφαλίζει ότι το Private Cloud περιβάλλον διαθέτει την απαραίτητη λειτουργική ομοιομορφία με τον Node 0 του Azure, επιτρέποντας την απρόσκοπτη επικοινωνία μεταξύ των δύο άκρων του tunnel.

Έχοντας οριστικοποιήσει τους δύο κύριους κόμβους της υβριδικής υποδομής και έχοντας εγκαταστήσει και το vm μέσα στον Node1, προχωρούμε στην υλοποίηση του τρίτου και τελευταίου στοιχείου του δικτύου μας, ο οποίος θα λειτουργήσει σε τοπικό επίπεδο.

### 3.4 Node 2 (Local Storage - VirtualBox)

Για την ολοκλήρωση της υποδομής, εντάξαμε τον Node 2, ο οποίος λειτουργεί σε τοπικό περιβάλλον εικονικοποίησης VirtualBox. Σε αντίθεση με τους υπόλοιπους κόμβους, ο Node 2 προϋπήρχε στον φυσικό σταθμό εργασίας, οπότε η διαδικασία επικεντρώθηκε στην προσαρμογή του για τις ανάγκες της υβριδικής υποδομής.

#### 3.4.1 Προδιαγραφές και Ενσωμάτωση Τοπικού Κόμβου

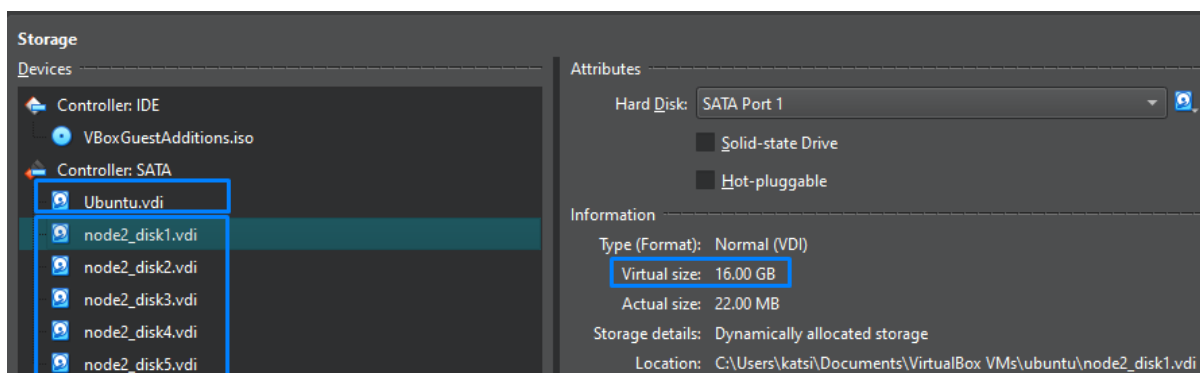
Η επιλογή του Node 2 ως προϋπάρχουσας οντότητας στο τοπικό VirtualBox έγινε με γνώμονα τη βέλτιστη αξιοποίηση των διαθέσιμων πόρων του φυσικού σταθμού εργασίας.

- **Αξιοποίηση Πόρων:** Σε αντίθεση με τους Cloud κόμβους, ο Node 2 διαθέτει τις υψηλότερες προδιαγραφές μνήμης και επεξεργαστικής ισχύος της υποδομής (βλ. **Πίνακα 3**). Η επιλογή αυτή είναι στρατηγική, καθώς το Ubuntu Desktop περιβάλλον και η ταυτόχρονη λειτουργία των υπηρεσιών Storage και DHCP απαιτούν το αυξημένο hardware footprint που προσφέρει ο συγκεκριμένος κόμβος.
- **Network Bridging:** Για την απρόσκοπτη παροχή υπηρεσιών DHCP στο τοπικό εργαστήριο, η δικτυακή διεπαφή ρυθμίστηκε σε Bridge Mode. Με αυτόν τον τρόπο, ο Node 2 αποκτά απευθείας πρόσβαση στο φυσικό δίκτυο, λειτουργώντας ως "Lab Edge" πύλη που διαχειρίζεται τη δυναμική απόδοση διευθύνσεων IP, ενώ παράλληλα προετοιμάζεται για τη διασύνδεση με το VPN tunnel.

#### 3.4.2 Υλοποίηση αρχιτεκτονικής αποθήκευσης (RAID και LVM)

Η υλοποίηση του αποθηκευτικού χώρου στον Node 2 αποτέλεσε ένα κρίσιμο στάδιο, καθώς ο κόμβος αυτός προορίζεται να λειτουργήσει ως το κεντρικό αποθετήριο δεδομένων της υποδομής. Η διαδικασία ξεκίνησε με τη διάθεση των απαραίτητων φυσικών πόρων μέσω του Hypervisor.

Συγκεκριμένα, στον ελεγκτή SATA της εικονικής μηχανής προστέθηκαν πέντε (5) επιπλέον εικονικοί δίσκοι, χωρητικότητας 16 GB έκαστος, πέραν του δίσκου που φιλοξενεί το λειτουργικό σύστημα. Η διάταξη αυτή, όπως αποτυπώνεται στις ρυθμίσεις του VirtualBox, αποτελεί τη βάση πάνω στην οποία οικοδομήθηκε η ιεραρχική δομή αποθήκευσης.



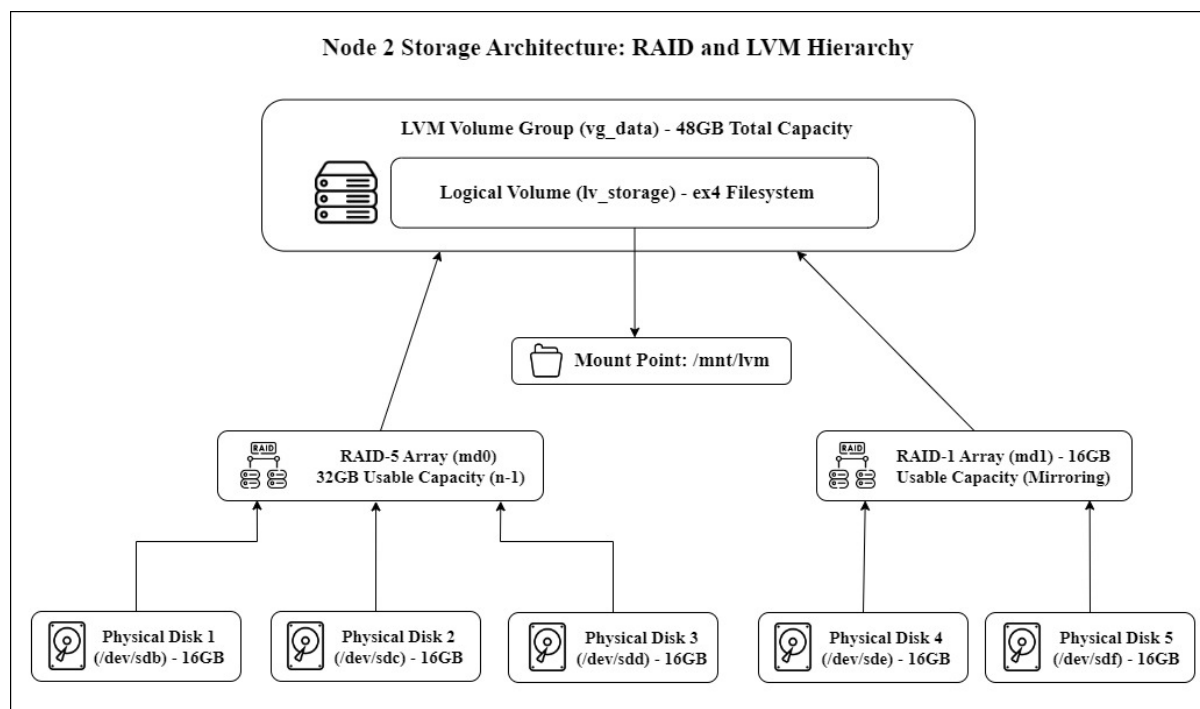
Εικόνα 3.15: Ρύθμιση των 5 εικονικών δίσκων στον ελεγκτή SATA του Node 2

Σημαντικό να αναφερθεί είναι πως πρέπει πριν προχωρήσουμε στην υλοποίηση του RAID, να επαληθεύσουμε την ορθή αναγνώριση των νέων μονάδων αποθήκευσης από το λειτουργικό σύστημα μέσω της εντολής **lsblk**. Όπως φαίνεται στην παρακάτω εικόνα, οι πέντε πρόσθετοι δίσκοι (sdb έως sdf) των 16 GB είναι διαθέσιμοι και έτοιμοι για χρήση.

```
sda      8:0    0    30G   0 disk
├─sda1   8:1    0     1M   0 part
└─sda2   8:2    0    30G   0 part /
sdb      8:16   0    16G   0 disk
sdc      8:32   0    16G   0 disk
sdd      8:48   0    16G   0 disk
sde      8:64   0    16G   0 disk
sdf      8:80   0    16G   0 disk
sr0      11:0    1  56.9M   0 rom  /media/drosos/VBox_GAs_7.1.4
drosos@kats:~$
```

Εικόνα 3.16: Λίστα αναγνωρισμένων block devices στον Node 2

Πριν την εκτέλεση των εντολών παραμετροποίησης στο τερματικό, σχεδιάστηκε η λογική αρχιτεκτονική του Storage με στόχο τον συνδυασμό της ανοχής σφαλμάτων (Redundancy) και της ευέλικτης διαχείρισης του χώρου (Abstraction). Το ακόλουθο διάγραμμα παρουσιάζει τη μετάβαση από τους φυσικούς δίσκους στις συστοιχίες RAID και την τελική ενοποίησή τους μέσω του LVM:



Εικόνα 3.17: Λογικό διάγραμμα ιεραρχίας RAID και LVM στον Node 2

Βάσει του παραπάνω σχεδιασμού, η υλοποίηση χωρίστηκε σε τρία διακριτά βήματα: την οργάνωση των δίσκων σε επίπεδο RAID (mdadm), τη δημιουργία του Volume Group (LVM) και την τελική προσάρτηση (Mounting) του συστήματος αρχείων. Στις υποενότητες που ακολουθούν, παρουσιάζονται οι εντολές και τα αποτελέσματα της υλοποίησης αυτής.

### 3.4.3 Δημιουργία Συστοιχιών RAID (md0 και md1)

Σε αυτό το σημείο, είμαστε έτοιμοι να προχωρήσουμε στην υλοποίηση των RAIDs χρησιμοποιώντας το εργαλείο mdadm. Η επιλογή του λογισμικού RAID (Software RAID) προσφέρει την απαραίτητη ευελιξία για το εικονικό μας περιβάλλον, εξασφαλίζοντας ανοχή σε σφάλματα χωρίς την ανάγκη εξειδικευμένου υλικού. Η διαδικασία υλοποιήθηκε σε δύο φάσεις, αξιοποιώντας τους πέντε εικονικούς δίσκους που δεσμεύτηκαν στο VirtualBox:

- **Δημιουργία RAID-5 (md0):** Η δημιουργία της συστοιχίας πραγματοποιήθηκε με τη χρήση του εργαλείου **mdadm**, ορίζοντας το επίπεδο RAID-5 και τις τρεις φυσικές συσκευές (`/dev/sdb`, `/dev/sdc`, `/dev/sdd`), όπως αποτυπώνεται στην παρακάτω εικόνα, ενώ η ωφέλιμη χωρητικότητα διαμορφώνεται στα 32 GB.

```
drosos@kats:~$ sudo mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb /dev/sdc /dev/sdd
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
```

Εικόνα 3.18: Εκτέλεση εντολών δημιουργίας του RAID-5 (md0)

- **Δημιουργία RAID-1 (md1):** Αντίστοιχα, για το RAID-1, επιστρατεύτηκαν οι δύο εναπομείναντες δίσκοι (`/dev/sde` και `/dev/sdf`) σε διάταξη mirroring, εξασφαλίζοντας ότι τα δεδομένα παραμένουν διαθέσιμα ακόμη και με την απώλεια του ενός εκ των δύο δίσκων, όπου η επιτυχή έναρξη της συστοιχίας να επιβεβαιώνεται στην παρακάτω εικόνα.

```
drosos@kats:~$ sudo mdadm --create /dev/md1 --level=1 --raid-devices=2 /dev/sde /dev/sdf
mdadm: Note: this array has metadata at the start and
may not be suitable as a boot device. If you plan to
store '/boot' on this device please ensure that
your boot-loader understands md/v1.x metadata, or use
--metadata=0.90
Continue creating array? y
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md1 started.
```

Εικόνα 3.19: Εκτέλεση εντολών δημιουργίας του RAID-1 (md1)

Στην συνέχεια, για την οριστική επιβεβαίωση της ορθής λειτουργίας και του συγχρονισμού των νέων RAIDs, πραγματοποιήθηκε έλεγχος της κατάστασης του πυρήνα (kernel) μέσω του αρχείου συστήματος `/proc/mdstat`. Το βήμα αυτό είναι κρίσιμο, καθώς διασφαλίζει ότι οι μονάδες `md0` και `md1` είναι πλήρως λειτουργικές (active) και συγχρονισμένες πριν χρησιμοποιηθούν ως βάση για το επόμενο επίπεδο αφαίρεσης.

```
drosos@kats:~$ cat /proc/mdstat
Personalities : [raid6] [raid5] [raid4] [raid1]
md1 : active raid1 sdf[1] sde[0]
      16759808 blocks super 1.2 [2/2] [UU]

md0 : active raid5 sdd[3] sdc[1] sdb[0]
      33519616 blocks super 1.2 level 5, 512k chunk, algorithm 2 [3/3] [UUU]

unused devices: <none>
```

Εικόνα 3.20: Επιβεβαίωση της κατάστασης 'active' για τις συστοιχίες RAID-5 και RAID-1

Όπως παρατηρείται στην παραπάνω εικόνα, το σύστημα αναγνωρίζει κανονικά τις δύο συστοιχίες, με την ένδειξη [UUU] και [UU] να επιβεβαιώνει ότι όλοι οι συμμετέχοντες δίσκοι είναι ενεργοί. Με τη βάση της αποθηκευτικής υποδομής πλέον σταθερή, η διαδικασία περνά στο επίπεδο του Logical Volume Management (LVM), όπου οι δύο διακριτές συστοιχίες θα ενοποιηθούν σε ένα κοινό Volume Group.

#### 3.4.4 Υλοποίηση Logical Volume Management (LVM)

Αφού επιβεβαιώθηκε η σταθερότητα των συστοιχιών RAID, προχωρήσαμε στην εγκατάσταση των απαραίτητων εργαλείων διαχείρισης (`lvm2`) και στην υλοποίηση του επιπέδου αφαίρεσης (Abstraction Layer) μέσω LVM. Η διαδικασία αυτή επέτρεψε την ενοποίηση των δύο RAID συσκευών σε μία κοινή λογική οντότητα.

- **Προετοιμασία Physical Volumes (PV):** Μετατροπή των RAID συσκευών σε αναγνωρίσιμες μονάδες από το LVM μέσω της εντολής `pvcreeate`.
- **Δημιουργία Volume Group (VG):** Συγχώνευση των δύο PVs στο group με την ονομασία `vg_data`.
- **Δημιουργία Logical Volume (LV):** Απόδοση όλου του διαθέσιμου χώρου (48 GB) στον τόμο `lv_storage`.

```
drosos@kats:~$ sudo pvcreate /dev/md0 /dev/md1
Physical volume "/dev/md0" successfully created.
Physical volume "/dev/md1" successfully created.
drosos@kats:~$ sudo vgcreate vg_data /dev/md0 /dev/md1
Volume group "vg_data" successfully created
drosos@kats:~$ sudo lvcreate -l 100%FREE -n lv_storage vg_data
Logical volume "lv_storage" created.
drosos@kats:~$ sudo mkfs.ext4 /dev/vg_data/lv_storage
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 12568576 4k blocks and 3145728 inodes
Filesystem UUID: ba1c7476-cd6a-4f72-911c-4045c5fe6865
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632, 2654208,
    4096000, 7962624, 11239424

Allocating group tables: done
Writing inode tables: done
Creating journal (65536 blocks): done
Writing superblocks and filesystem accounting information: done

drosos@kats:~$ sudo mkdir -p /mnt/lvm
drosos@kats:~$ sudo mount /dev/vg_data/lv_storage /mnt/lvm
drosos@kats:~$ df -h | grep lvm
/dev/mapper/vg_data-lv_storage 47G 24K 45G 1% /mnt/lvm
```

Εικόνα 3.21: Υλοποίηση LVM, διαμόρφωση και προσάρτηση

Η επιτυχής υλοποίηση της αρχιτεκτονικής επιβεβαιώνεται από την εντολή `df -h`, όπου ο λογικός τόμος `lv_storage` εμφανίζεται με ωφέλιμη χωρητικότητα 47 GB (μετά την αφαίρεση του filesystem overhead), επαληθεύοντας πλήρως τον υπολογισμό των 48 GB (32GB + 16GB) από τις συστοιχίες RAID.

Ως τελικό βήμα για την οριστικοποίηση του Storage Node, διασφαλίσαμε τη μονιμότητα (persistence) της προσάρτησης μέσω της ενημέρωσης του αρχείου `/etc/fstab`. Η ενέργεια αυτή εγγυάται ότι ο Node 2 θα αναγνωρίζει αυτόματα τον αποθηκευτικό χώρο σε κάθε επανεκκίνηση, καθιστώντας τον πλέον έτοιμο να εξυπηρετήσει την υβριδική υποδομή.

```
drosos@kats:~$ echo '/dev/vg_data/lv_storage /mnt/lvm ext4 defaults 0 0' | sudo tee -a /etc/fstab
/dev/vg_data/lv_storage /mnt/lvm ext4 defaults 0 0
```

Εικόνα 3.22: Εντολή εγγραφής στο `/etc/fstab` για την αυτόματη προσάρτηση του `lv_storage`

Η χρήση του LVM (Logical Volume Management) υπήρξε καθοριστική, καθώς επέτρεψε την ενοποίηση δύο διαφορετικών τεχνολογιών RAID (RAID 1 και RAID 5) σε ένα κοινό Volume Group (vg\_data). Με αυτόν τον τρόπο, δημιουργήθηκε ένα ενιαίο Logical Volume (lv\_storage) μεγέθους 48GB, το οποίο συνδυάζει την υψηλή ασφάλεια του RAID 1 με την αποδοτικότητα χώρου του RAID 5, προσφέροντας παράλληλα τη δυνατότητα μελλοντικής επέκτασης του συστήματος αρχείων χωρίς διακοπή των υπηρεσιών

Πίνακας 5: Σύγκριση και Ρόλος Τεχνολογιών Storage στον Node 2

| Τεχνολογία    | Μέθοδος                                                  | Ανοχή (Safety)                                         | Space         | Πλεονέκτημα                                           |
|---------------|----------------------------------------------------------|--------------------------------------------------------|---------------|-------------------------------------------------------|
| <b>RAID 1</b> | Mirroring σε 2 δίσκους                                   | Υψηλή - Αντέχει την ολική καταστροφή ενός δίσκου.      | 50%           | Μέγιστη ασφάλεια για κρίσιμα αρχεία συστήματος.       |
| <b>RAID 5</b> | Striping με Parity (Διαισθησμός σε 3+ δίσκους).          | Καλή: Αντέχει την αστοχία ενός οποιουδήποτε δίσκου.    | $N - 1$ (66%) | Ισορροπία μεταξύ ταχύτητας και χωρητικότητας.         |
| <b>LVM</b>    | Virtualization Layer (Ενοποίηση RAID arrays σε Volumes). | Εξαρτάται από το υποκείμενο RAID (προσθέτει ευελιξία). | ~100%         | Δυναμική διαχείριση: Ένωση ανόμοιων RAID (md0 + md1). |

Κλείνοντας αυτό το κεφάλαιο, μπορούμε να αναφέρουμε ότι με την ολοκλήρωση της προετοιμασίας και των τριών κόμβων, η υποδομή διαθέτει πλέον τους απαραίτητους πόρους σε Azure, Proxmox και VirtualBox. Πλέον, το ενδιαφέρον στρέφεται στο Κεφάλαιο 4, όπου θα υλοποιήσουμε το ασφαλές δίκτυο VPN που θα ενώσει αυτούς τους απομονωμένους πόρους σε ένα ενιαίο, λειτουργικό σύνολο.

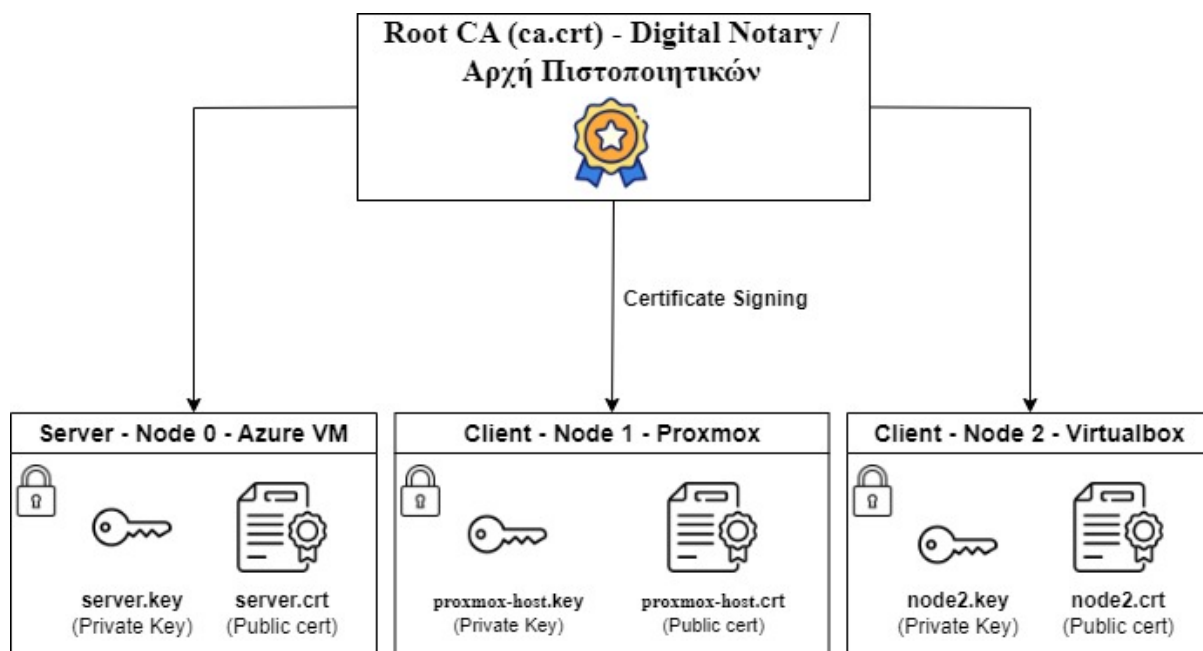


## 4 Κεφάλαιο 4: Ενοποίηση Δικτύου και VPN

Στο κεφάλαιο αυτό υλοποιείται η οργανική διασύνδεση των κόμβων σε ένα κοινό εικονικό δίκτυο (Site-to-Cloud). Χρησιμοποιείται η αρχιτεκτονική Hub-and-Spoke, με τον Node 0 (Azure) ως κεντρικό εξυπηρετητή και τους Node 1 (Proxmox), Node 2 (Laptop) ως απομακρυσμένα άκρα. Η χρήση TAP Mode (Layer 2 Bridging) εξασφαλίζει ενιαίο broadcast domain, επιτρέποντας στον DHCP server του Node 2 να αποδίδει διευθύνσεις σε όλο το εύρος της υποδομής.

### 4.1 Αρχιτεκτονική PKI και ο ρόλος της CA

Η διασύνδεση των κόμβων βασίζεται στο μοντέλο **Public Key Infrastructure (PKI)** για την εξασφάλιση αμφίδρομης ταυτοποίησης (Mutual Authentication). Η Αρχή Πιστοποιητικών (CA) λειτουργεί ως ο «ψηφιακός συμβολαιογράφος» της υποδομής, υπογράφοντας τα δημόσια πιστοποιητικά (.cert) των κόμβων, ενώ τα ιδιωτικά κλειδιά (.key) παραμένουν αυστηρά απόρρητα σε κάθε τοπικό δίσκο.

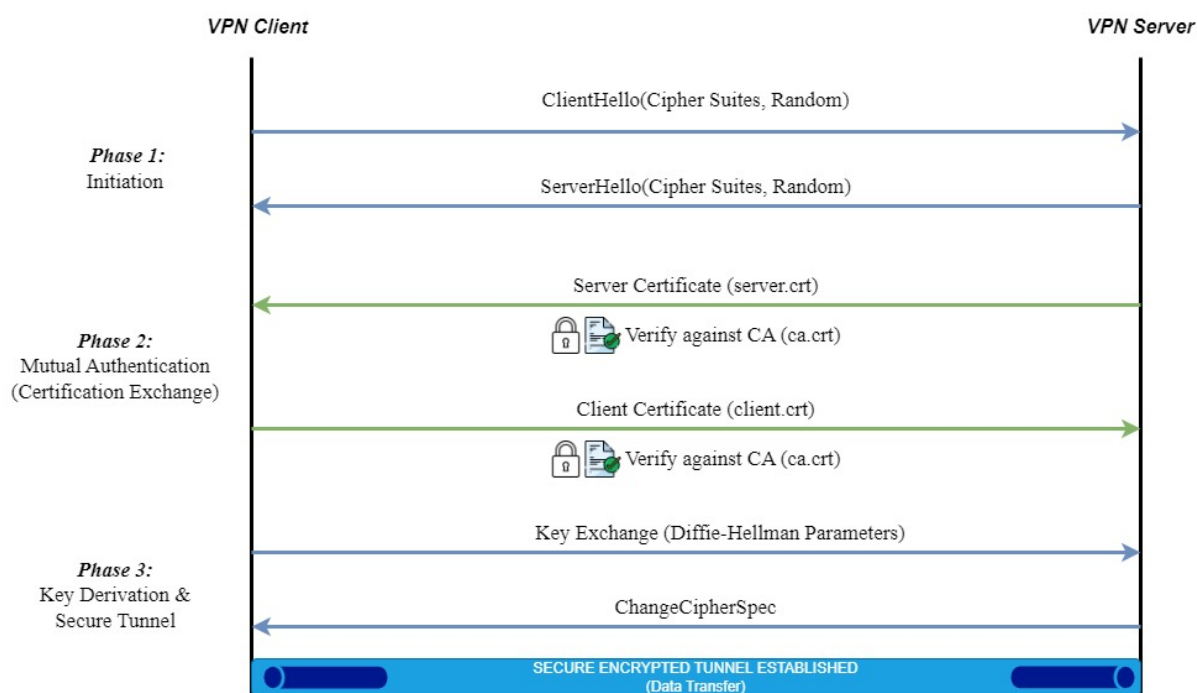


Εικόνα 4.1: Ιεραρχία εμπιστοσύνης και χάρτης διανομής κρυπτογραφικών αρχείων.

Η ύπαρξη αυτών των κρυπτογραφικών ταυτοτήτων επιτρέπει στους κόμβους να περάσουν από το επίπεδο του σχεδιασμού στο επίπεδο της ζωντανής επικοινωνίας μέσω της διαδικασίας του **handshake**.

Κατά τη φάση του handshake, οι κόμβοι ανταλλάσσουν τα υπογεγραμμένα πιστοποιητικά τους. Εφόσον όλοι οι συμμετέχοντες εμπιστεύονται την ίδια κοινή Αρχή (Root CA), επιβεβαιώνεται η εγκυρότητα της ταυτότητάς τους και επιτρέπεται η εγκαθίδρυση του κρυπτογραφημένου tunnel. Η αρχιτεκτονική αυτή αποτρέπει επιθέσεις τύπου Man-in-the-Middle, καθώς κανένας κόμβος δεν μπορεί να εισέλθει στο δίκτυο χωρίς έγκυρη υπογραφή από την CA.

### OpenVPN TLS Handshake and Mutual Authentication Process



Εικόνα 4.2: Φάσεις αμφίδρομης ταυτοποίησης και εγκαθίδρυσης ασφαλούς συνόδου.

- Φάση 1 (Hello): Συμφωνία αλγορίθμων κρυπτογράφησης.
- Φάση 2 (Auth): Ανταλλαγή και έλεγχος των .crt μέσω της CA.
- Φάση 3 (Tunnel): Δημιουργία των session keys και έναρξη ασφαλούς ροής δεδομένων.

## 4.2 Node 0: Υλοποίηση VPN Server και διαμόρφωση virtual bridge

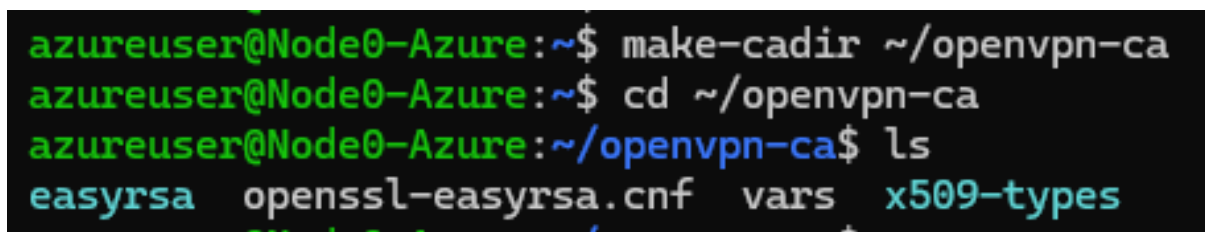
Στην ενότητα αυτή περιγράφεται η παραμετροποίηση του Node 0 (Azure VM), ο οποίος αποτελεί τον κεντρικό πυλώνα (Hub) της Site-to-Cloud αρχιτεκτονικής. Ο ρόλος του είναι διπλός: αρχικά λειτουργεί ως η Αρχή Πιστοποιητικών (CA) για την έκδοση των κρυπτογραφικών κλειδιών που αναλύθηκαν στην ενότητα 4.1, και ακολούθως φιλοξενεί την εικονική γέφυρα (bridge) br0. Η γέφυρα αυτή είναι απαραίτητη για την υποστήριξη του Ethernet Bridging, καθώς λειτουργεί ως το σημείο υποδοχής της διεπαφής tap0, επιτρέποντας στον Server να διαχειρίζεται την κυκλοφορία Layer 2 (broadcast frames) που καταφθάνει από τους απομακρυσμένους Clients.



#### 4.2.1 Ρύθμιση της υποδομής PKI και δημιουργία της CA

Στον Node 0 (Azure), η διαδικασία ξεκίνησε με την προετοιμασία της υποδομής δημοσίου κλειδιού (Public Key Infrastructure - PKI) μέσω του εργαλείου **Easy-RSA**. Ο στόχος ήταν η δημιουργία της Αρχής Πιστοποιητικών (Root CA), η οποία θα αποτελέσει το θεμέλιο εμπιστοσύνης για όλο το ενοποιημένο δίκτυο. Συνεπώς τα τεχνικά βήματα υλοποίησης που ακολουθήσαμε είναι τα εξής:

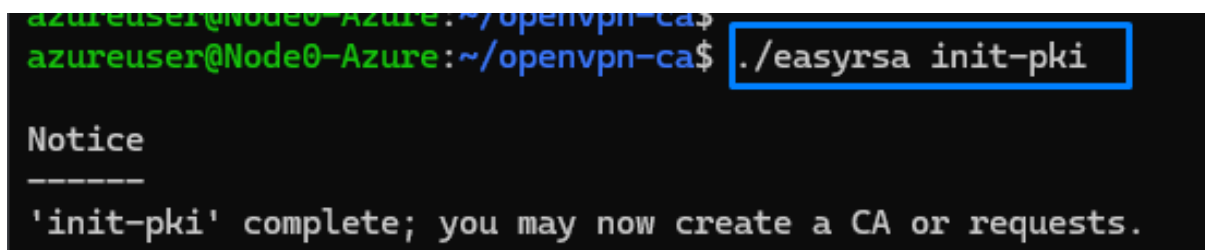
- **Προετοιμασία work directory:** Αρχικά, χρησιμοποιήθηκε το εργαλείο `make-cadir` για τη δημιουργία ενός αποκλειστικού καταλόγου εργασίας για την Αρχή Πιστοποιητικών (CA). Η χρήση αυτού του εργαλείου εξασφαλίζει ότι όλα τα απαραίτητα πρότυπα ρυθμίσεων και τα script του Easy-RSA βρίσκονται συγκεντρωμένα.



```
azureuser@Node0-Azure:~$ make-cadir ~/openvpn-ca
azureuser@Node0-Azure:~$ cd ~/openvpn-ca
azureuser@Node0-Azure:~/openvpn-ca$ ls
easyrsa  openssl-easyrsa.cnf  vars  x509-types
```

Εικόνα 4.3: Δημιουργία του dir `openvpn-ca` και προετοιμασία του περιβάλλοντος Easy-RSA στον Node 0

- **Αρχικοποίηση του PKI:** Πραγματοποιήθηκε η αρχικοποίηση της δομής του PKI, η οποία δημιουργεί τους απαραίτητους υποκαταλόγους για την αποθήκευση των κλειδιών και των πιστοποιητικών που θα εκδοθούν στη συνέχεια.



```
azureuser@Node0-Azure:~/openvpn-ca$ ./easyrsa init-pki
Notice
-----
'init-pki' complete; you may now create a CA or requests.
```

Εικόνα 4.4: Δημιουργία της δομής του PKI

- **Δημιουργία της Root CA:** Το σημαντικότερο βήμα ήταν η δημιουργία του πιστοποιητικού ρίζας (ca.crt) και του ιδιωτικού κλειδιού της Αρχής (ca.key). Το πιστοποιητικό αυτό αποτελεί τη βάση της εμπιστοσύνης για όλο το VPN δίκτυο. Κατά τη διαδικασία, ορίστηκε το Common Name της αρχής ως Easy-RSA CA.

[illegible]

Εικόνα 4.5: Επιτυχής δημιουργία του πιστοποιητικού της Αρχής Πιστοποιητικών (CA)

#### 4.2.2 Έκδοση πιστοποιητικών server

Σε αυτή τη φάση πραγματοποιήθηκε η έκδοση της «ψηφιακής ταυτότητας» του Node 0. Με την ολοκλήρωση αυτού του σταδίου, ο εξυπηρετητής αποκτά τα απαραίτητα κρυπτογραφικά κλειδιά για την αυθεντικοποίηση των συνδέσεων και την κρυπτογράφηση των δεδομένων. Για τον λόγο αυτόν, εργαστήκαμε ως εξής:

- **Δημιουργία αίτησης πιστοποιητικού (Certificate Request):** Παρήχθη το ιδιωτικό κλειδί του εξυπηρετητή (server.key) και η αντίστοιχη αίτηση υπογραφής πιστοποιητικού (server.req). Η χρήση της παραμέτρου nopass διασφαλίζει ότι η υπηρεσία OpenVPN θα μπορεί να εκκινεί αυτόματα χωρίς χειροκίνητη παρέμβαση.

```
azureuser@Node0-Azure:~/openvpn-ca$ ./easysrsa gen-req server nopass
Using Easy-RSA 'vars' configuration:
* /home/azureuser/openvpn-ca/vars

Using SSL:
* openssl OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan
-----
Common Name (eg: your user, host, or server name) [server]:

Notice
-----
Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /home/azureuser/openvpn-ca/pki/reqs/server.req
* key: /home/azureuser/openvpn-ca/pki/private/server.key

azureuser@Node0-Azure:~/openvpn-ca$ |
```

Εικόνα 4.6: Επιτυχής δημιουργία των αρχείων Private-Key και Public-Certificate-Request για τον Node 0

- **Ψηφιακή υπογραφή από την CA:** Η αίτηση υποβλήθηκε στην Αρχή Πιστοποίησης (CA) του Node 0, η οποία την επαλήθευσε και εξέδωσε το τελικό δημόσιο πιστοποιητικό server.crt. Το αρχείο αυτό αποθηκεύτηκε στη διαδρομή /pki/issued/server.crt.

```
azureuser@Node0-Azure:~/openvpn-ca$ ./easysrsa sign-req server server
Using Easy-RSA 'vars' configuration:
* /home/azureuser/openvpn-ca/vars

Notice
-----
Certificate created at:
* /home/azureuser/openvpn-ca/pki/issued/server.crt

azureuser@Node0-Azure:~/openvpn-ca$ |
```

Εικόνα 4.7: Επιτυχής ψηφιακή υπογραφή και έκδοση του πιστοποιητικού εξυπηρετητή.

- **Παραγωγή παραμέτρων ασφαλείας:** Στην συνέχεια, ολοκληρώθηκε και η δημιουργία των παραμέτρων Diffie-Hellman (dh.pem), μεγέθους 2048 bit, για τη διασφάλιση της ανταλλαγής κλειδιών συνεδρίας. Παράλληλα, εκτελέστηκε η παραγωγή του κλειδιού ta.key για την ενεργοποίηση του TLS-Auth (HMAC Firewall), παρέχοντας ένα επιπλέον επίπεδο προστασίας έναντι επιθέσεων DoS και μη εξουσιοδοτημένων προσπαθειών σύνδεσης.

```

azureuser@Node0-Azure:~/openvpn-ca$ ./easysrsa gen-dh
Using Easy-RSA 'vars' configuration:
* /home/azureuser/openvpn-ca/vars

DH parameters of size 2048 created at:
* /home/azureuser/openvpn-ca/pki/dh.pem

azureuser@Node0-Azure:~/openvpn-ca$ |

```

Εικόνα 4.8: Παραγωγή παραμέτρων Diffie-Hellman και κλειδιού TLS-Auth στον Node 0.

- **Ενίσχυση ασφάλειας (HMAC Signature):** Πραγματοποιήθηκε η παραγωγή του κλειδιού ta.key για την υλοποίηση ενός πρόσθετου επιπέδου ασφάλειας, γνωστού ως HMAC Firewall.
- **Προστασία από επιθέσεις:** Το συγκεκριμένο κλειδί παρέχει προστασία πέρα από αυτή που προσφέρει το SSL/TLS, ενισχύοντας τη δυνατότητα επαλήθευσης της ακεραιότητας των πακέτων.
- **Λειτουργία firewall:** Με τη χρήση του ta.key, ο εξυπηρετητής μπορεί να απορρίπτει άμεσα μη εξουσιοδοτημένα πακέτα πριν καν ξεκινήσει η υπολογιστικά δαπανηρή διαδικασία της κρυπτογράφησης, προστατεύοντας έτσι την υποδομή από επιθέσεις άρνησης υπηρεσίας (DoS).

```

azureuser@Node0-Azure:~/openvpn-ca$ openvpn --genkey --secret ta.key
2026-02-15 12:54:23 DEPRECATED OPTION: The option --secret is deprecated.
2026-02-15 12:54:23 WARNING: Using --genkey --secret filename is DEPRECATED. Use --genkey secret filename
azureuser@Node0-Azure:~/openvpn-ca$ ls -l ta.key
-rw----- 1 azureuser azureuser 636 Feb 15 12:54 ta.key
azureuser@Node0-Azure:~/openvpn-ca$

```

Εικόνα 4.9: Παραγωγή του κλειδιού ta.key για την ενίσχυση της ακεραιότητας και τη δημιουργία HMAC firewall στον OpenVPN server

#### 4.2.3 Παραμετροποίηση και ρύθμιση της υπηρεσίας OpenVPN

Στην παρούσα υποενότητα περιγράφεται η διαδικασία παραμετροποίησης της υπηρεσίας OpenVPN στον Node 0. Μετά την επιτυχή έκδοση των κρυπτογραφικών κλειδιών, είναι απαραίτητη η μεταφορά τους στον κατάλογο λειτουργίας του συστήματος. Η κίνηση αυτή εξασφαλίζει ότι ο δαίμονας της υπηρεσίας έχει άμεση πρόσβαση στα απαραίτητα αρχεία για την επικύρωση της ταυτότητας των χρηστών.

```

azureuser@Node0-Azure:~/openvpn-ca$ sudo cp ~/openvpn-ca/pki/ca.crt /etc/openvpn/
azureuser@Node0-Azure:~/openvpn-ca$ sudo cp ~/openvpn-ca/pki/issued/server.crt /etc/openvpn/
azureuser@Node0-Azure:~/openvpn-ca$ sudo cp ~/openvpn-ca/pki/private/server.key /etc/openvpn/
azureuser@Node0-Azure:~/openvpn-ca$ sudo cp ~/openvpn-ca/pki/dh.pem /etc/openvpn/
azureuser@Node0-Azure:~/openvpn-ca$ sudo cp ~/openvpn-ca/ta.key /etc/openvpn/
azureuser@Node0-Azure:~/openvpn-ca$ ls -l /etc/openvpn/
total 36
-rw----- 1 root root 1204 Feb 15 14:54 ca.crt
drwxr-xr-x 2 root root 4096 Nov 24 22:32 client
-rw----- 1 root root 424 Feb 15 14:54 dh.pem
drwxr-xr-x 2 root root 4096 Nov 24 22:32 server
-rw----- 1 root root 4609 Feb 15 14:54 server.crt
-rw----- 1 root root 1704 Feb 15 14:54 server.key
-rw----- 1 root root 636 Feb 15 14:55 ta.key
-rwxr-xr-x 1 root root 1468 Nov 24 22:32 update-resolv-conf
azureuser@Node0-Azure:~/openvpn-ca$

```

Εικόνα 4.10: Μεταφορά των κρυπτογραφικών κλειδιών στον κατάλογο λειτουργίας /etc/openvpn/ και επαλήθευση της ακεραιότητας του καταλόγου.

Στη συνέχεια, πραγματοποιείται η δημιουργία του αρχείου ρυθμίσεων server.conf. Η παραμετροποίηση αυτή αποτελεί το κρίσιμότερο στάδιο, καθώς καθορίζει τη λειτουργία του εξυπηρετητή σε Tap Mode (Layer 2), δημιουργώντας την εικονική γέφυρα επικοινωνίας με την τοπική υποδομή.

```

cat: /etc/openvpn-ca/server.conf: No such file or directory
azureuser@Node0-Azure:~/openvpn-ca$ cat /etc/openvpn/server.conf
port 1194
proto udp
dev tap
mode server
tls-server
ca /etc/openvpn/ca.crt
cert /etc/openvpn/server.crt
key /etc/openvpn/server.key
dh /etc/openvpn/dh.pem
tls-auth /etc/openvpn/ta.key 0
cipher AES-256-GCM
auth SHA256
user nobody
group nogroup
persist-key
persist-tun
status openvpn-status.log
verb 3

```

Εικόνα 4.11: Το αρχείο παραμέτρων server.conf.

Οι κρίσιμες παράμετροι που διαφοροποιούν την παρούσα υλοποίηση από μια τυπική εγκατάσταση VPN αναλύονται ως εξής:

- Interface Type (dev tap): Σε αντίθεση με τη λειτουργία tun (Routed IP), η επιλογή tap είναι απαραίτητη για τη δημιουργία του εικονικού Bridge. Επιτρέπει τη διακίνηση Ethernet frames, καθιστώντας το tunnel διαφανές στα πρωτόκολλα του επιπέδου 2.
- Cryptographic Hardening: \* Ενσωμάτωση των ca.crt, server.crt και server.key για την αμοιβαία ταυτοποίηση (mutual authentication).
  - Χρήση του tls-auth ta.key 0 ως επιπλέον επίπεδο ασφαλείας (HMAC signature), το οποίο αποτρέπει επιθέσεις DoS και port scanning, απορρίπτοντας πακέτα που

δεν φέρουν τη σωστή υπογραφή πριν καν ξεκινήσει η διαδικασία του SSL/TLS handshake.

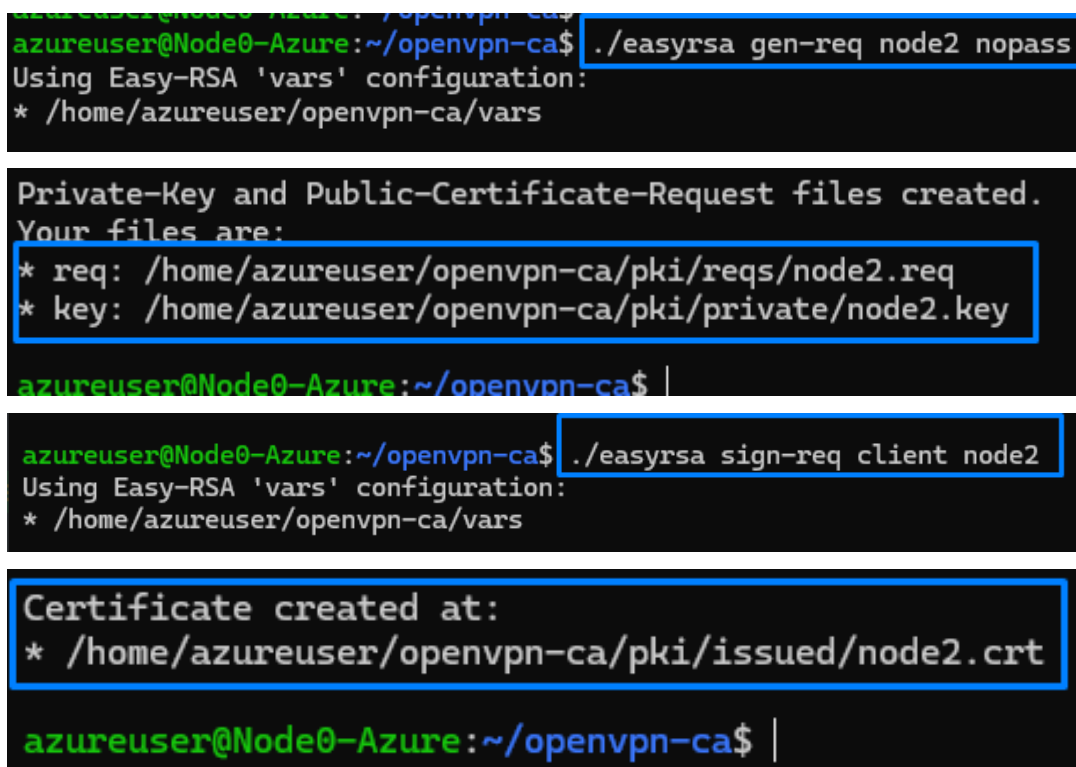
- Bridge Mode Orientation: Η σκόπιμη παράλειψη της οδηγίας server [network] [mask] μετατρέπει τον Node 0 σε έναν "διαφανή διακομιστή". Με αυτόν τον τρόπο, ο Server δεν παρεμβαίνει στην απόδοση διευθύνσεων, επιτρέποντας στον Node 2 (Laptop) να λειτουργήσει ως ο αποκλειστικός DHCP Server του ενιαίου εικονικού δικτύου.

Με την αποθήκευση του αρχείου ρυθμίσεων, ο OpenVPN Server στον Node 0 είναι πλέον αρχιτεκτονικά έτοιμος. Διαθέτει την απαραίτητη "ταυτότητα" (πιστοποιητικά) και τους "κανόνες λειτουργίας" (config). Ολοκληρώνοντας αυτό το στάδιο, η υποδομή στο Azure είναι έτοιμη να δεχτεί το εικονικό κανάλι επικοινωνίας.

#### 4.2.4 Έκδοση πιστοποιητικών clients και μεταφορά αρχείων (Node 1 και Node 2)

Με τον OpenVPN Server στον Node 0 να βρίσκεται σε πλήρη λειτουργική ετοιμότητα, το επόμενο στάδιο αφορά τη δημιουργία των ψηφιακών ταυτοτήτων για τους απομακρυσμένους κόμβους του δικτύου. Στο πλαίσιο της υποδομής PKI (Public Key Infrastructure), ο Node 0 αναλαμβάνει τον ρόλο της Αρχής Πιστοποιητικών (CA), η οποία εκδίδει και υπογράφει τα απαραίτητα πιστοποιητικά για τον Node 1 (Proxmox VM) και τον Node 2 (Virtualbox).

Η διαδικασία αυτή διασφαλίζει την **αμοιβαία αυθεντικοποίηση (Mutual Authentication)**: ο Server θα δέχεται συνδέσεις μόνο από κόμβους που κατέχουν έγκυρο «διαβατήριο» υπογεγραμμένο από τη δική του CA, προστατεύοντας το δίκτυο από μη εξουσιοδοτημένη πρόσβαση.



```
azureuser@Node0-Azure:~/openvpn-ca$ ./easysrsa gen-req node2 nopass
Using Easy-RSA 'vars' configuration:
* /home/azureuser/openvpn-ca/vars

Private-Key and Public-Certificate-Request files created.
Your files are:
* req: /home/azureuser/openvpn-ca/pki/reqs/node2.req
* key: /home/azureuser/openvpn-ca/pki/private/node2.key

azureuser@Node0-Azure:~/openvpn-ca$ |

azureuser@Node0-Azure:~/openvpn-ca$ ./easysrsa sign-req client node2
Using Easy-RSA 'vars' configuration:
* /home/azureuser/openvpn-ca/vars

Certificate created at:
* /home/azureuser/openvpn-ca/pki/issued/node2.crt

azureuser@Node0-Azure:~/openvpn-ca$ |
```

Εικόνα 4.12: Παραγωγή και υπογραφή του κρυπτογραφικού πιστοποιητικού για τον Node 2 από την κεντρική CA του Node 0.



**Σημείωση (Node 1)**

Ακολουθώντας την ίδια ακριβώς μεθοδολογία, εκδόθηκε το πιστοποιητικό και για τον Node 1 (Proxmox Hypervisor). Η ενέργεια αυτή πραγματοποιήθηκε ώστε η υποδομή PKI να είναι έτοιμη για τη διασύνδεση και του ακαδημαϊκού κόμβου. Παράλληλα έγιναν οι μεταφορές των αρχείων και για τον Node2.

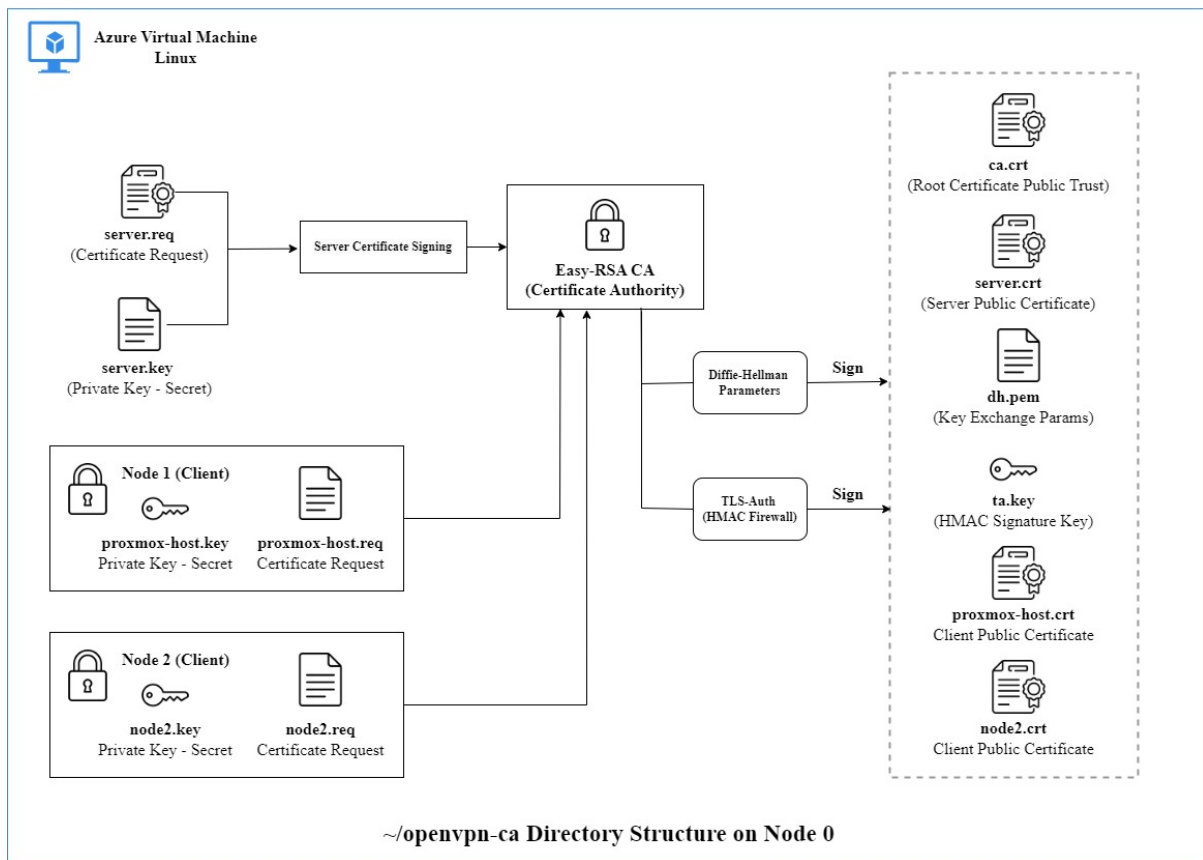
```
root@node1:~# ls -la /etc/openvpn/client/
total 32
drwxr-xr-x 2 root root 4096 Feb 24 00:38 .
drwxr-xr-x 4 root root 4096 Feb 19 01:02 ..
-rw-r--r-- 1 root root 1204 Feb 19 00:55 ca.crt
-rw-r--r-- 1 root root 4502 Feb 19 00:54 proxmox-host.crt
-rw-r--r-- 1 root root 1704 Feb 19 00:55 proxmox-host.key
-rw-r--r-- 1 root root 367 Feb 24 00:38 proxmox_vpn.conf
-rw-r--r-- 1 root root 636 Feb 19 00:56 ta.key
root@node1:~# |

drosos@kats:~$ ls -la /etc/openvpn/client/
total 32
drwxr-xr-x 2 root root 4096 Feb 18 17:36 .
drwxr-xr-x 4 root root 4096 Feb 3 17:42 ..
-rw-r--r-- 1 root root 1204 Feb 18 17:35 ca.crt
-rw-r--r-- 1 root root 232 Feb 18 17:36 client.conf
-rw-r--r-- 1 root root 4487 Feb 18 17:35 node2.crt
-rw-r--r-- 1 root root 1704 Feb 18 17:35 node2.key
-rw-r--r-- 1 root root 636 Feb 18 17:35 ta.key
drosos@kats:~$ |
```

Εικόνα 4.13: Node 1/Node2: Επαλήθευση αποθήκευσης των παραχθέντων αρχείων στον κατάλληλο κατάλογο.

Ολοκληρώνοντας τις παραπάνω διαδικασίες, ο Node 0 (Azure VPN Server) διαθέτει πλέον μια πλήρως οργανωμένη υποδομή πιστοποιητικών και κλειδιών ασφαλείας. Το παρακάτω διάγραμμα (Εικόνα 4.14) συνοψίζει τη λογική συσχέτιση μεταξύ της Αρχής Πιστοποιητικών (CA), των πιστοποιητικών του εξυπηρετητή και των πρόσθετων παραμέτρων ασφαλείας στον κατάλογο εργασίας του συστήματος.

## Node 0



Εικόνα 4.14: Διάγραμμα της δομής PKI και της ιεραρχίας των κρυπτογραφικών αρχείων στον Node 0.

### 4.3 Node 1: Παραμετροποίηση VPN Client (Proxmox)

Με την υποδομή στον Node 0 να βρίσκεται σε αναμονή, το επόμενο στάδιο αφορά τη ρύθμιση του Node 1 (Proxmox Hypervisor) ως πελάτη (client) του VPN. Η παραμετροποίηση αυτή είναι κρίσιμη, καθώς ο Proxmox πρέπει να λειτουργήσει ως γέφυρα μεταξύ των τοπικών εικονικών μηχανών (VMs) και του απομακρυσμένου διακομιστή στο Azure.

Η ρύθμιση πραγματοποιείται μέσω του αρχείου `proxmo_vpn.conf`. Το αρχείο ρυθμίσεων συντάσσεται με γνώμονα τη λειτουργία Layer 2 (Tap Mode). Οι βασικές παράμετροι που εισάγονται είναι οι εξής:

- **client:** Ορίζει τη λειτουργία του OpenVPN ως πελάτη.
- **dev tap:** Ενεργοποιεί τη χρήση εικονικής διεπαφής επιπέδου 2, επιτρέποντας τη διαβίβαση Ethernet frames.
- **remote [Azure\_IP] 1194:** Καθορίζει τη δημόσια διεύθυνση του Node 0 και τη θύρα επικοινωνίας.
- **resolv-retry infinite:** Διασφαλίζει ότι ο client θα προσπαθεί διαρκώς να επανασυνδεθεί σε περίπτωση διακοπής.
- **nobind:** Επιτρέπει στο σύστημα να επιλέξει δυναμικά τη θύρα εξόδου.



```
root@node1:~# cat proxmox_vpn.conf
cat: proxmox_vpn.conf: No such file or directory
root@node1:~#
root@node1:~#
root@node1:~# cat /etc/openvpn/client/proxmox_vpn.conf
client
dev tap
proto udp
remote 51.11.240.65 1194
resolv-retry infinite
nobind
persist-key
persist-tun

ca /etc/openvpn/client/ca.crt
cert /etc/openvpn/client/proxmox-host.crt
key /etc/openvpn/client/proxmox-host.key
tls-auth /etc/openvpn/client/ta.key 1

remote-cert-tls server
cipher AES-256-GCM
auth SHA256
verb 3
route-nopull
pull-filter ignore "dhcp-option DNS"
```

Εικόνα 4.15: Το αρχείο proxmox\_vpn.conf

Ιδιαίτερη έμφαση δόθηκε στην ασφάλεια της σύνδεσης, ενσωματώνοντας τις παραμέτρους `tls-auth` για την επαλήθευση της υπογραφής των πακέτων και `cipher AES-256-GCM` για την κρυπτογράφηση των δεδομένων, διασφαλίζοντας ότι η επικοινωνία μεταξύ του ακαδημαϊκού δικτύου και του Cloud είναι προστατευμένη.

#### 4.4 Node 2: Παραμετροποίηση VPN Client και DHCP Server

Ο Node 2 (Laptop) αποτελεί τον κεντρικό πυλώνα της υλοποίησης, καθώς αναλαμβάνει διπλό ρόλο: αφενός λειτουργεί ως απομαχρυσμένος πελάτης (VPN Client) που γεφυρώνει τη φυσική υποδομή με το Cloud, και αφετέρου λειτουργεί ως ο διαχειριστής του δικτύου (DHCP Server). Η παραμετροποίηση αυτή είναι η πλέον σύνθετη, καθώς απαιτεί τον συντονισμό των υπηρεσιών OpenVPN και ISC-DHCP-Server πάνω σε μια κοινή εικονική γέφυρα (Bridge), η οποία θα εξυπηρετεί όλο το Layer 2 broadcast domain.

#### 4.4.1 Επαλήθευση και συμπληρωματική εγκατάσταση υπηρεσιών

Πριν από την έναρξη της παραμετροποίησης του Node 2, πραγματοποιήθηκε έλεγχος των εγκατεστημένων πακέτων. Ο έλεγχος αποκάλυψε ότι ενώ η υπηρεσία OpenVPN ήταν εγκατεστημένη, τα εργαλεία `bridge-utils` (διαχείριση γέφυρας Layer 2) και `isc-dhcp-server` (απόδοση διευθύνσεων IP) απουσίαζαν. Ακολούθησε η εγκατάστασή τους ώστε ο κόμβος να ανταποκριθεί στον σύνθετο ρόλο του Bridge Endpoint και DHCP Server.

```
drosos@kats:~/openvpn-keys$ dpkg -s openvpn bridge-utils isc-dhcp-server | grep -E "Package:|Status:"
Package: openvpn
Status: install ok installed
Package: bridge-utils
Status: install ok installed
Package: isc-dhcp-server
Status: install ok installed
```

Εικόνα 4.16: Επιτυχής εγκατάσταση και επαλήθευση των απαραίτητων πακέτων στον Node 2.

#### 4.4.2 Παραμετροποίηση VPN Client (Node 2)

Για τη διασύνδεση του Laptop με το Azure, δημιουργήθηκε το αρχείο `client.conf`. Η παραμετροποίηση ακολουθεί το πρότυπο του Node 1 (Tap Mode), χρησιμοποιώντας το πιστοποιητικό `node2.crt`. Η ρύθμιση αυτή επιτρέπει στον Node 2 να συμμετέχει στο Layer 2 broadcast domain ως ισότιμο μέλος.

```
drosos@kats:~$ cat /etc/openvpn/client/client.conf
client
dev tap0
proto udp
remote 51.11.240.65 1194
resolv-retry infinite
nobind
persist-key
persist-tun

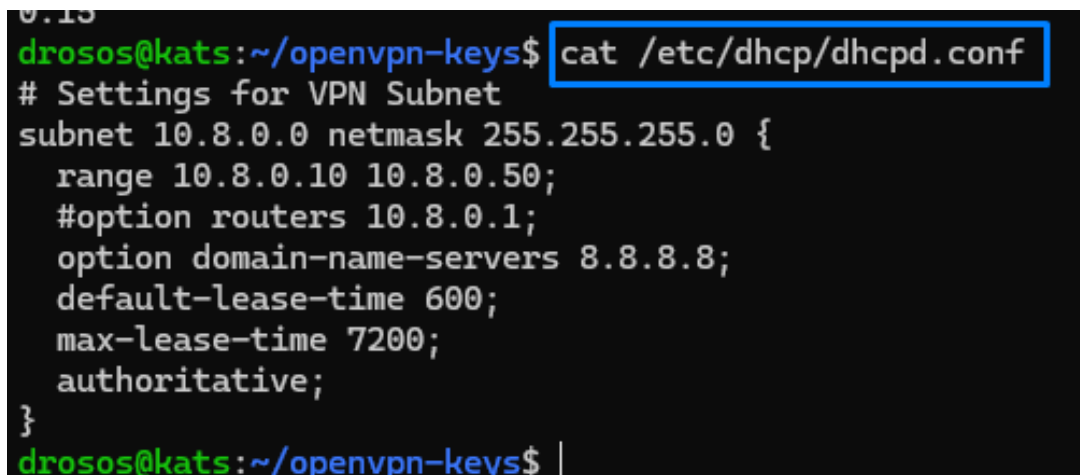
# paths
ca ca.crt
cert node2.crt
key node2.key
tls-auth ta.key 1

remote-cert-tls server
cipher AES-256-GCM
auth SHA256
verb 3
```

Εικόνα 4.17: Παραμετροποίηση του αρχείου `client.conf`

#### 4.4.3 Παραμετροποίηση DHCP server

Με την εικονική γέφυρα br0 να αποτελεί το σημείο σύγκλισης, ρυθμίστηκε ο ISC DHCP Server για την αυτόματη απόδοση διευθύνσεων IP στους απομακρυσμένους κόμβους (Node 0 και Node 1).



```
drosos@kats:~/openvpn-keys$ cat /etc/dhcp/dhcpd.conf
# Settings for VPN Subnet
subnet 10.8.0.0 netmask 255.255.255.0 {
    range 10.8.0.10 10.8.0.50;
    #option routers 10.8.0.1;
    option domain-name-servers 8.8.8.8;
    default-lease-time 600;
    max-lease-time 7200;
    authoritative;
}
drosos@kats:~/openvpn-keys$
```

Εικόνα 4.18: Παραμετροποίηση του αρχείου dhcpd.conf για το δίκτυο 10.8.0.0/24.

Στο αρχείο /etc/dhcp/dhcpd.conf, καθορίζονται οι εξής παράμετροι:

- **Ορισμός subnet:** Το δίκτυο ορίζεται ως 10.8.0.0 με netmask 255.255.255.0.
- **Εύρος IP (Range):** Καθορίζεται το εύρος 10.8.0.10 έως 10.8.0.100 για δυναμική απόδοση.
- **Lease Times:** Ορίζεται default-lease-time 600 δευτερόλεπτα για ταχεία ανανέωση κατά τις δοκιμές.

##### Κρίσιμη Τεχνική Παρατήρηση

Η επιλογή option routers 10.8.0.1 έχει τεθεί σε σχόλιο (commented out). Αυτό αποτρέπει την αλλαγή του Default Gateway στον Node 0, αποφεύγοντας το SSH lockout και τη διακοπή επικοινωνίας με το Azure.

#### 4.4.4 Καθορισμός Listening Interface

Είναι απαραίτητο να οριστεί η διεπαφή στην οποία θα «ακούει» η υπηρεσία DHCP. Όπως φαίνεται στην παρακάτω εικόνα, αυτό υλοποιείται στο αρχείο /etc/default/isc-dhcp-server.

```
}
drosos@kats:~/openvpn-keys$ sudo nano /etc/default/isc-dhcp-server
drosos@kats:~/openvpn-keys$ cat /etc/default/isc-dhcp-server
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4_CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4_PID=/var/run/dhcpd.pid
#DHCPDv6_PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD_CONF/ DHCPD_PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="br0"
INTERFACESv6=""
drosos@kats:~/openvpn-keys$ |
```

Εικόνα 4.19: Ορισμός της γέφυρας br0 ως INTERFACESv4.

Η επιλογή της br0 εξασφαλίζει:

- **Σύζευξη με το VPN:** Ο DHCP server λαμβάνει τα DHCP Discover πακέτα που καταφθάνουν μέσω του tap0.
- **Αποφυγή Rogue DHCP:** Η υπηρεσία περιορίζεται στο εικονικό δίκτυο και δεν επηρεάζει το φυσικό δίκτυο (Wi-Fi/Ethernet) του Laptop.

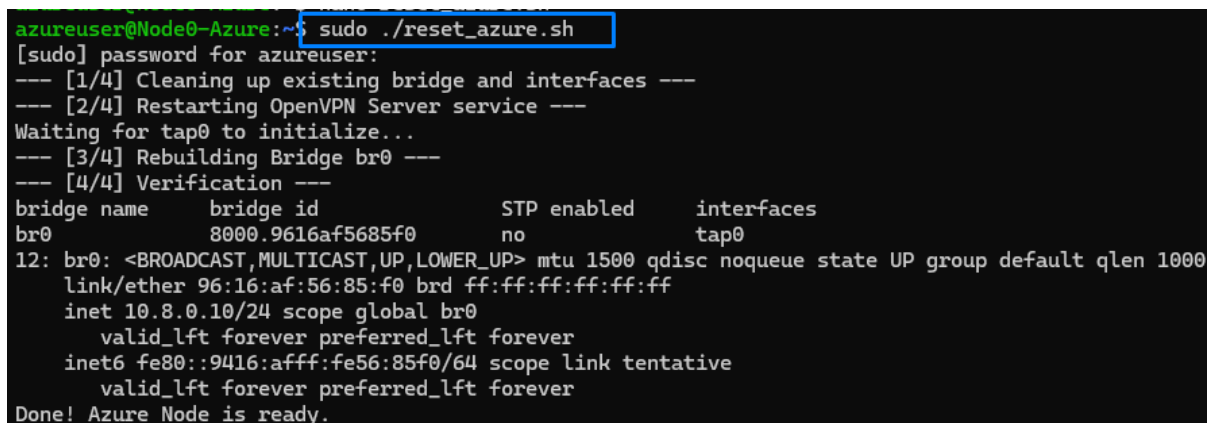
## 4.5 Αυτοματοποίηση και Ενεργοποίηση Υποδομής (Scripts)

Με την ολοκλήρωση της παραμετροποίησης των αρχείων .conf και την έκδοση των πιστοποιητικών σε όλους τους κόμβους, η υποδομή είναι πλέον αρχιτεκτονικά έτοιμη. Ωστόσο, η χειροκίνητη ενεργοποίηση των διεπαφών Layer 2, η δημιουργία των γέφυρων (bridges) και η σειριακή εκκίνηση των υπηρεσιών OpenVPN και DHCP ενέχουν τον κίνδυνο ανθρώπινου σφάλματος και καθυστερήσεων.

Για τη διασφάλιση της επαναληψιμότητας (reproducibility) και της ταχείας αποκατάστασης του δικτύου σε περίπτωση αστοχίας, αναπτύχθηκαν τρία διαφορετικά Bash scripts. Τα scripts αυτά αυτοματοποιούν τη διαδικασία «δεσίματος» των εικονικών διεπαφών tap0 στις γέφυρες br0, εξασφαλίζοντας ότι το Layer 2 tunnel είναι λειτουργικό αμέσως μετά την εκτέλεσή τους.

#### 4.5.1 Node 0: Server-Side Bridge αυτοματισμός

Το script στον Node 0 (Azure) αναλαμβάνει την προετοιμασία του περιβάλλοντος πριν την εκκίνηση του OpenVPN. Αρχικά, καθαρίζει τυχόν προϋπάρχουσες ρυθμίσεις δικτύου, δημιουργεί τη γέφυρα `br0` και την ενεργοποιεί. Με την εκκίνηση της υπηρεσίας OpenVPN, η εικονική διεπαφή `tap0` προστίθεται αυτόματα στη γέφυρα, επιτρέποντας στον server να λειτουργήσει ως ο κεντρικός κόμβος του bridge.



```
azureuser@Node0-Azure:~$ sudo ./reset_azure.sh
[sudo] password for azureuser:
--- [1/4] Cleaning up existing bridge and interfaces ---
--- [2/4] Restarting OpenVPN Server service ---
Waiting for tap0 to initialize...
--- [3/4] Rebuilding Bridge br0 ---
--- [4/4] Verification ---
bridge name      bridge id          STP enabled      interfaces
br0              8000.9616af5685f0  no               tap0
12: br0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 96:16:af:56:85:f0 brd ff:ff:ff:ff:ff:ff
    inet 10.8.0.10/24 scope global br0
        valid_lft forever preferred_lft forever
    inet6 fe80::9416:aaff:fe56:85f0/64 scope link tentative
        valid_lft forever preferred_lft forever
Done! Azure Node is ready.
```

Εικόνα 4.20: Επιτυχής εκτέλεση του script αυτοματισμού στον Node 0.

Όπως παρατηρείται στην Εικόνα 4.20, η διαδικασία ολοκληρώνεται σε τέσσερα στάδια:

1. **Cleaning:** Εκκαθάριση παλαιών interfaces για αποφυγή διενέξεων.
2. **Restarting:** Επανεκκίνηση του OpenVPN daemon.
3. **Rebuilding:** Δημιουργία της γέφυρας `br0` και ένταξη της `tap0`.
4. **Verification:** Επιβεβαίωση ότι η γέφυρα είναι ενεργή (state UP) και έχει λάβει τη διεύθυνση 10.8.0.10.

#### 4.5.2 Node 1: Proxmox Bridge & VPN Connectivity

Στον Node 1 (Proxmox Hypervisor), το script αυτοματοποίησης επιτελεί τον κρίσιμο ρόλο της σύζευξης του τοπικού Hypervisor με την υποδομή στο Cloud. Η διαδικασία περιλαμβάνει τη δυναμική δημιουργία της γέφυρας `br0` και την ένταξη της διεπαφής `tap0` σε αυτήν αμέσως μετά την επιτυχή σύνδεση του VPN client.

Η αρχιτεκτονική αυτή επιτρέπει στις εικονικές μηχανές (όπως το VM 100) να εκπέμπουν πακέτα (broadcasts), τα οποία δρομολογούνται μέσω του Layer 2 tunnel στον DHCP server του Node 2.

```
root@node1:~# ./reset_proxmox.sh
--- [1/5] Cleaning up old network configurations ---
--- [2/5] Restarting OpenVPN Client (proxmox_vpn) ---
Waiting for tap0 interface to initialize (15s)...
--- [3/5] Setting up Bridge and Linking VPN ---
--- [4/5] Requesting IP from Node 2 (Laptop) via DHCP ---
Internet Systems Consortium DHCP Client 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/br0/76:02:85:98:c0:8c
Sending on   LPF/br0/76:02:85:98:c0:8c
Sending on   Socket/fallback
DHCPREQUEST for 10.8.0.15 on br0 to 255.255.255.255 port 67
DHCPNAK from 10.8.0.1
DHCPDISCOVER on br0 to 255.255.255.255 port 67 interval 8
DHCPOFFER of 10.8.0.16 from 10.8.0.1
DHCPREQUEST for 10.8.0.16 on br0 to 255.255.255.255 port 67
DHCPACK of 10.8.0.16 from 10.8.0.1
bound to 10.8.0.16 -- renewal in 284 seconds.
--- [5/5] Connectivity Verification ---
    inet 10.8.0.16/24 brd 10.8.0.255 scope global dynamic br0
Testing connectivity to Laptop (Node 2)...
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=184 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=93.3 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=92.7 ms

--- 10.8.0.1 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 92.652/123.454/184.451/43.131 ms
root@node1:~#
```

Εικόνα 4.21: Εκτέλεση του script `reset_proxmox.sh` και επιτυχής λήψη IP μέσω του tunnel.

Όπως παρατηρείται στην Εικόνα 4.21, η εκτέλεση του script επιβεβαιώνει τη λειτουργικότητα της υποδομής σε τρία επίπεδα:

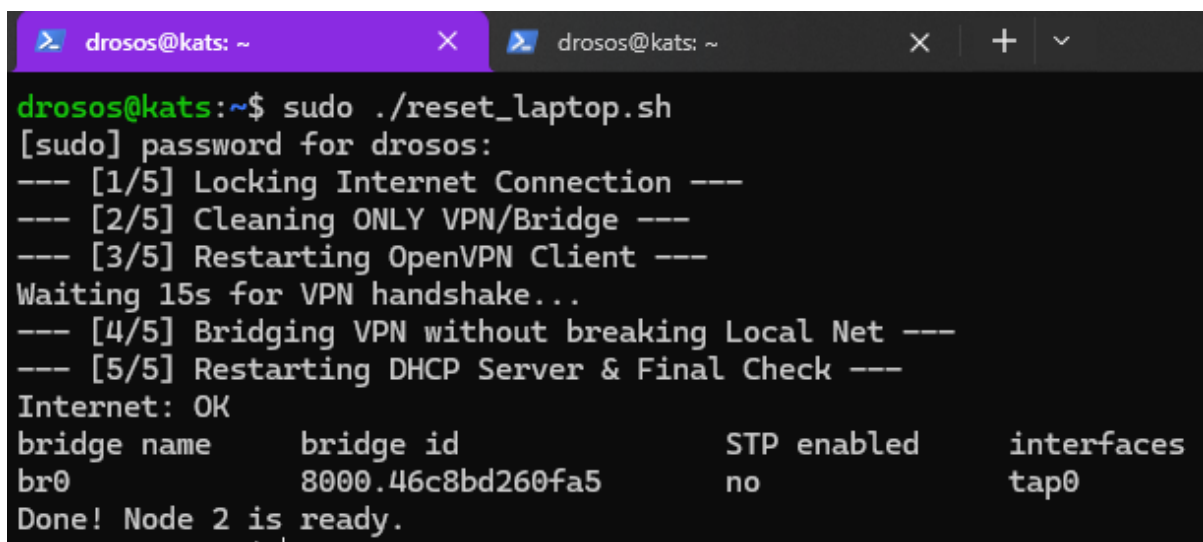
1. **DHCP Handshake:** Ο Proxmox στέλνει DHCPREQUEST και λαμβάνει DHCPACK από τη διεύθυνση 10.8.0.1 (Node 2), αποκτώντας τη διεύθυνση 10.8.0.16.
2. **Interface Initialization:** Η γέφυρα br0 τίθεται σε κατάσταση UP με το σωστό MTU (1500).
3. **Connectivity Verification:** Η διαδικασία ολοκληρώνεται με επιτυχή έλεγχο ping



προς τον Node 2, με μέσο χρόνο απόκρισης (RTT) τα 92ms, επιβεβαιώνοντας τη σταθερότητα του κρυπτογραφημένου καναλιού.

### 4.5.3 Node 2: Gateway & DHCP Deployment

Ο Node 2 (Laptop) διαδραματίζει τον πλέον σύνθετο ρόλο στην αυτοματοποίηση της υποδομής, καθώς το script του πρέπει να διασφαλίσει τη διατήρηση της τοπικής συνδεσιμότητας (Internet) ενώ παράλληλα «σηκώνει» το εικονικό δίκτυο. Το script `reset_laptop.sh` εκτελεί μια προσεκτικά σχεδιασμένη ακολουθία ενεργειών: αρχικοποιεί το VPN tunnel, δημιουργεί τη γέφυρα `br0`, ενσωματώνει τη διεπαφή `tap0` και, τέλος, εκκινεί τον DHCP server για την εξυπηρέτηση ολόκληρου του εικονικού LAN.



```
drosos@kats: ~$ sudo ./reset_laptop.sh
[sudo] password for drosos:
--- [1/5] Locking Internet Connection ---
--- [2/5] Cleaning ONLY VPN/Bridge ---
--- [3/5] Restarting OpenVPN Client ---
Waiting 15s for VPN handshake...
--- [4/5] Bridging VPN without breaking Local Net ---
--- [5/5] Restarting DHCP Server & Final Check ---
Internet: OK
bridge name      bridge id          STP enabled      interfaces
br0              8000.46c8bd260fa5  no              tap0
Done! Node 2 is ready.
```

Εικόνα 4.22: Επιτυχής εκτέλεση του script `reset_laptop.sh` στον Node 2.

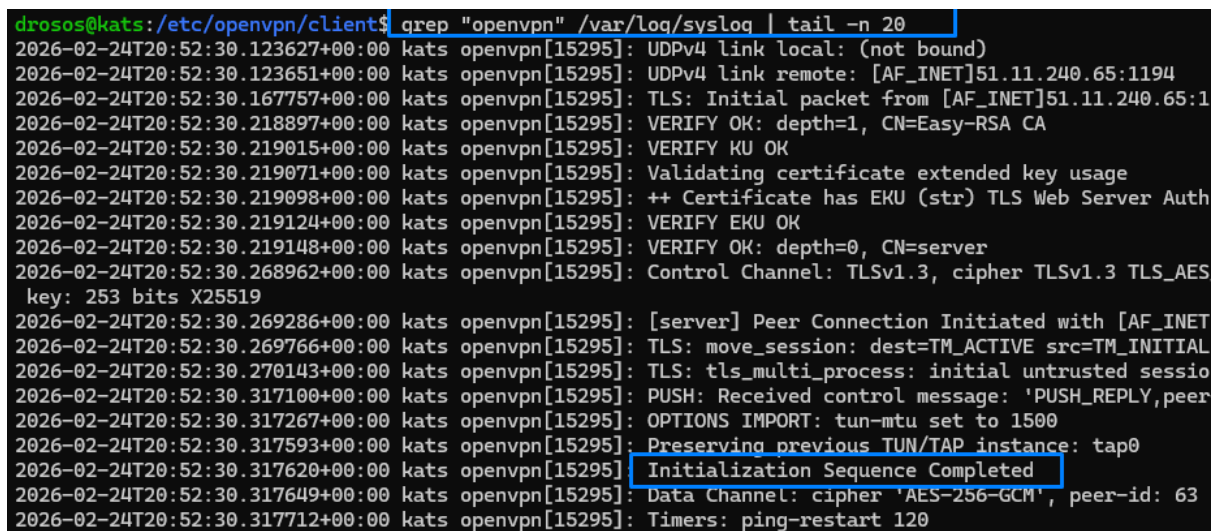
Όπως φαίνεται στην Εικόνα 4.22, η επιτυχής εκτέλεση επιβεβαιώνεται από τα εξής σημεία:

1. **Locking Connection:** Διασφάλιση της στατικής IP του Laptop (192.168.1.5) ώστε να μην διακοπεί η πρόσβαση στο Internet κατά τη διάρκεια του bridging.
2. **VPN Handshake:** Αναμονή 15 δευτερολέπτων για την πλήρη εγκαθίδρυση του tunnel πριν την προσθήκη στη γέφυρα.
3. **Bridge Verification:** Η γέφυρα `br0` εμφανίζεται ενεργή με τη διεπαφή `tap0` επιτυχώς συνδεδεμένη.
4. **Service Readiness:** Η τελική ένδειξη "Internet: OK" και "Node 2 is ready" επιβεβαιώνει ότι ο DHCP server λειτουργεί χωρίς να επηρεάζει την εξωτερική επικοινωνία του συστήματος.

Με την επιτυχή εκτέλεση των τριών scripts, η Layer 2 υποδομή είναι πλέον πλήρως λειτουργική. Ο Node 2 μοιράζει διευθύνσεις IP στους απομακρυσμένους κόμβους, δημιουργώντας ένα ενιαίο, ασφαλές και διαφανές δίκτυο πάνω από το δημόσιο Internet. Αυτή η σταθερή δικτυακή βάση επιτρέπει πλέον την υλοποίηση του συστήματος αυτοματισμού και συλλογής δεδομένων που αναλύεται στο επόμενο κεφάλαιο.

## 4.6 Επαλήθευση Εγκαθίδρυσης Tunnel (Logs)

Πριν από την παρουσίαση της τελικής αρχιτεκτονικής, η ορθή λειτουργία της υπηρεσίας OpenVPN επιβεβαιώνεται μέσω των αρχείων καταγραφής (logs) του συστήματος. Χρησιμοποιώντας την εντολή `grep "openvpn" /var/log/syslog | tail -n 20` στον Node 2, ανακτήθηκε η πλήρης αλληλουχία της σύνδεσης.



```
droso@kats:/etc/openvpn/client$ grep "openvpn" /var/log/syslog | tail -n 20
2026-02-24T20:52:30.123627+00:00 kats openvpn[15295]: UDPv4 link local: (not bound)
2026-02-24T20:52:30.123651+00:00 kats openvpn[15295]: UDPv4 link remote: [AF_INET]51.11.240.65:1194
2026-02-24T20:52:30.167757+00:00 kats openvpn[15295]: TLS: Initial packet from [AF_INET]51.11.240.65:1
2026-02-24T20:52:30.218897+00:00 kats openvpn[15295]: VERIFY OK: depth=1, CN=Easy-RSA CA
2026-02-24T20:52:30.219015+00:00 kats openvpn[15295]: VERIFY KU OK
2026-02-24T20:52:30.219071+00:00 kats openvpn[15295]: Validating certificate extended key usage
2026-02-24T20:52:30.219098+00:00 kats openvpn[15295]: ++ Certificate has EKU (str) TLS Web Server Auth
2026-02-24T20:52:30.219124+00:00 kats openvpn[15295]: VERIFY EKU OK
2026-02-24T20:52:30.219148+00:00 kats openvpn[15295]: VERIFY OK: depth=0, CN=server
2026-02-24T20:52:30.268962+00:00 kats openvpn[15295]: Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES
key: 253 bits X25519
2026-02-24T20:52:30.269286+00:00 kats openvpn[15295]: [server] Peer Connection Initiated with [AF_INET]
2026-02-24T20:52:30.269766+00:00 kats openvpn[15295]: TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL
2026-02-24T20:52:30.270143+00:00 kats openvpn[15295]: TLS: tls_multi_process: initial untrusted sessio
2026-02-24T20:52:30.317100+00:00 kats openvpn[15295]: PUSH: Received control message: 'PUSH_REPLY,peer
2026-02-24T20:52:30.317267+00:00 kats openvpn[15295]: OPTIONS IMPORT: tun-mtu set to 1500
2026-02-24T20:52:30.317593+00:00 kats openvpn[15295]: Preserving previous TUN/TAP instance: tap0
2026-02-24T20:52:30.317620+00:00 kats openvpn[15295]: Initialization Sequence Completed
2026-02-24T20:52:30.317649+00:00 kats openvpn[15295]: Data Channel: cipher 'AES-256-GCM', peer-id: 63
2026-02-24T20:52:30.317712+00:00 kats openvpn[15295]: Timers: ping-restart 120
```

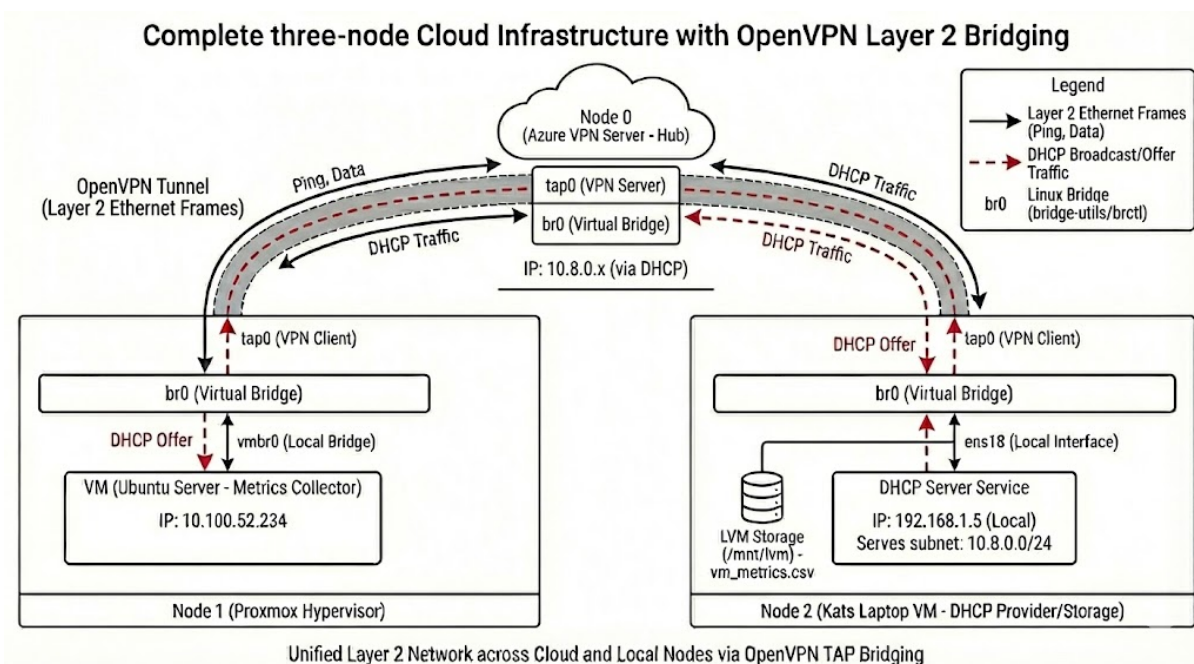
Εικόνα 4.23: Logs συστήματος στον Node 2: Επιτυχής ολοκλήρωση σύνδεσης με το μήνυμα "Initialization Sequence Completed".

Όπως παρατηρείται στην Εικόνα 4.23, η σύνδεση χρησιμοποιεί το πρωτόκολλο TLSv1.3 με κρυπτογράφηση AES-256-GCM. Η επιτυχής αυθεντικοποίηση (VERIFY OK) και η απόδοση των ρυθμίσεων μέσω του PUSH\_REPLY καταλήγουν στην ενεργοποίηση του εικονικού διαύλου επικοινωνίας, επιτρέποντας πλέον την ενοποίηση του δικτύου σε επίπεδο Layer 2.

## 4.7 Συνολική Αρχιτεκτονική Διασύνδεσης

Με την επιτυχή εκτέλεση των scripts αυτοματισμού, η υποδομή αποκτά την τελική της μορφή. Το παρακάτω διάγραμμα, απεικονίζει τη λογική σύζευξη των τριών κόμβων σε ένα ενιαίο Layer 2 δίκτυο, όπου τα γεωγραφικά όρια καταργούνται μέσω του VPN Tunnel.





Εικόνα 4.24: Ολοκληρωμένο διάγραμμα διασύνδεσης

Όπως παρατηρείται, ο Node 2 λειτουργεί ως κεντρικό hub του δικτύου, ο Node 0 ως ο ενδιάμεσος σταθμός (Cloud Relay) και ο Node 1 ως ο Hypervisor που φιλοξενεί τις εικονικές μηχανές.

#### 4.7.1 Επαλήθευση Λειτουργίας (Verification)

Για την οριστική επιβεβαίωση της ορθής λειτουργίας της υποδομής, πραγματοποιήθηκαν δοκιμές σε τρία επίπεδα:

- **Επαλήθευση DHCP Logs:** Η απόδειξη της Layer 2 γέφυρας είναι η καταγραφή των αιτημάτων DHCP που διασχίζουν το tunnel. Χρησιμοποιώντας την εντολή `tail -f /var/log/syslog | grep dhcpcd` στον Node 2, καταγράφηκε η πλήρης αλληλουχία (DISCOVER, OFFER, REQUEST, ACK).

```

drosos@kats:~$ tail -f /var/log/syslog | grep dhcpcd
2026-02-24T19:31:31.047583+00:00 kats dhcpcd[15334]: DHCPDISCOVER from 76:02:85:98:c0:8c (node1) via br0
2026-02-24T19:31:31.047775+00:00 kats dhcpcd[15334]: DHCPDISCOVER on 10.8.0.16 to 76:02:85:98:c0:8c (node1) via br0
2026-02-24T19:31:31.142257+00:00 kats dhcpcd[15334]: DHCPREQUEST for 10.8.0.16 (10.8.0.1) from 76:02:85:98:c0:8c (node1) via br0
2026-02-24T19:31:31.154552+00:00 kats dhcpcd[15334]: DHCPACK on 10.8.0.16 to 76:02:85:98:c0:8c (node1) via br0
2026-02-24T19:32:11.815779+00:00 kats dhcpcd[15334]: DHCPDISCOVER from 96:16:af:56:85:f0 via br0
2026-02-24T19:32:12.816781+00:00 kats dhcpcd[15334]: DHCPDISCOVER on 10.8.0.10 to 96:16:af:56:85:f0 (Node0-Azure) via br0
2026-02-24T19:32:12.864092+00:00 kats dhcpcd[15334]: DHCPREQUEST for 10.8.0.10 (10.8.0.1) from 96:16:af:56:85:f0 (Node0-Azure) via br0
2026-02-24T19:32:12.871401+00:00 kats dhcpcd[15334]: DHCPACK on 10.8.0.10 to 96:16:af:56:85:f0 (Node0-Azure) via br0

```

Εικόνα 4.25: Logs του DHCP Server στον Node 2: Απόδοση IP 10.8.0.16 στον Node 1 και 10.8.0.10 στον Node 0.

Όπως φαίνεται στην Εικόνα 4.25, ο DHCP server αναγνωρίζει τις MAC addresses των απομακρυσμένων κόμβων και τους αποδίδει διευθύνσεις στο subnet 10.8.0.0/24, επιβεβαιώνοντας ότι το broadcast domain είναι ενιαίο.

- **Δοκιμές Συνδεσιμότητας (Ping & Traceroute)** Πραγματοποιήθηκαν δοκιμές Ping μεταξύ του Node 2 και του Node 1 (Proxmox), καθώς και προς την εικονική μηχανή (VM 100). Η επιτυχής απόκριση επιβεβαιώνει τη διαφάνεια του bridge.

```
drosos@kats:~$ ping -c 4 10.8.0.10
PING 10.8.0.10 (10.8.0.10) 56(84) bytes of data.
64 bytes from 10.8.0.10: icmp_seq=1 ttl=64 time=47.3 ms
64 bytes from 10.8.0.10: icmp_seq=2 ttl=64 time=47.5 ms
64 bytes from 10.8.0.10: icmp_seq=3 ttl=64 time=46.7 ms
64 bytes from 10.8.0.10: icmp_seq=4 ttl=64 time=67.6 ms

--- 10.8.0.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3017ms
rtt min/avg/max/mdev = 46.715/52.292/67.624/8.856 ms
drosos@kats:~$ ping -c 4 10.8.0.16
PING 10.8.0.16 (10.8.0.16) 56(84) bytes of data.
64 bytes from 10.8.0.16: icmp_seq=1 ttl=64 time=93.4 ms
64 bytes from 10.8.0.16: icmp_seq=2 ttl=64 time=93.7 ms
64 bytes from 10.8.0.16: icmp_seq=3 ttl=64 time=93.1 ms
64 bytes from 10.8.0.16: icmp_seq=4 ttl=64 time=93.3 ms

--- 10.8.0.16 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3008ms
rtt min/avg/max/mdev = 93.083/93.370/93.692/0.216 ms
```

Εικόνα 4.26: Επαλήθευση συνδεσιμότητας μέσω Ping από τον Node 1 προς τον Node 2.

Η επιτυχής ανταλλαγή πακέτων με μέσο χρόνο 92ms επιβεβαιώνει ότι το κρυπτογραφημένο κανάλι επικοινωνίας είναι σταθερό και έτοιμο να υποδεχθεί την κίνηση των δεδομένων από το Proxmox API.

Συμπληρωματικά, πραγματοποιήθηκε έλεγχος μέσω της εντολής traceroute για την επαλήθευση της διαδρομής των πακέτων.

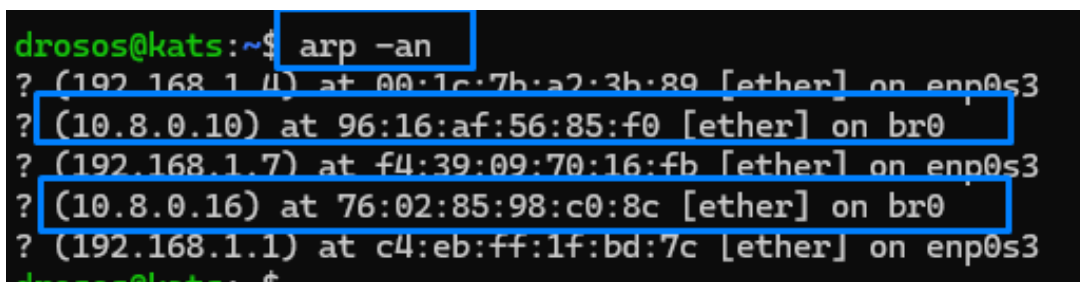
```
2 10.8.0.16 187.271ms 92.803ms 92.436ms
drosos@kats:~$ traceroute 10.8.0.16
traceroute to 10.8.0.16 (10.8.0.16), 64 hops max
 1 10.8.0.16 92.986ms 92.294ms 92.733ms
drosos@kats:~$ traceroute 10.8.0.10
traceroute to 10.8.0.10 (10.8.0.10), 64 hops max
 1 10.8.0.10 47.615ms 46.551ms 47.777ms
drosos@kats:~$ |
```

Εικόνα 4.27: Έλεγχος διαδρομής (Traceroute) προς τους απομακρυσμένους κόμβους.

Όπως φαίνεται στην Εικόνα 4.27, η επικοινωνία με τον Node 0 ολοκληρώνεται σε ένα μόλις άλμα (single hop), γεγονός που αποδεικνύει τη λειτουργία του Layer 2 tunnel. Στην περίπτωση του Node 1 (Proxmox), η εμφάνιση δεύτερου άλματος οφείλεται στην εσωτερική δρομολόγηση του Hypervisor (Nested Virtualization) και στη διαχείριση της εικονικής γέφυρας εντός του Proxmox. Η χαμηλή καθυστέρηση και η άμεση απόκριση επιβεβαιώνουν ότι

η κίνηση παραμένει εντός του κρυπτογραφημένου δικτύου, διασφαλίζοντας την ιδιωτικότητα της επικοινωνίας με το API.

- **Έλεγχος Layer 2 Adjacency (ARP Table):** Η επιβεβαίωση της λειτουργίας της υποδομής σε επίπεδο Layer 2 προκύπτει από την εξέταση του πίνακα ARP (Address Resolution Protocol) στον Node 2. Σε ένα τυπικό Routed VPN (Layer 3), οι MAC addresses των απομακρυσμένων κόμβων δεν θα ήταν ορατές. Στην παρούσα υλοποίηση, η χρήση του Tap mode επιτρέπει τη διαφάνεια των Ethernet frames. Εκτελώντας την εντολή `arp -an` στον Node 2, επιβεβαιώνεται ότι το σύστημα αναγνωρίζει τις φυσικές διευθύνσεις των απομακρυσμένων κόμβων εντός της γέφυρας `br0`.



```
drosos@kats:~$ arp -an
? (192.168.1.4) at 00:1c:7b:a2:3b:89 [ether] on enp0s3
? (10.8.0.10) at 96:16:af:56:85:f0 [ether] on br0
? (192.168.1.7) at f4:39:09:70:16:fb [ether] on enp0s3
? (10.8.0.16) at 76:02:85:98:c0:8c [ether] on br0
? (192.168.1.1) at c4:eb:ff:1f:bd:7c [ether] on enp0s3
drosos@kats:~$
```

Εικόνα 4.28: Πίνακας ARP του Node 2: Οι εγγραφές για τους Node 0 (10.8.0.10) και Node 1 (10.8.0.16) εμφανίζονται με τις MAC addresses τους στη διεπαφή `br0`.

Όπως παρατηρείται στην Εικόνα 4.28, οι διευθύνσεις:

- **10.8.0.10 (Azure):** Αντιστοιχίζεται στη MAC address `96:16:af:56:85:f0`.
- **10.8.0.16 (Proxmox):** Αντιστοιχίζεται στη MAC address `76:02:85:98:c0:8c`.

Το γεγονός ότι οι εγγραφές αυτές χαρακτηρίζονται ως `[ether]` πάνω στη διεπαφή `br0`, αποδεικνύει ότι οι κόμβοι "πιστεύουν" πως βρίσκονται στο ίδιο φυσικό switch, επιβεβαιώνοντας την επιτυχία της Layer 2 γεφύρωσης.

## 5 Κεφάλαιο 5: Αυτοματισμός και Monitoring

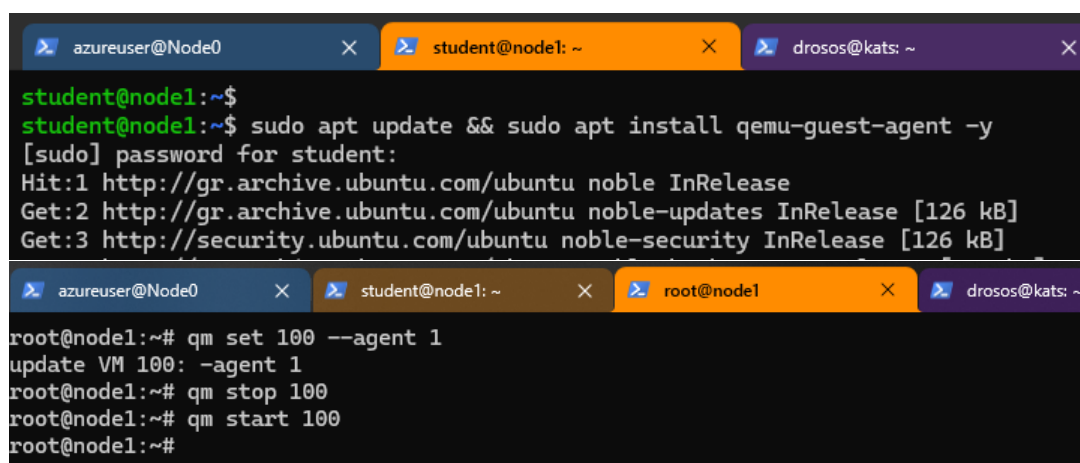
Στο παρόν κεφάλαιο, η υλοποίηση μεταβαίνει από το στάδιο της εγκατάστασης στο στάδιο της διαχείρισης. Με την υποδομή VPN ενεργή, καθίσταται δυνατή η κεντρική παρακολούθηση των πόρων του Node 1 (Proxmox) από τον Node 2 (Management Node). Στόχος είναι η αυτοματοποιημένη συλλογή δεδομένων λειτουργίας και παραμετροποίησης των εικονικών μηχανών, διασφαλίζοντας την εποπτεία της υγείας του συστήματος χωρίς ανθρώπινη παρέμβαση. Στο παρακάτω λογικό διάγραμμα, αποτυπώνεται αυτό ακριβώς:

### 5.1 Ενεργοποίηση Guest-Agent στον Node 1

Όπως αναλύθηκε διεξοδικά στην υποενότητα 3.3, στον Node 1 (Proxmox Lab) έχει ήδη εγκατασταθεί μια εικονική μηχανή Ubuntu Server 24.04 LTS, η οποία αποτελεί το αντικείμενο παρακολούθησης της υποδομής. Ωστόσο, για να καταστεί δυνατή η άντληση προηγμένων δεδομένων από το API του Proxmox με υψηλή ακρίβεια, απαιτείται η εγκατάσταση και ενεργοποίηση του QEMU Guest Agent εντός του VM. Ο Guest Agent λειτουργεί ως ένας ενδιάμεσος δίαυλος επικοινωνίας (bridge) μεταξύ του Hypervisor και του λειτουργικού συστήματος του φιλοξενούμενου (Guest). Η σημασία του στην παρούσα υλοποίηση είναι κρίσιμη, καθώς επιτρέπει στο API να «βλέπει» στο εσωτερικό του VM, παρέχοντας πληροφορίες που δεν είναι διαθέσιμες από το εξωτερικό περιβάλλον εικονικοποίησης, όπως:

- Ακριβείς διευθύνσεις IP των εσωτερικών διεπαφών.
- Λεπτομερή χρήση μνήμης RAM (διαχωρισμός cached/free).
- Δυνατότητα εκτέλεσης ασφαλών εντολών διαχείρισης (graceful shutdown/reboot) μέσω API calls.

Η εγκατάσταση πραγματοποιήθηκε εντός της VM με τις ακόλουθες εντολές:

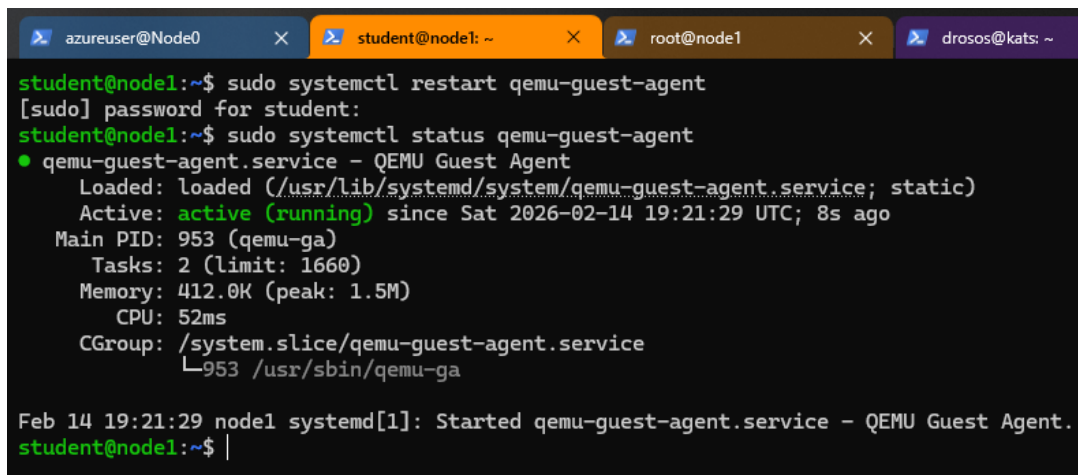


```
student@node1:~$
student@node1:~$ sudo apt update && sudo apt install qemu-guest-agent -y
[sudo] password for student:
Hit:1 http://gr.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://gr.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]

root@node1:~# qm set 100 --agent 1
update VM 100: -agent 1
root@node1:~# qm stop 100
root@node1:~# qm start 100
root@node1:~#
```

Εικόνα 5.1: Εγκατάσταση και ενεργοποίηση του qemu-guest-agent στον Node 1

Και στο παρακάτω screenshot, βλέπουμε πως η υπηρεσία είναι σε mode: "running".



```
student@node1:~$ sudo systemctl restart qemu-guest-agent
[sudo] password for student:
student@node1:~$ sudo systemctl status qemu-guest-agent
● qemu-guest-agent.service - QEMU Guest Agent
   Loaded: loaded (/usr/lib/systemd/system/qemu-guest-agent.service; static)
   Active: active (running) since Sat 2026-02-14 19:21:29 UTC; 8s ago
     Main PID: 953 (qemu-ga)
        Tasks: 2 (limit: 1660)
      Memory: 412.0K (peak: 1.5M)
         CPU: 52ms
       CGroup: /system.slice/qemu-guest-agent.service
               └─953 /usr/sbin/qemu-ga

Feb 14 19:21:29 node1 systemd[1]: Started qemu-guest-agent.service - QEMU Guest Agent.
student@node1:~$ |
```

Εικόνα 5.2: Ενεργοποίηση του qemu-guest-agent στον Node 1

Μετά την εγκατάσταση, στις ρυθμίσεις της VM στο περιβάλλον του Proxmox (Options -> QEMU Guest Agent), ενεργοποιήθηκε η σχετική επιλογή ώστε ο Hypervisor να αναγνωρίσει τον πράκτορα. Η επιτυχής ενεργοποίηση επιβεβαιώνεται από την εμφάνιση της IP διεύθυνσης της VM απευθείας στο GUI του Proxmox, σηματοδοτώντας ότι η VM είναι έτοιμη να παρέχει δεδομένα στα Endpoints που θα αναλυθούν στην επόμενη ενότητα.

## 5.2 Ανάλυση του Proxmox API και Endpoints

Για την άντληση των δεδομένων τηλεμετρίας, το Python script επικοινωνεί με δύο συγκεκριμένα Endpoints του Proxmox API. Η επιλογή τους έγινε με γνώμονα τον διαχωρισμό μεταξύ δυναμικής κατάστασης και στατικής παραμετροποίησης:

- **Endpoint Κατάστασης (/status/current):** Παρέχει δεδομένα σε πραγματικό χρόνο (real-time metrics). Από εδώ αντλούνται τιμές όπως η τρέχουσα χρήση CPU, η δεσμευμένη μνήμη RAM (mem), η δικτυακή κίνηση (netin/netout) και ο χρόνος λειτουργίας (uptime).
- **Endpoint Παραμετροποίησης (/config):** Παρέχει τα δομικά χαρακτηριστικά του VM. Από εδώ αντλούνται πληροφορίες όπως ο αριθμός των επεξεργαστικών πυρήνων (cores), η συνολικά ανατεθειμένη μνήμη (memory) και το όνομα του VM.

Η συνδυαστική χρήση των δύο αυτών πηγών επιτρέπει τη δημιουργία ενός πλήρους προφίλ για την υγεία και τη διαμόρφωση της εικονικής υποδομής.

## 5.3 Ανάπτυξη του Python Automation Script

Η κεντρική λειτουργία του αυτοματισμού βασίζεται σε ένα script γραμμένο σε γλώσσα προγραμματισμού Python 3, το οποίο εκτελείται στον Node 2. Το script αναλαμβάνει την επικοινωνία με τον Proxmox Host (Node 1) μέσω του REST API, την επεξεργασία των JSON δεδομένων και την τελική αποθήκευσή τους σε δομημένη μορφή CSV.



### 5.3.1 Αυθεντικοποίηση και Ασφάλεια API

Πριν την εκτέλεση οποιασδήποτε κλήσης δεδομένων, απαιτείται η αυθεντικοποίηση του script. Για τον σκοπό αυτό, χρησιμοποιήθηκε το σύστημα API Tokens του Proxmox, το οποίο παρέχει υψηλότερο επίπεδο ασφάλειας σε σχέση με τη χρήση κωδικών πρόσβασης.

```

root@node1:~# qm set 100 --agent 1
update VM 100: -agent 1
root@node1:~# qm stop 100
root@node1:~# qm start 100
root@node1:~# pveum user token add root@pam automation --privsep 0

```

| key          | value                                |
|--------------|--------------------------------------|
| full-tokenid | root@pam!automation                  |
| info         | {'privsep': '0'}                     |
| value        | 7850c8d4-a715-48c5-b94e-f100ec5b49b0 |

```

root@node1:~# pveum acl modify / --token 'root@pam!automation' --roles PVEVMAdmin
root@node1:~#

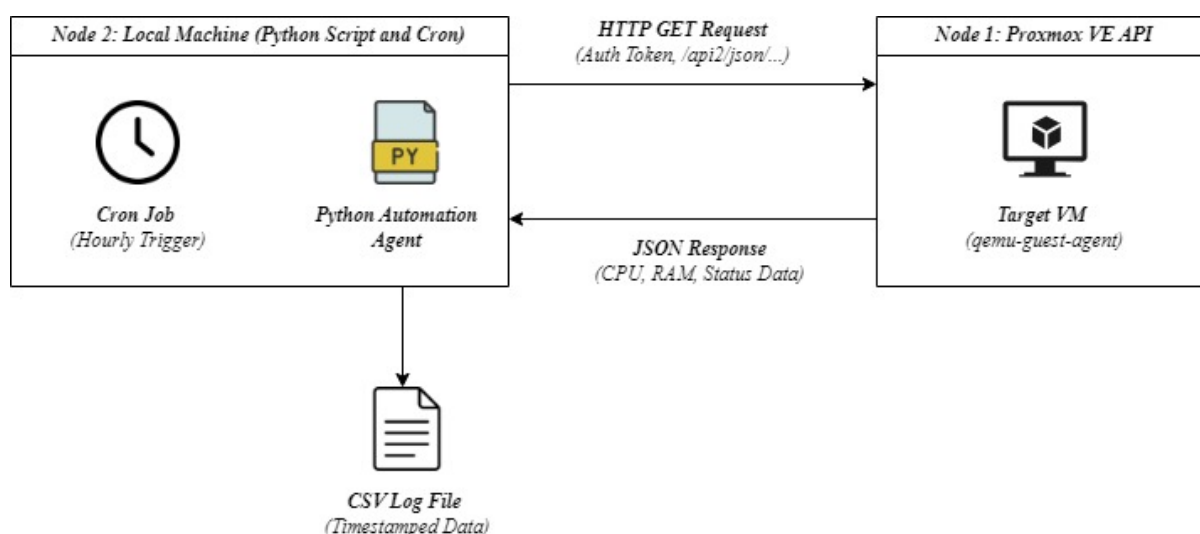
```

Εικόνα 5.3: Δημιουργία του token

Όπως φαίνεται στην παραπάνω εικόνα, δημιουργήθηκε ένα Token ID (root@pam!automation) και ένα μοναδικό Secret Key. Αυτά τα στοιχεία ενσωματώθηκαν στις κεφαλίδες (headers) των HTTP αιτημάτων του script, επιτρέποντας την απρόσκοπτη πρόσβαση στα endpoints χωρίς τη διαχείριση sessions ή cookies.

### 5.3.2 Αρχιτεκτονική και Λογική Κώδικα

Η λογική του script ακολουθεί μια αυστηρά καθορισμένη ροή, η οποία διασφαλίζει την ακεραιότητα των συλλεγόμενων δεδομένων.



Εικόνα 5.4: Αρχιτεκτονική Python script

Το διάγραμμα ροής απεικονίζει τον αυτοματοποιημένο κύκλο συλλογής τηλεμετρίας. Η αρχιτεκτονική αυτή διασφαλίζει ότι η παρακολούθηση της υποδομής εκτελείται χωρίς ανθρώπινη παρέμβαση, επιτρέποντας την ιστορική καταγραφή της απόδοσης των κόμβων.

Λειτουργικά Στάδια:

- **Αυθεντικοποίηση:** Χρησιμοποιείται το API Token (PVEAPIToken), το οποίο επιτρέπει την ασφαλή πρόσβαση χωρίς την ανάγκη διαχείρισης sessions.
- **Αίτημα Δεδομένων:** Μέσω της βιβλιοθήκης requests, αποστέλλονται HTTPS κλήσεις στα endpoints. Έχει γίνει πρόβλεψη για την παράκαμψη των SSL προειδοποιήσεων (verify=False) λόγω της χρήσης self-signed πιστοποιητικών στον Node 1.
- **Επεξεργασία (Parsing):** Τα αποτελέσματα λαμβάνονται σε μορφή JSON, μετατρέπονται σε dictionaries και συγχωνεύονται σε ένα ενιαίο σύνολο δεδομένων.
- **Αποθήκευση (CSV):** Χρησιμοποιείται η βιβλιοθήκη csv για την εγγραφή των δεδομένων. Το script ελέγχει αν το αρχείο υπάρχει ήδη· αν όχι, δημιουργεί τις επικεφαλίδες (headers), ενώ σε κάθε επόμενη εκτέλεση προσθέτει μια νέα γραμμή (append mode) με τη σήμανση της τρέχουσας ημερομηνίας και ώρας.

## 5.4 Προγραμματισμός Εργασιών (Cron Job)

Προκειμένου η αρχιτεκτονική παρακολούθησης να καταστεί πλήρως αυτόνομη και να μην απαιτείται η χειροκίνητη παρέμβαση του διαχειριστή, υλοποιήθηκε ο χρονοπρογραμματισμός της εκτέλεσης του script μέσω της υπηρεσίας Cron του Linux. Η χρήση του Cron διασφαλίζει ότι η συλλογή των δεδομένων από τον Node 1 πραγματοποιείται σε σταθερά χρονικά διαστήματα, επιτρέποντας τη δημιουργία μιας αξιόπιστης ιστορικής βάσης δεδομένων σε μορφή CSV. Για τις ανάγκες του πειράματος, ορίστηκε η εκτέλεση του script στον Node 2 μία φορά κάθε ώρα. Η ρύθμιση πραγματοποιήθηκε μέσω του αρχείου παραμέτρων του χρήστη (user crontab) με την εντολή **crontab -e**. Η σύνταξη που χρησιμοποιήθηκε στο αρχείο ρυθμίσεων (crontab -e) είναι η ακόλουθη:

Πίνακας 6: Ανάλυση και Δομή του Αυτοματοποιημένου Cron Job

| 0 * * * * /usr/bin/python3 /home/user/automation_script.py » automation.log 2>&1 |                                                                                                                                               |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Χρονικός Προγραμματισμός</b>                                                  | 0 * * * *<br>Ορίζει την εκτέλεση στο μηδενικό λεπτό κάθε ώρας (π.χ. 10:00, 11:00, κ.ο.κ.).                                                    |
| <b>Εκτελέσιμο / Διερμηνέας</b>                                                   | /usr/bin/python3<br>Η απόλυτη διαδρομή του διερμηνέα της Python για την αποφυγή σφαλμάτων περιβάλλοντος (environment path).                   |
| <b>Καταγραφή (Logging)</b>                                                       | » automation.log 2>&1<br>Ανακατεύθυνση των αποτελεσμάτων (stdout) και των σφαλμάτων (stderr) σε αρχείο καταγραφής για έλεγχο της λειτουργίας. |

```

drosos@kats: ~
GNU nano 7.2 /tmp/crontab.fQ3zJZ/crontab *
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
0 * * * * /usr/bin/python3 /home/drosos/automation.py >> /home/drosos/automation.log 2>&1

```

Εικόνα 5.5: Παραμετροποίηση του Cron για την ωριαία συλλογή τηλεμετρίας

Με την ενεργοποίηση του Cron Job, ο Node 2 λειτουργεί πλέον ως ένας αυτόνομος Monitoring Station, ο οποίος «χτυπά» το API του Proxmox και τροφοδοτεί το αρχείο **vm\_metrics.csv** με φρέσκες μετρήσεις, ακόμη και αν ο διαχειριστής δεν είναι συνδεδεμένος στο σύστημα. Η επαλήθευση της περιοδικής εκτέλεσης πραγματοποιείται μέσω του ελέγχου των χρονικών σημάνσεων (timestamps) στο παραγόμενο αρχείο. Η παραπάνω εντολή διασφαλίζει ότι στο 0ο λεπτό κάθε ώρας, ο διερμηνέας της Python θα εκτελεί το script, τροφοδοτώντας τη βάση δεδομένων (CSV) με φρέσκα στοιχεία από τον Node 1.

```

drosos@kats:~$ crontab -e
no crontab for drosos - using an empty one

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano      <---- easiest
 2. /usr/bin/vim.tiny
 3. /bin/ed

Choose 1-3 [1]: 1
crontab: installing new crontab
drosos@kats:~$ python3 automation.py
Initiating data collection from Proxmox API...
Success! Data has been appended to vm_metrics.csv
drosos@kats:~$ cat vm_metrics.csv

```

Εικόνα 5.6: Επιβεβαίωση εκτέλεσης της διεργασίας από το αρχείο καταγραφής συστήματος.



## 5.5 Επαλήθευση και Εξαγωγή Δεδομένων

Μετά την παρέλευση ορισμένου χρονικού διαστήματος από την ενεργοποίηση του αυτοματισμού, πραγματοποιήθηκε ο τελικός έλεγχος της υποδομής για την επαλήθευση της ορθής συλλογής και αποθήκευσης των δεδομένων. Η επαλήθευση εστιάζει στην ακεραιότητα του αρχείου CSV και στην επιβεβαίωση ότι οι μετρήσεις αντικατοπτρίζουν την πραγματική κατάσταση του VM στον Node 1.

### 5.5.1 Δομή και Περιεχόμενο Αρχείου CSV

Το αρχείο `vm_metrics.csv` αποτελεί το τελικό προϊόν της διαδικασίας. Όπως φαίνεται στην παρακάτω εικόνα, το script έχει οργανώσει τα δεδομένα σε στήλες, όπου οι δύο πρώτες αφορούν τη χρονική σήμανση (Timestamp) και οι επόμενες τις τεχνικές μετρικές.

| Date     | Time     | name         | diskwrite | agent  | running-qemu | pressurecpufull |
|----------|----------|--------------|-----------|--------|--------------|-----------------|
| 14-02-26 | 19:46:14 | Node1-Ubuntu | 17736704  | 1      | 10.1.2       | 0               |
| 14-02-26 | 19:57:18 | 0            | 1         | 10.1.2 | 18678784     | Node1-Ubuntu    |
| 14-02-26 | 20:00:03 | 19198976     | 10.1.2    | 1      | 0            | Node1-Ubuntu    |

Εικόνα 5.7: Αρχείο `vm_metrics`

### 5.5.2 Επιβεβαίωση Περιοδικότητας

Η επιτυχία του χρονοπρογραμματισμού (Cron Job) αποδεικνύεται από τη διαδοχή των εγγραφών στο αρχείο.

```
buntu-24.04.3-live-server-amd64.i
:vm-100-disk-0,iotread=1,size=30
2026-02-24,18:00:10,"{'tap100i0':
-fleecing': True, 'query-bitmap-i
599)'}",0,"{'ide2': {'idle_time_n
fset': 0, 'failed_zone_append_ope
ush_operations': 0, 'invalid_flush
r_operations': 0, 'invalid_wr_ope
d': 0, 'wr_merged': 0, 'timed_sta
operations': 5311, 'wr_bytes': 14
_operations': 0, 'failed_unmap_op
666432, 'unmap_operations': 4339,
ne_append_operations': 0, 'wr_tot
ne_append_merged': 0, 'zone_appen
,0,635124322,"{'free_mem': 424660
_faults': 1488511, 'last_update':
6bb2d-24f5-4910-a09d-285bcd1d29dec
105e5979d44129e81e6b4eaf44a30,l26,
2026-02-24,19:00:04,running,88768
```

Εικόνα 5.8: Επαλήθευση εγγραφών στο `vm_metrics.csv` με χρήση της εντολής `tail`.

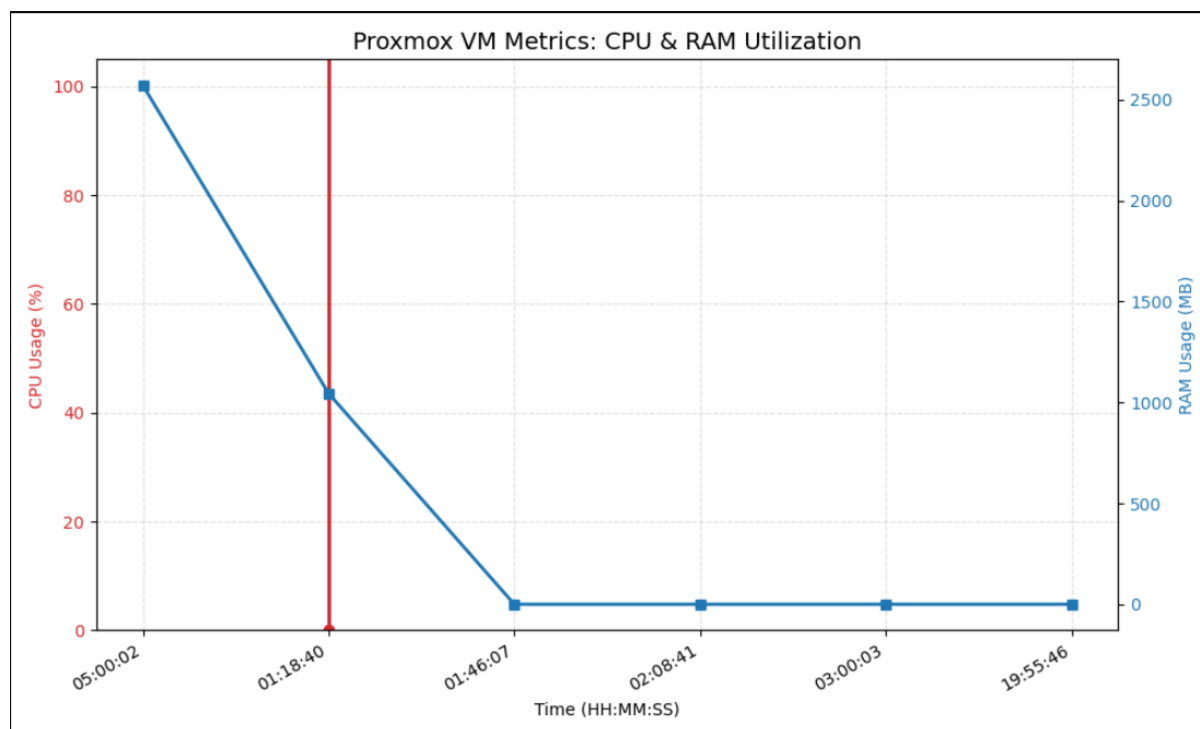
Όπως παρατηρείται, οι μετρήσεις έχουν ληφθεί με ακρίβεια ανά μία ώρα, γεγονός που αποδεικνύει ότι ο Node 2 λειτουργεί πλέον ως ένας αξιόπιστος σταθμός ελέγχου (Management Station) για την υβριδική υποδομή. Η αυτοματοποιημένη αυτή ροή μειώνει το διαχειριστικό φορτίο και εξασφαλίζει την ύπαρξη ιστορικών δεδομένων για την υποστήριξη αποφάσεων (Capacity Planning).

## 5.6 Οπτικοποίηση Δεδομένων Τηλεμετρίας

Για την ολοκληρωμένη εποπτεία της υποδομής, αναπτύχθηκε ένα συμπληρωματικό Python script οπτικοποίησης, το οποίο επεξεργάζεται αυτόματα το αρχείο `vm_metrics.csv`. Το script χρησιμοποιεί τη βιβλιοθήκη `Pandas` για την οργάνωση των δεδομένων και τη `Matplotlib` για τη δημιουργία του γραφήματος.

Η διαδικασία ακολουθεί τα εξής βήματα:

1. **Parsing:** Ανάγνωση των ιστορικών δεδομένων από το CSV.
2. **Normalization:** Μετατροπή των timestamps και των τιμών (CPU
3. **Rendering:** Παραγωγή του τελικού γραφήματος που απεικονίζει τη συμπεριφορά του VM 100.



Εικόνα 5.9: Γράφημα διακύμανσης CPU και RAM Utilization του VM 100.

Το γράφημα (Εικόνα 5.9) αναδεικνύει τη χρησιμότητα της διασύνδεσης: ο διαχειριστής μπορεί πλέον να παρακολουθεί την υγεία των απομακρυσμένων εικονικών μηχανών μέσω του ασφαλούς Layer 2 tunnel, διευκολύνοντας τη διάγνωση προβλημάτων και τον σχεδιασμό πόρων (Capacity Planning).

## 6 Κεφάλαιο 6: Προβλήματα & Αντιμετώπιση

Η υλοποίηση μιας υβριδικής υποδομής Layer 2 VPN ενέχει σημαντικές τεχνικές προκλήσεις, καθώς απαιτεί τον συντονισμό διαφορετικών περιβαλλόντων (Azure Cloud, Proxmox Hypervisor και Τοπικό Δίκτυο). Στην ενότητα αυτή παρατίθενται τα κυριότερα προβλήματα που αναδύθηκαν και οι λύσεις που δόθηκαν.

### 6.1 Ρυθμίσεις Ασφαλείας και Firewalls (Azure & Local)

Κατά την αρχική φάση της διασύνδεσης, παρατηρήθηκε αδυναμία εγκαθίδρυσης του tunnel μεταξύ του Node 2 και του Node 0 (Azure), παρόλο που η υπηρεσία OpenVPN λειτουργούσε κανονικά. **Αντιμετώπιση:** Μετά από έλεγχο, διαπιστώθηκε ότι το *Network Security Group (NSG)* του Azure μπλόκαρε την εισερχόμενη κίνηση στη θύρα UDP 1194. Χρειάστηκε η προσθήκη ειδικού κανόνα ασφαλείας (*Inbound Port Rule*) στην κονσόλα διαχείρισης του Azure. Παράλληλα, ρυθμίστηκε το τοπικό firewall (ufw) στον Node 0 ώστε να επιτρέπει την κίνηση προς το tun/tap interface.

### 6.2 Πολυπλοκότητα Διαχείρισης Πιστοποιητικών (PKI)

Η διαδικασία έκδοσης των πιστοποιητικών μέσω του *Easy-RSA* αποτέλεσε σημείο δυσκολίας, καθώς η παραμικρή αστοχία στην ονοματολογία ή στη μεταφορά των αρχείων (.crt, .key) οδηγούσε σε αποτυχία του TLS Handshake. **Αντιμετώπιση:** Πραγματοποιήθηκε καθαρισμός της υποδομής PKI (clean-all) και εκ νέου έκδοση των πιστοποιητικών με αυστηρή οργάνωση των φακέλων. Δόθηκε ιδιαίτερη προσοχή στη μεταφορά μόνο των απαραίτητων αρχείων σε κάθε κόμβο (Client keys) και στη διατήρηση του ca.key αποκλειστικά στον Server, διασφαλίζοντας την ακεραιότητα της αυθεντικοποίησης.

### 6.3 Σειρά Εκκίνησης και Συγχρονισμός Interfaces

Ένα κρίσιμο τεχνικό ζήτημα ήταν η εξάρτηση των υπηρεσιών από την κατάσταση των interfaces. Αν ο DHCP Server στον Node 2 εκκινούσε πριν τη δημιουργία της γέφυρας br0 ή πριν το OpenVPN "σηκώσει" τη διεπαφή tap0, η υπηρεσία αποτύγχανε να ξεκινήσει (*Failing to bind to interface*). **Αντιμετώπιση:** Το πρόβλημα λύθηκε μέσω του αυτοματισμού με Bash Scripts. Εισήχθησαν εντολές ελέγχου και χρονικές καθυστερήσεις (sleep), οι οποίες διασφαλίζουν ότι η γέφυρα είναι ενεργή και το tunnel λειτουργικό πριν την ενεργοποίηση του DHCP server.

### 6.4 Διαχείριση του Layer 2 Bridging

Η υλοποίηση του Bridging απαιτούσε τη χειροκίνητη αντιστοίχιση των tap διεπαφών στις γέφυρες br0 σε κάθε κόμβο. Η λανθασμένη σειρά εντολών συχνά οδηγούσε σε απώλεια της τοπικής συνδεσιμότητας. **Αντιμετώπιση:** Αναπτύχθηκε μια σταθερή αλληλουχία εντολών στα scripts (ip link set, brctl addif), η οποία εγγυάται ότι οι διεπαφές προστίθενται στη γέφυρα με τη σωστή σειρά, διατηρώντας παράλληλα την IP επικοινωνία του κάθε κόμβου με το διαδίκτυο.

## Συμπεράσματα και Μελλοντικές επεκτάσεις

**Τεχνικά και Γνωστικά Συμπεράσματα:** Η παρούσα εργασία απέδειξε ότι η γεφύρωση ετερογενών πόρων μεταξύ δημόσιου νέφους (Azure) και ιδιωτικής υποδομής (Proxmox) είναι εφικτή και εξαιρετικά αποδοτική μέσω της τεχνολογίας Layer 2 VPN. Η πολυπλοκότητα ενός υβριδικού νέφους δεν επιλύεται αποκλειστικά με ισχυρό υλικό, αλλά με τον σωστό σχεδιασμό του δικτυακού επιπέδου. Η υλοποίηση του OpenVPN σε TAP mode κατάφερε να «εξαφανίσει» τις γεωγραφικές αποστάσεις, επιτρέποντας σε έναν server στο Azure να επικοινωνεί σε επίπεδο Ethernet frames με έναν τοπικό hypervisor, σαν να βρίσκονταν στο ίδιο φυσικό switch.

**Επιχειρηματική Αξιοποίηση (Business Use Cases):** Η αρχιτεκτονική που αναπτύχθηκε προσφέρει άμεσες λύσεις σε σύγχρονες επιχειρηματικές ανάγκες:

- **Legacy Application Support:** Πολλές παλαιότερες εφαρμογές απαιτούν Layer 2 συνδεσιμότητα (broadcast/multicast) για να λειτουργήσουν. Η λύση μας επιτρέπει τη μεταφορά τους στο Cloud χωρίς αλλαγές στον κώδικά τους.
- **Disaster Recovery (DR):** Δημιουργήθηκε μια υποδομή όπου οι IPs παραμένουν σταθερές. Σε περίπτωση αστοχίας του τοπικού κόμβου, η επιχείρηση μπορεί να ανακτήσει τις υπηρεσίες της στο Azure χωρίς ανασχεδιασμό του δικτύου.
- **Cloud Bursting:** Δίνεται η δυνατότητα διατήρησης κρίσιμων δεδομένων τοπικά για λόγους ασφάλειας (GDPR/Compliance), ενώ η υπολογιστική ισχύς του Azure χρησιμοποιείται δυναμικά μόνο όταν υπάρχει ανάγκη.

**Μελλοντικές Επεκτάσεις και Εκσυγχρονισμός:** Παρά την επιτυχή επίτευξη των στόχων, η υποδομή μπορεί να εξελιχθεί περαιτέρω με την ενσωμάτωση σύγχρονων εργαλείων DevOps:

- **Infrastructure as Code (IaC) με Terraform:** Η χρήση του Terraform θα επέτρεπε την αυτόματη δημιουργία των εικονικών μηχανών στο Azure και των δικτυακών κανόνων (NSGs) μέσα σε ελάχιστα λεπτά, εξαλείφοντας το σφάλμα της χειροκίνητης παραμετροποίησης.
- **Configuration Management με Ansible:** Ενώ τα Bash scripts αυτοματοποιήσαν την εκκίνηση, το Ansible θα μπορούσε να διαχειριστεί τις ρυθμίσεις (OpenVPN configs, certificates, DHCP scopes) σε εκατοντάδες κόμβους ταυτόχρονα, εξασφαλίζοντας ότι όλοι οι clients είναι συγχρονισμένοι.
- **Real-time Monitoring με Grafana & Prometheus:** Το Python script που αναπτύχθηκε αποτελεί τη βάση για τη συλλογή δεδομένων. Η ενσωμάτωση μιας βάσης χρονοσειρών (Prometheus) και ενός dashboard στο Grafana θα προσέφερε οπτικοποίηση της υγείας του δικτύου και της κατανάλωσης πόρων σε πραγματικό χρόνο με δυνατότητα ειδοποιήσεων (alerts).

## Παράρτημα Α: Github repository

Για την υλοποίησης και τη διευκόλυνση της αναπαραγωγής των αποτελεσμάτων, το σύνολο των αρχείων ρυθμίσεων (configuration files), των Bash scripts αυτοματοποίησης και του κώδικα Python για τη συλλογή τηλεμετρίας, έχει αναρτηθεί στο προσωπικό μου repository στο GitHub.

Στο repository περιλαμβάνονται:

- Τα `serverconf`, `proxmox_vpnconf` και `clientconf`.
- Τα scripts για τη εκτέλεση στα Nodes.
- Τα Python scripts για την επικοινωνία με το Proxmox API, και το plot.
- Οδηγίες εγκατάστασης (README.md).

**Σύνδεσμος Αποθετηρίου:**

<https://github.com/drososkats/cloud-infrastructure-L2-vpn>



## Βιβλιογραφικές Αναφορές

- [1] Harokopio University of Athens. *Course Materials: Distributed Systems (DIT292)*. <https://eclass.hua.gr/modules/document/?course=DIT292>. 2024.
- [2] OpenVPN Community. *Ethernet Bridging with OpenVPN (TAP interfaces)*. <https://openvpn.net/community-resources/ethernet-bridging/>. 2024.
- [3] Proxmox Server Solutions. *Proxmox VE API Documentation and Reference*. [https://pve.proxmox.com/wiki/Proxmox\\_VE\\_API](https://pve.proxmox.com/wiki/Proxmox_VE_API). 2024.
- [4] Microsoft Azure. *Azure Virtual Network documentation*. <https://learn.microsoft.com/en-us/azure/virtual-network/>. 2024.
- [5] Kenneth Reitz. *Requests: HTTP for Humans (Python library)*. <https://requests.readthedocs.io/en/latest/>. 2024.
- [6] Kernel.org. *Linux Bridge Management and Documentation*. <https://wiki.linuxfoundation.org/networking/bridge>. 2024.
- [7] The Matplotlib Development Team. *Matplotlib: Visualization with Python*. <https://matplotlib.org/stable/contents.html>. 2024.
- [8] Internet Systems Consortium (ISC). *ISC DHCP Server Manual and Configuration Guide*. <https://www.isc.org/dhcp/>. 2024.
- [9] The Pandas Development Team. *pandas: powerful Python data analysis toolkit*. <https://pandas.pydata.org/docs/>. 2024.
- [10] Linux Foundation. *Scheduling Tasks with Cron and Crontab*. <https://man7.org/linux/man-pages/man8/cron.8.html>. 2024.
- [11] Canonical Ltd. *Ubuntu Server Guide - Network Configuration*. <https://ubuntu.com/server/docs/network-configuration>. 2024.
- [12] ECMA International. *The JSON Data Interchange Syntax*. <https://www.json.org/json-en.html>. 2024.
- [13] OpenSSH. *OpenSSH manual: Secure Shell and Tunneling*. <https://www.openssh.com/manual.html>. 2024.